

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC KINH TẾ - KỸ THUẬT CÔNG NGHIỆP



KHÓA LUẬN TỐT NGHIỆP

TÊN ĐỀ TÀI KHÓA LUẬN:

**NGHIÊN CỨU VÀ XÂY DỰNG HỆ THỐNG PHÁT HIỆN, NGĂN
CHẶN TẤN CÔNG DDOS TRONG MẠNG ĐỊNH NGHĨA BẰNG
PHẦN MỀM (SDN) SỬ DỤNG KỸ THUẬT HỌC SÂU
(DEEP LEARNING).**

Ngành đào tạo: Công nghệ thông tin

Mã ngành đào tạo: 7480201DKK

Họ và tên sinh viên: Nguyễn Khắc Tân

Nguyễn Đức Huy

Giảng viên hướng dẫn: Ths. Trương Hồng Tiệp

HÀ NỘI, 2026

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC KINH TẾ - KỸ THUẬT CÔNG NGHIỆP



KHÓA LUẬN TỐT NGHIỆP

TÊN ĐỀ TÀI KHÓA LUẬN:

**NGHIÊN CỨU VÀ XÂY DỰNG HỆ THỐNG PHÁT HIỆN, NGĂN
CHẶN TẤN CÔNG DDOS TRONG MẠNG ĐỊNH NGHĨA BẰNG
PHẦN MỀM (SDN) SỬ DỤNG KỸ THUẬT HỌC SÂU
(DEEP LEARNING).**

Ngành đào tạo: Công nghệ thông tin

Mã ngành đào tạo: 7480201DKK

Họ và tên sinh viên: Nguyễn Khắc Tân

Nguyễn Đức Huy

Giảng viên hướng dẫn: Ths. Trương Hồng Tiệp

HÀ NỘI, 2026

LỜI CAM ĐOAN

Chúng em xin cam đoan đề tài: “Nghiên cứu và xây dựng hệ thống phát hiện, ngăn chặn tấn công DDos trong mạng định nghĩa bằng phần mềm (SDN) sử dụng kỹ thuật học sâu (Deep learning)” là công trình nghiên cứu do nhóm chúng em thực hiện dưới sự hướng dẫn của Ths. Trương Hồng Tiệp.

Các số liệu, kết quả thực nghiệm thu được trong khóa luận là hoàn toàn trung thực, khách quan và không sao chép từ bất kỳ công trình nghiên cứu nào khác. Những phần trích dẫn, sử dụng tài liệu tham khảo đều đã được liệt kê và chú thích nguồn gốc rõ ràng theo đúng quy định.

Chúng em xin hoàn toàn chịu trách nhiệm trước Hội đồng khoa học và Nhà trường về tính chính xác cũng như tính bản quyền của lời cam đoan này.

Hà Nội, ngày ,...tháng,...năm 2026

Sinh Viên

Nguyễn Đức Huy, Nguyễn Khắc Tân

LỜI CẢM ƠN

Lời đầu tiên, nhóm chúng em xin chân thành cảm ơn Ban giám hiệu trường đại học Kinh Tế Kỹ Thuật Công Nghiệp đã tạo điều kiện tốt nhất về cơ sở vật chất và môi trường học tập cho chúng em trong suốt những năm qua.

Tiếp đến là các thầy cô trong nhà trường, người đã trực tiếp đặt nền móng kiến thức cho sinh viên, đặc biệt chúng em xin bày tỏ lòng biết ơn thầy Ths. Trương Hồng Tiệp, người đã vẽ ra cái khung cốt lõi nhất của đề tài, định hướng và giúp chúng em hoàn thành khóa luận đúng tiến độ.

Cuối cùng, chúng em xin gửi lời cảm ơn đến gia đình và bạn bè - những người đã luôn ủng hộ, động viên và hỗ trợ nhóm về mọi mặt trong suốt quá trình học tập và làm khóa luận tốt nghiệp này.

Do kiến thức còn hạn chế, khóa luận khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp quý báu từ các thầy, cô trong hội đồng.

Chúng em xin chân thành cảm ơn!

Hà Nội, ngày ,...tháng, ...năm 2026

Sinh Viên

Nguyễn Đức Huy, Nguyễn Khắc Tân

MỤC LỤC

PHẦN MỞ ĐẦU	1
CHƯƠNG 1: TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT ĐỀ TÀI.....	4
1.1. Cơ sở lý luận về Mạng định nghĩa bằng phần mềm (SDN).....	4
1.1.1. Khái niệm và Kiến trúc SDN	4
1.1.2. Giao thức OpenFlow	4
1.1.3. Các yếu tố ảnh hưởng đến hiệu năng mạng SDN	4
1.2. Tổng quan về Tấn công Từ chối Dịch vụ (DDoS).....	4
1.2.1. Định nghĩa và Phân loại	4
1.2.2. Tác động của DDoS đối với SDN.....	7
1.3. Cơ sở lý thuyết về Học sâu (Deep Learning).....	7
1.3.1. Từ Machine Learning đến Deep Learning	7
1.3.2. Mô hình Mạng nơ-ron (CNN/GRU)	8
1.4. Giả thuyết khoa học và Phương pháp nghiên cứu	9
1.4.1. Giả thuyết khoa học	9
1.4.2. Phương pháp đánh giá mô hình (Evaluation Metrics)	9
CHƯƠNG 2: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG MÔ HÌNH HỆ THỐNG.....	12
2.1. Phân tích đánh giá các hướng nghiên cứu liên quan đến đề tài	12
2.1.1. Các nghiên cứu trên thế giới	12
2.1.2. Các nghiên cứu trong nước	12
2.1.3. Khoảng trống nghiên cứu (Research Gap).....	12
2.2. Thực trạng có liên quan đến KLTN	12
2.2.1. Hoạt động quản lý và vận hành mạng SDN.....	12
2.2.2. Hoạt động giám sát và bảo mật hiện hành	13
2.3. Đánh giá thực trạng.....	13
2.3.1. Những kết quả đạt được (Ưu điểm của mô hình hiện tại)	13
2.3.2. Những hạn chế và Nguyên nhân	13

2.4. Đề xuất giải pháp để giải quyết vấn đề	13
2.4.1. Kiến trúc giải pháp tổng thể.....	14
2.4.2. Giải pháp cải thiện các hạn chế.....	15
2.4.3. Thiết kế mô hình hệ thống (System Design)	16
CHƯƠNG 3: CÀI ĐẶT, THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	31
3.1. Cài đặt mô hình hệ thống	31
3.1.1. Chuẩn bị môi trường thực nghiệm	31
3.1.2. Quy trình Huấn luyện Mô hình (Giai đoạn Offline)	35
3.1.3. Tích hợp hệ thống (Giai đoạn Online)	51
3.2. Thực nghiệm và Đánh giá kết quả nghiên cứu	59
3.2.1. Kịch bản thực nghiệm	59
3.2.2. Đánh giá hiệu năng mô hình AI (Model Performance)	63
3.2.3. Đánh giá hiệu năng mạng SDN (Network Performance)	64
3.2.4. Tổng kết các kết quả đạt được	65
3.2.5. Những hạn chế và Đề xuất hướng phát triển	67
KẾT LUẬN CHUNG	69
KHUYẾN NGHỊ	72
TÀI LIỆU THAM KHẢO.....	73

CÁC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Giải nghĩa tiếng Việt
DDoS	Distributed Denial of Service	Tấn công từ chối dịch vụ phân tán: Gây nghẽn mạng bằng lưu lượng giả.
SDN	Software Defined Networking	Mạng điều khiển bằng phần mềm: Tách biệt phần điều khiển khỏi phần cứng.
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập: Chuyên trích xuất đặc trưng không gian
GRU	Gated Recurrent Unit	Đơn vị cổng lặp: Chuyên xử lý dữ liệu chuỗi và thời gian
DL	Deep Learning	Học sâu: Nhánh nâng cao của AI sử dụng mạng nơ-ron nhiều lớp.
ML	Machine Learning	Học máy: Tập hợp các thuật toán giúp máy tính tự học từ dữ liệu.
AI	Artificial Intelligence	Trí tuệ nhân tạo: Mô phỏng trí thông minh của con người trên máy tính.
TCP	Transmission Control Protocol	Giao thức điều khiển truyền vận: Đảm bảo truyền dữ liệu tin cậy, chính xác.
IP	Internet Protocol	Giao thức liên mạng: Định danh địa chỉ và định tuyến dữ liệu trên mạng.
CPU	Central Processing Unit	Bộ xử lý trung tâm: "Não bộ" thực hiện mọi tính toán của hệ thống.
ICMP	Internet Control Message Protocol	Giao thức điều khiển tin nhắn: Dùng để thông báo lỗi và kiểm tra

DANH MỤC CÁC BẢNG

Bảng 1.1. Tấn công thuộc nhóm bẻ gãy thông	5
Bảng 1.2. Tấn công thuộc nhóm cạn kiệt trạng thái	6
Bảng 1.3. Các loại tấn công thuộc nhóm L7	6
Bảng 1.4. Bảng so sánh giữa Machine Learning và Deep Learning	8
Bảng 1.5. Bảng phương pháp đánh giá mô hình.....	10
Bảng 2.1. Bảng so sánh tổng hợp đặc điểm các loại tấn công.....	14
Bảng 2.2. Bảng phân vùng hệ thống mạng	19
Bảng 2.3. Ý nghĩa trong phát hiện tấn công dựa vào Entropy.....	21
Bảng 2.4. Ý nghĩa dựa vào Packet rate và Byte rate.....	23
Bảng 2.5. Ý nghĩa của ngưỡng thích nghi động	29
Bảng 3.1. Các thông số môi trường thực tế	32
Bảng 3.2. Các hàm sử dụng để chuẩn hóa dữ liệu.....	37
Bảng 3.3. Bảng tổng hợp quá trình nâng cấp dataset.....	39
Bảng 3.4. Các đặc trưng có trong dữ liệu mẫu	42
Bảng 3.5. Tóm tắt quá trình kiến trúc AI	44
Bảng 3.6. Bài học rút ra từ việc cải tiến mô hình	45
Bảng 3.7. Các file sinh ra sau khi huấn luyện mô hình	51
Bảng 3.8. Bảng tổng kết thực nghiệm.....	65

DANH MỤC CÁC HÌNH

Hình 1.1. Mô hình mạng nơ-ron	9
Hình 2.1. Biểu đồ hiển thị lưu lượng mạng bình thường.....	15
Hình 2.2. Sơ đồ kiến trúc SDN	16
Hình 2.3. Hệ thống mạng L2 ban đầu.....	17
Hình 2.4. Mô hình thiết kế mạng (liên thông & ổn định).....	19
Hình 2.5. Mô hình Web https được xây dựng với các dịch vụ sử dụng Db1..	20
Hình 2.6. Logic tính Shannon Entropy cho cổng.....	22
Hình 2.7. Cơ chế vi phân tạo ra kết quả Loss giá trị âm khi train lớp autoencoder	23
Hình 3.1. Cài song song Ubuntu trên Window	31
Hình 3.2. Sơ đồ mạng L2 ban đầu	33
Hình 3.3. Cấu hình mô hình mạng.....	34
Hình 3.4. Sự sụp đổ liên thông mạng V3-thiếu Reactive Forwarding.....	34
Hình 3.5. Kết quả liên thông của mô hình mạng	35
Hình 3.6. Ánh xạ các thuộc tính dữ liệu tổng hợp.....	36
Hình 3.7. Đánh giá dataset V1	37
Hình 3.8. Thiết lập dữ liệu chuẩn.....	38
Hình 3.9. Quá trình thu thập dataset V7.....	38
Hình 3.10. Đánh giá dataset V4	39
Hình 3.11. Đánh giá dataset V7	40
Hình 3.12. Kết quả train AI V4 – quá nhạy với zero day	46
Hình 3.13. Kết quả train mô hình đạt độ chính xác cao trên tập test.....	48
Hình 3.14. Ma trận nhầm lẫn của AI v3 dùng Dataset v4.....	49
Hình 3.15. Nạp tri thức cho AI.....	52
Hình 3.16. Tiến trình thu dữ liệu đầu vào cho mô hình AI xử lý.....	53
Hình 3.17. Logic khởi tạo luồng dữ liệu thời gian thực và bộ lọc IP/IPv6	54
Hình 3.18. Phát hiện, ngăn chặn và giải thích tấn công HTTP thành công	55
Hình 3.19. Các lớp khiên kiểm tra lưu lượng	56
Hình 3.20. Cơ chế 2 khiên và ngưỡng động giúp bảo vệ người dùng	57

Hình 3.21. Mô hình mạng khi hoạt động bình thường	59
Hình 3.22. Biểu đồ thể hiện lưu lượng mạng bị tấn công.....	60
Hình 3.23. Biểu đồ tấn công tầng ứng dụng (HTTP).....	61
Hình 3.24. Sơ đồ mạng khi bị tấn công SYN Flood	61
Hình 3.25. Phát hiện, ngăn chặn và giải thích UDP_flood thành công	62
Hình 3.26. Phát hiện, ngăn chặn và giải thích SYN_flood thành công	62
Hình 3.27. Đánh giá hiệu năng mô hình thông qua ma trận nhầm lẫn	63
Hình 3.28. Phát hiện, ngăn chặn và giải thích tấn công Slowloris thành công	63

PHẦN MỞ ĐẦU

Tên đề tài: Nghiên cứu và xây dựng hệ thống phát hiện, ngăn chặn tấn công ddos trong mạng định nghĩa bằng phần mềm (SDN) sử dụng kỹ thuật học sâu (Deep learning).

1. Lý do lựa chọn đề tài

Trong kỷ nguyên chuyển đổi số, kiến trúc mạng truyền thống đang dần bộc lộ những hạn chế về khả năng mở rộng và tính linh hoạt trong quản lý. Mạng định nghĩa bằng phần mềm (**SDN - Software Defined Networking**) ra đời như một giải pháp đột phá, cho phép tách biệt mặt phẳng điều khiển (Control Plane) khỏi mặt phẳng dữ liệu (Data Plane), giúp việc quản trị mạng trở nên tập trung và có khả năng lập trình hóa.

Tuy nhiên, chính đặc điểm quản lý tập trung này lại biến bộ điều khiển (Controller) thành "điểm nghẽn duy nhất" (Single Point of Failure). Các cuộc tấn công Từ chối dịch vụ phân tán (DDoS) nhắm vào cơ sở hạ tầng SDN có thể làm tê liệt toàn bộ hệ thống mạng chỉ trong thời gian ngắn thông qua việc làm tràn bảng lưu lượng (Flow Table) hoặc gây quá tải tài nguyên xử lý của Controller. Cùng với đó, **tấn công DDoS tốc độ thấp (Low-rate)** lại mang mức độ nguy hiểm ngấm đáng lo ngại hơn rất nhiều. Bằng cách gửi lượng nhỏ gói tin và tiêu thụ ít băng thông, tấn công Low-rate khéo léo mô phỏng hành vi của lưu lượng hợp lệ, từ đó dễ dàng "qua mặt" các hệ thống phòng thủ hiện hành để âm thầm làm cạn kiệt tài nguyên hệ thống. Việc các hệ thống hiện tại thường xuyên bỏ qua loại tấn công tinh vi này đã tạo ra một lỗ hổng bảo mật nghiêm trọng. Do đó, việc nghiên cứu ứng dụng Học sâu (Deep Learning) nhằm phân tích, trích xuất các đặc trưng tiềm ẩn để phát hiện và ngăn chặn toàn diện cả hai hình thức tấn công High-rate và Low-rate là yêu cầu vô cùng cấp thiết và mang giá trị thực tiễn cao.

Xuất phát từ thực tế đó, việc ứng dụng **Học sâu (Deep Learning)** – với khả năng tự học các đặc trưng phi tuyến tính phức tạp của lưu lượng mạng – vào an ninh mạng đang là hướng đi cấp thiết. Do đó, chúng em lựa chọn đề tài: **"NGHIÊN CỨU VÀ XÂY DỰNG HỆ THỐNG PHÁT HIỆN, NGĂN CHẶN TẤN CÔNG DDOS TRONG MẠNG SDN SỬ DỤNG KỸ THUẬT HỌC SÂU"**. Đề tài này giải quyết bài toán thực tiễn về an ninh mạng hiện đại, hoàn toàn phù hợp với chuyên ngành đào tạo về Hệ thống thông tin, An toàn thông tin và Trí tuệ nhân tạo.

2. Mục tiêu và Nhiệm vụ nghiên cứu

2.1. Mục tiêu:

- **Mục tiêu tổng quát:** Xây dựng thành công hệ thống giám sát an ninh mạng trên nền tảng SDN, tích hợp mô hình Deep Learning để tự động phát hiện và ngăn chặn các cuộc tấn công DDoS trong thời gian thực.
- **Mục tiêu cụ thể:**
 - Xây dựng được mô hình Deep Learning đạt độ chính xác cao (trên 90%) trong việc phân loại lưu lượng bình thường và tấn công.
 - Tích hợp thành công mô hình vào bộ điều khiển Onos để ra quyết định xử lý gói tin tự động.
 - Giảm thiểu thời gian phát hiện và độ trễ của mạng khi xảy ra tấn công.

2.2. Nhiệm vụ:

Để đạt được các mục tiêu trên, khóa luận tập trung thực hiện các nhiệm vụ cụ thể sau:

1. **Nghiên cứu lý thuyết:** Tổng quan về kiến trúc SDN, giao thức OpenFlow, các kỹ thuật tấn công DDoS và cơ sở lý thuyết về Deep Learning.
2. **Thu thập và Xử lý dữ liệu:** Thu thập các bộ dữ liệu liên quan đến tấn công Ddos mới, xử lý, lựa chọn các thuộc tính cần thiết.
3. **Huấn luyện mô hình:** Thiết kế, cài đặt và huấn luyện các thuật toán Deep Learning bằng ngôn ngữ Python; đánh giá và lựa chọn mô hình tối ưu nhất.
4. **Triển khai thực nghiệm:** Xây dựng kịch bản mô phỏng mạng trên công cụ Mininet, tích hợp module phát hiện vào Controller.
5. **Đánh giá và Tối ưu:** Thực hiện tấn công giả lập, đo lường hiệu năng hệ thống và đề xuất hướng cải tiến.

3. Đối tượng và Phạm vi nghiên cứu

3.1. Đối tượng nghiên cứu:

- **Công nghệ mạng:** Kiến trúc SDN, Giao thức OpenFlow, Bộ điều khiển Onos (Onos Controller).

- **Đối tượng dữ liệu:** Các đặc trưng thống kê của lưu lượng mạng (Flow statistics) như: thời gian flow, số lượng gói tin, kích thước gói tin, cờ TCP...
- **Công nghệ AI:** Các thuật toán Học sâu (Deep Learning) dùng cho bài toán phân loại.

3.2. Phạm vi nghiên cứu:

- **Phạm vi không gian:** Hệ thống được triển khai trên môi trường mô phỏng ảo hóa sử dụng **Mininet** và **Onos Controller** chạy trên hệ điều hành Ubuntu (không triển khai trên thiết bị phần cứng vật lý).
- **Phạm vi tấn công:** Tập trung nghiên cứu và ngăn chặn các dạng tấn công DDoS làm ngập lụt (Volumetric Attacks) phổ biến ở tầng giao vận: **TCP SYN Flood, UDP Flood, HTTP Flood** và cao hơn như **Slowloris**.
- **Phạm vi dữ liệu:** Nghiên cứu trích xuất thuộc tính từ các bộ dữ liệu bao gồm cả dữ liệu thu thập được từ mô hình mạng thực tế.

4. Phương pháp thực hiện

Khóa luận sử dụng phối hợp các phương pháp nghiên cứu khoa học sau:

- **Phương pháp nghiên cứu lý thuyết:** Thu thập, phân tích, tổng hợp tài liệu từ sách chuyên khảo, các bài báo khoa học (IEEE, ACM...) để xây dựng cơ sở lý luận vững chắc.
- **Phương pháp phân tích và xử lý dữ liệu:** Sử dụng các kỹ thuật thống kê để phân tích đặc điểm của các bộ dữ liệu mạng, áp dụng các kỹ thuật tiền xử lý để chuẩn bị dữ liệu cho AI.
- **Phương pháp lập trình và mô phỏng thực nghiệm:**
 - o Sử dụng ngôn ngữ **Python** và các thư viện **Torch/Sklearn** để xây dựng mô hình AI.
 - o Sử dụng công cụ **Mininet** để giả lập topo mạng SDN.
 - o Sử dụng thư viện **Socket** để tạo lưu lượng tấn công kiểm thử.
- **Phương pháp so sánh và đánh giá:** Sử dụng các chỉ số định lượng (Accuracy, Precision, Recall, F1-Score) và Ma trận nhầm lẫn (Confusion Matrix) để đánh giá hiệu quả của giải pháp so với các nghiên cứu trước đó.

CHƯƠNG 1: TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT ĐỀ TÀI

1.1. Cơ sở lý luận về Mạng định nghĩa bằng phần mềm (SDN)

1.1.1. Khái niệm và Kiến trúc SDN

Mạng định nghĩa bằng phần mềm (SDN) là một kiến trúc mạng mới, thay đổi tư duy thiết kế mạng truyền thống bằng cách tách biệt Mặt phẳng điều khiển (Control Plane) ra khỏi Mặt phẳng dữ liệu (Data Plane).

Mặt phẳng dữ liệu (Data Plane): Gồm các thiết bị chuyển mạch (Switch) chỉ làm nhiệm vụ chuyển tiếp gói tin đơn thuần dựa trên các bảng lưu lượng (Flow Table).

Mặt phẳng điều khiển (Control Plane): Là "bộ não" của mạng, nơi Bộ điều khiển (Controller) quyết định luồng đi của gói tin.

Mặt phẳng ứng dụng (Application Plane): Nơi triển khai các ứng dụng mạng như cân bằng tải, tường lửa, và hệ thống phát hiện xâm nhập.

1.1.2. Giao thức OpenFlow

OpenFlow là tiêu chuẩn công nghệ cho giao tiếp giữa Controller và Switch. Nó cho phép Controller thao tác trực tiếp vào bảng Flow Table của Switch (thêm, sửa, xóa các luật chuyển tiếp). Đây là cơ sở kỹ thuật để đề tài thực hiện việc ngăn chặn tấn công bằng cách đẩy các luật "Drop" xuống Switch^[2].

1.1.3. Các yếu tố ảnh hưởng đến hiệu năng mạng SDN

- **Độ trễ giữa Controller và Switch:** Nếu đường truyền này bị nghẽn (do DDoS), mạng sẽ tê liệt.
- **Dung lượng bảng lưu lượng (Flow Table Capacity):** Switch có bộ nhớ giới hạn. Tấn công DDoS có thể làm tràn bảng này, khiến Switch không thể nhận luật mới.

1.2. Tổng quan về Tấn công Từ chối Dịch vụ (DDoS)

1.2.1. Định nghĩa và Phân loại

Tấn công từ chối dịch vụ (DDoS - Denial of Service) là hành vi nhằm làm sập một máy chủ hoặc hệ thống mạng bằng cách gửi ồ ạt lưu lượng truy cập (traffic) hoặc các thông tin kích hoạt sự cố, khiến người dùng hợp pháp không thể truy cập được dịch vụ. Từ chối dịch vụ phân tán là hình thức phát triển cao hơn, nhằm làm sập dịch vụ trực tuyến bằng cách làm tràn ngập lưu lượng truy cập từ rất nhiều nguồn khác nhau.

Để thực hiện DDoS, kẻ tấn công lây nhiễm phần mềm độc hại vào các thiết bị kết nối Internet để tạo thành một mạng lưới máy tính ma (botnet hay zombie). Từ xa, hacker sẽ ra lệnh cho toàn bộ các thiết bị trong mạng botnet này đồng loạt gửi yêu cầu đến địa chỉ IP mục tiêu, khiến máy chủ bị quá tải và không thể xử lý các lưu lượng bình thường. Việc tấn công từ nhiều vị trí khiến DDoS diễn ra nhanh hơn, lưu lượng lớn hơn và rất khó bị theo dõi hay ngăn chặn so với DDoS.

Các cuộc tấn công DDoS ngày càng mạnh mẽ và tinh vi, được chia thành ba nhóm cơ bản:

- a. Nhóm Tấn công Băng thông (Volumetric DDoS): Dùng sức mạnh cơ bắp. Bơm một lượng dữ liệu khổng lồ (tính bằng Gbps hoặc Tbps) làm tắc nghẽn hoàn toàn đường truyền mạng (Switch/Router) trước khi dữ liệu kịp đến đích.

Tên cuộc tấn công	Giao thức	Cơ chế chính	Tác động hệ thống
UDP Flood	UDP	Gửi lượng lớn gói tin UDP đến các cổng ngẫu nhiên.	Vất kiệt băng thông và tài nguyên xử lý gói ICMP lỗi.
ICMP Flood	ICMP	Gửi liên tục các yêu cầu Echo Request (Ping).	Gây nghẽn đường truyền do máy chủ phải phản hồi Echo Reply.
DNS Amplification	UDP (DNS)	Giả mạo IP mục tiêu, yêu cầu DNS server trả về bản ghi lớn.	Khuếch đại lưu lượng tấn công gấp nhiều lần sức mạnh kẻ tấn công.

Bảng 1.1. Tấn công thuộc nhóm băng thông

- b. Nhóm Tấn công Cạn kiệt Trạng thái (Protocol/State-Exhaustion DDoS): Không cần đường truyền lớn, nhóm này nhắm vào việc vắt kiệt "bảng trạng thái" (State Table) của các thiết bị mạng (Tường lửa, Load Balancer, Switch) hoặc hệ điều hành của máy chủ.

Tên cuộc tấn công	Thành phần mục tiêu	Cơ chế chính	Hậu quả
TCP SYN Flood	TCP Stack (Bắt tay 3 bước)	Gửi gói SYN nhưng không bao giờ phản hồi ACK cuối cùng.	Tràn bảng kết nối (Half-open), chặn người dùng hợp lệ.
Ping of Death	IP Stack / Kernel	Gửi gói tin ICMP vượt kích thước tối đa (65,535 bytes).	Gây lỗi tràn bộ đệm (Buffer Overflow), làm sập hệ điều hành.
Smurf Attack	ICMP / Broadcast	Gửi gói tin ICMP Echo đến địa chỉ Broadcast với IP giả mạo.	Tạo ra cơn bão phản hồi ICMP đổ dồn về phía nạn nhân.

Bảng 1.2. Tấn công thuộc nhóm cạn kiệt trạng thái

- c. Nhóm Tấn công Tầng Ứng dụng (L7 Application DDoS): Đóng giả làm người dùng hợp lệ. Các gói tin đi qua lớp L3/L4 một cách hoàn hảo, bắt tay TCP đầy đủ. Chúng nhắm vào việc tiêu diệt các luồng xử lý, CPU và RAM của ứng dụng web.

Tên cuộc tấn công	Đối tượng nhắm tới	Cơ chế chính	Đặc điểm nhận dạng
HTTP Flood	Web Server (Nginx, Apache)	Đóng giả người dùng thực gửi liên tục các yêu cầu GET/POST.	Trông giống lưu lượng hợp lệ, vượt qua được lớp L3/L4.
Slowloris	Kết nối HTTP	Mở nhiều kết nối và giữ chúng mở lâu nhất có thể bằng cách gửi dữ liệu rất chậm.	Chiếm dụng toàn bộ các Worker Thread của máy chủ Web.
Database DDoS	SQL Server / DB	Thực hiện các truy vấn phức tạp, tốn tài nguyên tính toán cao.	Vất kiệt CPU của Database Server dù băng thông mạng vẫn thấp.

Bảng 1.3. Các loại tấn công thuộc nhóm L7

Trong phạm vi đề tài, nhóm nghiên cứu đã phân tích sâu về phân loại của tấn công DDoS để chọn 4 loại đại diện cho toàn bộ spectrum:

- UDP Flood → Volumetric (ăn băng thông)
- SYN Flood → Protocol (ăn connection table)
- HTTP Flood → Application (ăn CPU/DB resources)
- Slowloris → Slow Application (khó phát hiện nhất)

1.2.2. Tác động của DDoS đối với SDN

Các cuộc tấn công này để lại những hậu quả nặng nề cho cá nhân và tổ chức, điển hình như:

- Làm sập hệ thống và máy chủ, khiến người dùng hoàn toàn không thể truy cập dịch vụ.
- Gây thiệt hại nghiêm trọng về doanh thu cho doanh nghiệp và tiêu tốn chi phí lớn để khắc phục sự cố.
- Làm gián đoạn công việc, ảnh hưởng đến hiệu suất hoạt động do không có kết nối mạng.
- Ảnh hưởng xấu đến danh tiếng công ty, có thể khiến người dùng rời bỏ và chuyển sang sử dụng dịch vụ của đối thủ.
- Với các kỹ thuật tấn công cao cấp, DDoS còn được dùng làm "bức bình phong" để hacker đánh cắp dữ liệu khách hàng hoặc tiền bạc.

1.3. Cơ sở lý thuyết về Học sâu (Deep Learning)

1.3.1. Từ Machine Learning đến Deep Learning

Sự phát triển của **Học sâu (Deep Learning - DL)** đại diện cho một bước nhảy vọt so với **Học máy truyền thống (Traditional Machine Learning - ML)**. Điểm khác biệt cốt lõi nằm ở cách thức dữ liệu được xử lý và biểu diễn.

- Sự phụ thuộc vào con người (Feature Engineering): Trong các thuật toán ML truyền thống như SVM, Random Forest hay K-Nearest Neighbors, hiệu suất của mô hình phụ thuộc rất lớn vào kỹ năng của chuyên gia trong việc trích chọn đặc trưng thủ công (Manual Feature Engineering). Nếu các đặc trưng được chọn không đại diện tốt cho bản chất dữ liệu, mô hình sẽ hoạt động kém hiệu quả.
- Khả năng tự học (Representation Learning): DL giải quyết vấn đề này bằng cách sử dụng các lớp nơ-ron phân cấp để thực hiện "học biểu diễn". Ở các lớp đầu tiên, mạng học các đặc trưng đơn giản (như các cạnh, góc trong hình ảnh hoặc các mẫu biến động cơ bản trong lưu lượng mạng).

Ở các lớp sâu hơn, các đặc trưng này được tổng hợp thành những khái niệm trừu tượng và phức tạp hơn.

Đặc điểm	Machine Learning (Truyền thống)	Deep Learning
Dữ liệu đầu vào	Cần dữ liệu có cấu trúc, được xử lý kỹ.	Hiệu quả với cả dữ liệu thô, không cấu trúc.
Trích chọn đặc trưng	Thủ công bởi chuyên gia con người.	Tự động thông qua các lớp ẩn (Hidden Layers).
Phụ thuộc phần cứng	Có thể chạy trên CPU thông thường.	Yêu cầu hiệu năng cao (GPU/TPU) do khối lượng tham số lớn.
Thời gian huấn luyện	Nhanh (vài phút đến vài giờ).	Rất lâu (vài ngày đến vài tuần tùy quy mô).
Tính giải thích	Dễ hiểu, quy tắc rõ ràng.	"Hộp đen" (Black Box), khó giải thích cơ chế ra quyết định.

Bảng 1.4. Bảng so sánh giữa Machine Learning và Deep Learning

1.3.2. Mô hình Mạng nơ-ron (CNN/GRU)

Mạng nơ-ron nhân tạo là nền tảng của DL, mô phỏng cách thức hoạt động của hệ thần kinh sinh học thông qua các đơn vị tính toán cơ bản gọi là Perceptron.

a. Mạng nơ-ron tích chập (CNN) - Trích xuất đặc trưng không gian

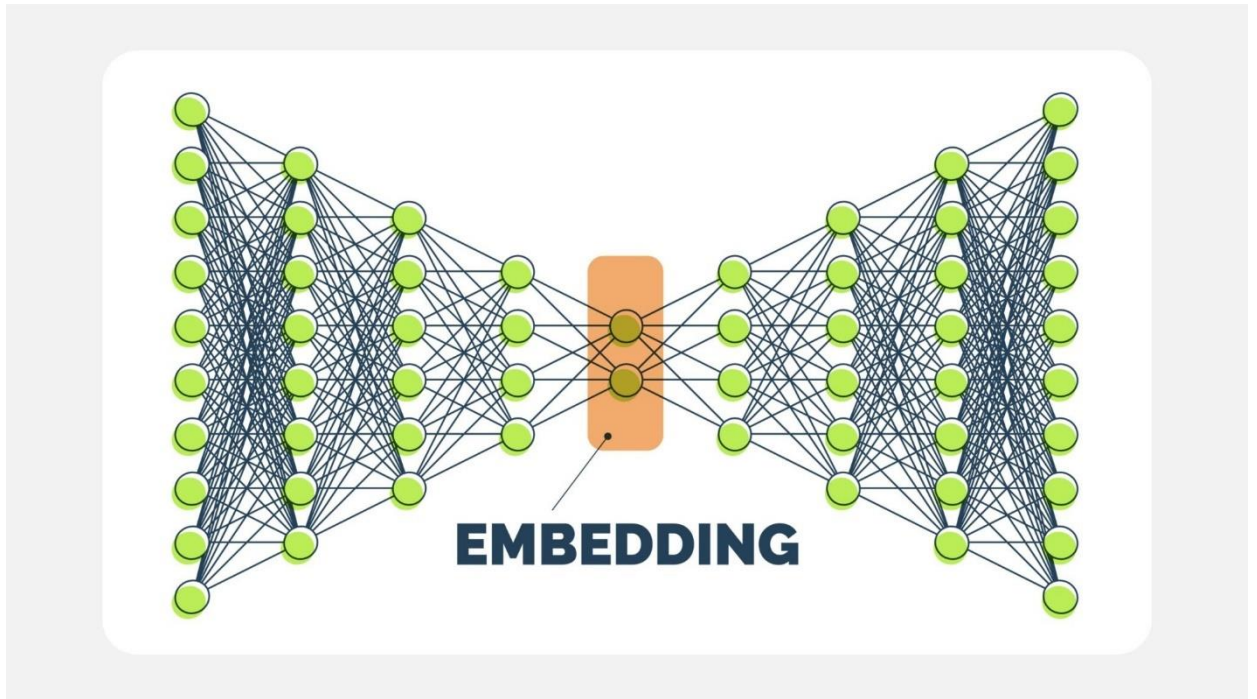
Mặc dù CNN thường được biết đến trong xử lý ảnh, nhưng trong lĩnh vực an ninh mạng, CNN được sử dụng để phát hiện các mẫu cục bộ trong các gói tin.

Cơ chế: Sử dụng các bộ lọc (Filters) trượt qua dữ liệu đầu vào để tạo ra các bản đồ đặc trưng. Điều này giúp mô hình nhận diện được các hành vi tấn công dựa trên sự sắp xếp bất thường của các trường dữ liệu trong một dòng lưu lượng.

b. Đơn vị hồi quy có cổng (GRU) - Phân tích đặc trưng thời gian

Đối với dữ liệu mạng, thứ tự của các gói tin mang thông tin quan trọng. GRU là một biến thể của RNN (Recurrent Neural Network), được thiết kế để xử lý dữ liệu chuỗi mà không bị hiện tượng "biến mất đạo hàm".

Cơ chế Gating: GRU sử dụng hai cổng: **Cổng cập nhật (Update Gate)** và **Cổng thiết lập lại (Reset Gate)** để quyết định thông tin nào từ quá khứ cần được giữ lại và thông tin nào cần loại bỏ (Cho et al 2014). Điều này cực kỳ hữu ích để phát hiện các cuộc tấn công diễn ra trong thời gian dài như APT hay DDoS.



Hình 1.1. Mô hình mạng nơ-ron

1.4. Giả thuyết khoa học và Phương pháp nghiên cứu

1.4.1. Giả thuyết khoa học

Đề tài được xây dựng dựa trên giả thuyết rằng:

"Các cuộc tấn công DDoS để lại những dấu vết thống kê đặc trưng (Statistical patterns) trong lưu lượng mạng mà con người khó nhận biết nhưng các thuật toán Học sâu (Deep Learning) có thể phát hiện chính xác. Nếu kết hợp khả năng phát hiện của AI với khả năng điều khiển tập trung của SDN, ta có thể tạo ra cơ chế phòng thủ tự động với độ trễ thấp."

1.4.2. Phương pháp đánh giá mô hình (Evaluation Metrics)

Để đánh giá hiệu quả của giải pháp, đề tài sử dụng các chỉ số tiêu chuẩn trong khoa học dữ liệu, dựa trên Ma trận nhầm lẫn (Confusion Matrix):

Phân vùng	Thực tế: Tấn công	Thực tế: Bình thường
Dự đoán: Tấn công	True Positive (TP)	False Positive (FP)

Dự đoán: Bình thường	False Negative (FN)	True Negative (TN)
-----------------------------	---------------------	--------------------

Bảng 1.5. Bảng phương pháp đánh giá mô hình

Từ 4 giá trị cốt lõi trên, đề tài sử dụng các công thức toán học sau để đánh giá toàn diện năng lực của "bộ não" AI:

a. Độ chính xác tổng thể (Accuracy): Là tỷ lệ phần trăm các luồng mạng (cả bình thường và tấn công) được mô hình phân loại đúng trên tổng số lượng mẫu kiểm thử.

- Công thức: $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$
- Ý nghĩa: Cung cấp cái nhìn tổng quan về hiệu suất. Tuy nhiên, trong an ninh mạng, dữ liệu thường mất cân bằng (lượng tấn công đôi khi rất nhỏ so với lượng truy cập bình thường), do đó Accuracy không phải là thước đo duy nhất quyết định sự thành công.

b. Độ chính xác theo nhãn (Precision) Là tỷ lệ số luồng tấn công thực sự trên tổng số các luồng mà mô hình cảnh báo là tấn công.

- Công thức: $Precision = \frac{TP}{TP+FP}$
- Ý nghĩa thực tiễn: Chỉ số này vô cùng quan trọng đối với trải nghiệm người dùng. Precision càng cao đồng nghĩa với việc tỷ lệ Báo động giả (False Positive) càng thấp. Điều này đảm bảo Controller ONOS sẽ không vô tình ban hành luật cấm (DROP rule) chặn nhầm những người dùng hợp lệ đang cố gắng truy cập dịch vụ.

c. Độ nhạy (Recall / True Positive Rate) Là tỷ lệ phần trăm các luồng tấn công thực tế đã được mô hình phát hiện thành công.

- Công thức: $Recall = \frac{TP}{TP+FN}$
- Ý nghĩa thực tiễn: Thể hiện khả năng không bỏ sót của hệ thống. Recall càng cao, số lượng các cuộc tấn công lẫn trốn (đặc biệt là các loại tấn công âm mưu kết nối như Slowloris) vượt qua được hàng rào bảo vệ càng thấp, đảm bảo sự sống còn cho máy chủ Web.

d. Điểm số F1 (F1-Score) Là trung bình điều hòa giữa Precision và Recall.

- Công thức: $F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$
- Ý nghĩa thực tiễn: Đây là thước đo quan trọng nhất của đề tài. Việc tối ưu hóa cả Precision (không chặn nhầm) và Recall (không bỏ sót) thường

mâu thuẫn với nhau. F1-Score giúp đánh giá sự cân bằng hoàn hảo của mô hình. Đặc biệt, với các tập dữ liệu mất cân bằng nghiêm trọng (như số lượng mẫu tấn công Slowloris cực kỳ ít so với UDP Flood), một điểm F1-Score cao sẽ minh chứng cho việc mô hình thực sự thấu hiểu bản chất của loại tấn công đó chứ không phải do dự đoán theo số đông.

CHƯƠNG 2: PHÂN TÍCH, THIẾT KẾ VÀ XÂY DỰNG MÔ HÌNH HỆ THỐNG

2.1. Phân tích đánh giá các hướng nghiên cứu liên quan đến đề tài

2.1.1. Các nghiên cứu trên thế giới

- **Giai đoạn đầu (Thống kê & Entropy):** Các nghiên cứu ban đầu tập trung vào phương pháp thống kê (như tính toán Entropy của IP đích). Ưu điểm là nhẹ, nhưng nhược điểm là khó phát hiện các tấn công DDoS cường độ thấp hoặc chậm (Low-rate DDoS).
- **Giai đoạn Machine Learning (ML):** Các tác giả đã áp dụng SVM, K-Nearest Neighbors, Random Forest. Kết quả tốt hơn thống kê, nhưng vẫn phụ thuộc nhiều vào việc con người phải tự tay trích xuất đặc trưng (Feature Engineering), tốn thời gian và dễ bỏ sót thông tin.
- **Giai đoạn hiện tại (Deep Learning):** Các nghiên cứu gần đây (từ 2019-nay) chuyển hướng sang CNN, LSTM, GRU. *Ví dụ: Nghiên cứu của Tác giả A (năm 2022) sử dụng CNN-LSTM đạt độ chính xác 98% nhưng thời gian huấn luyện còn lâu.*

2.1.2. Các nghiên cứu trong nước

- Tại Việt Nam, các nghiên cứu chủ yếu tập trung vào việc so sánh hiệu năng giữa các bộ điều khiển (ONOS, ODL) khi chịu tải tấn công.
- Một số đồ án/luận văn gần đây đã bắt đầu tích hợp AI, nhưng đa phần dừng lại ở mô phỏng phát hiện (Detection) mà chưa chú trọng nhiều vào cơ chế giảm thiểu (Mitigation) tự động hoặc tối ưu hóa thời gian thực.

2.1.3. Khoảng trống nghiên cứu (Research Gap)

- Hầu hết các nghiên cứu tập trung vào độ chính xác (Accuracy) mà bỏ qua yếu tố **thời gian trễ (Latency)**. Trong SDN, nếu phát hiện chậm, Controller sẽ bị sập trước khi kịp chặn tấn công. Đây là điểm mà đề tài này sẽ tập trung cải thiện.

2.2. Thực trạng có liên quan đến KLTN

2.2.1. Hoạt động quản lý và vận hành mạng SDN

- Trong mô hình hiện tại, Quản trị viên sử dụng Controller để định tuyến tập trung.
- Cơ chế hoạt động: Khi có gói tin mới (New Flow), Switch gửi thông điệp Packet-In lên Controller để hỏi đường đi. Controller tính toán và gửi lệnh

Flow-Mod xuống. Đây là quy trình tiêu chuẩn nhưng tốn tài nguyên xử lý.

2.2.2. Hoạt động giám sát và bảo mật hiện hành

- Các hệ thống hiện tại thường sử dụng **Ngưỡng cố định (Static Threshold)**. Ví dụ: "*Nếu 1 IP gửi quá 1000 gói tin/giây thì chặn*".
- Hệ thống IPS/IDS truyền thống hoạt động độc lập, không giao tiếp sâu với Controller của SDN, dẫn đến phản ứng chậm chạp khi cấu trúc mạng thay đổi.

2.3. Đánh giá thực trạng

2.3.1. Những kết quả đạt được (Ưu điểm của mô hình hiện tại)

- Kiến trúc SDN giúp quản lý mạng linh hoạt, dễ dàng thay đổi cấu hình mà không cần chạm vào từng thiết bị phần cứng.
- Các phương pháp phát hiện dựa trên ngưỡng đơn giản, dễ triển khai, tốn ít tài nguyên tính toán.

2.3.2. Những hạn chế và Nguyên nhân

- **Hạn chế 1: Tỷ lệ báo động giả (False Positive) cao.**
 - *Nguyên nhân:* Do sử dụng ngưỡng cố định. Trong các đợt Flash Crowd (ví dụ: lượng truy cập tăng đột biến ngày Black Friday), lưu lượng hợp lệ tăng cao giống hệt tấn công, hệ thống cũ sẽ chặn nhầm người dùng thật.
- **Hạn chế 2: Điểm yếu trầm trọng tại Controller (Saturation Attack).**
 - *Nguyên nhân:* Khi tấn công DDoS xảy ra, hàng triệu gói tin lạ khiến Switch gửi hàng triệu tin Packet-In lên Controller. Controller bị quá tải (CPU 100%) và treo, khiến toàn bộ mạng tê liệt.
- **Hạn chế 3: Không phát hiện được tấn công chưa biết (Zero-day).**
 - *Nguyên nhân:* Các hệ thống dựa trên chữ ký (Signature-based) chỉ chặn được những gì đã biết trong cơ sở dữ liệu.

2.4. Đề xuất giải pháp để giải quyết vấn đề

Dựa trên các phân tích trên, KLTN đề xuất giải pháp: "**Hệ thống phòng thủ cộng tác giữa SDN và Deep Learning**".

2.4.1. Kiến trúc giải pháp tổng thể

Mô hình đề xuất gồm 3 module chính hoạt động theo vòng khép kín:

1. Module Thu thập (Traffic Collector):

- Sử dụng API của Onos Controller để định kỳ trích xuất thống kê (Flow Statistics) từ các Switch kết hợp với kỹ thuật Nfstream tiên tiến để bóc tách lưu lượng tầng 7 (thay vì bắt toàn bộ gói tin gây nặng máy).

2. Module Phân tích (Deep Learning Engine):

- Sử dụng mạng nơ-ron (CNN-GRU) đã huấn luyện để phân tích dữ liệu thống kê vừa thu thập.
- Phân loại lưu lượng: 0 (Bình thường) - 1 (Tấn công), nâng cao hơn là sẽ phân loại cụ thể các tấn công.

3. Module Ngăn chặn (Mitigation):

- Nếu phát hiện tấn công (Nhãn 1), lập tức tạo luật (dựa vào độ tin cậy của dự đoán mà có quyết định linh động với kẻ tấn công) và đẩy xuống Switch thông qua OpenFlow.
- Cơ chế chặn theo IP nguồn hoặc cổng tấn công để không ảnh hưởng người dùng khác.

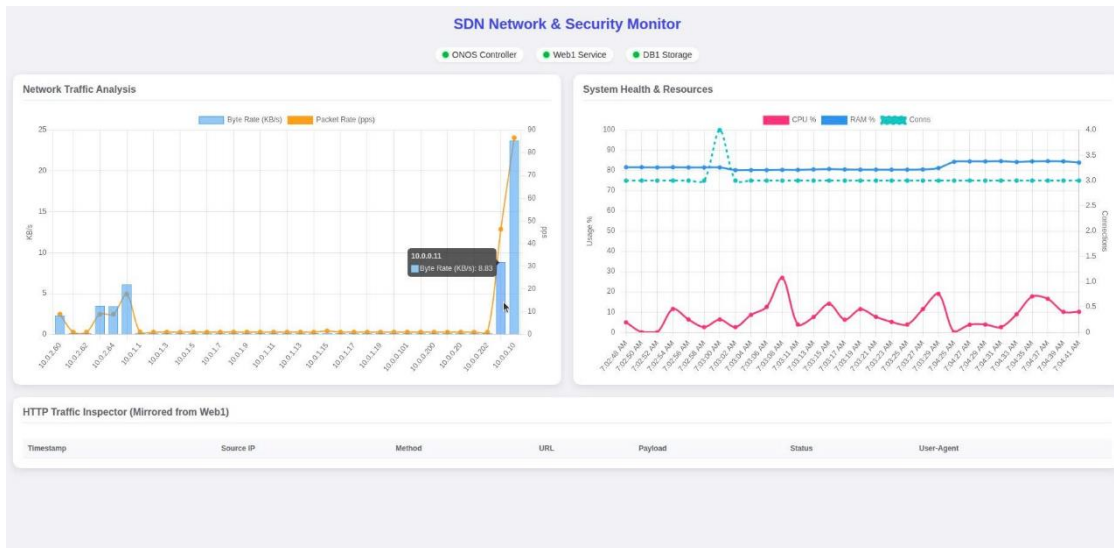
So sánh tổng hợp các loại hình tấn công được chọn:

Đặc điểm	UDP Flood	SYN Flood	HTTP Flood	Slowloris
Layer	3 (Network)	4 (Transport)	7 (Application)	7 (Application)
Target	Bandwidth	Connection Table	CPU/DB	Connection Pool
Speed	Very Fast	Fast	Medium	Very Slow
Volume	High (GB/s)	Medium (MB/s)	Low (MB/s)	Very Low (KB/s)
Detection	Easy	Medium	Hard	Very Hard
Dataset	Có sẵn	Có sẵn	Có sẵn	KHÔNG CÓ

Bảng 2.1. Bảng so sánh tổng hợp đặc điểm các loại tấn công

→ **Lý do chọn 4 loại:** Đại diện cho toàn bộ spectrum DDoS, từ dễ đến khó, đặc biệt Slowloris là độc nhất và không có trong bất kỳ dataset public nào.

Ngoài 3 module chính, hệ thống còn có Biểu đồ phân tích (Analytical Charts): Trực quan hóa dữ liệu dưới dạng đồ thị đường (Line Chart) để theo dõi xu hướng lưu lượng được tích hợp làm 1 thành phần trong hệ thống mạng để giảm tải cho phần cứng thiết bị.



Hình 2.1. Biểu đồ hiển thị lưu lượng mạng bình thường

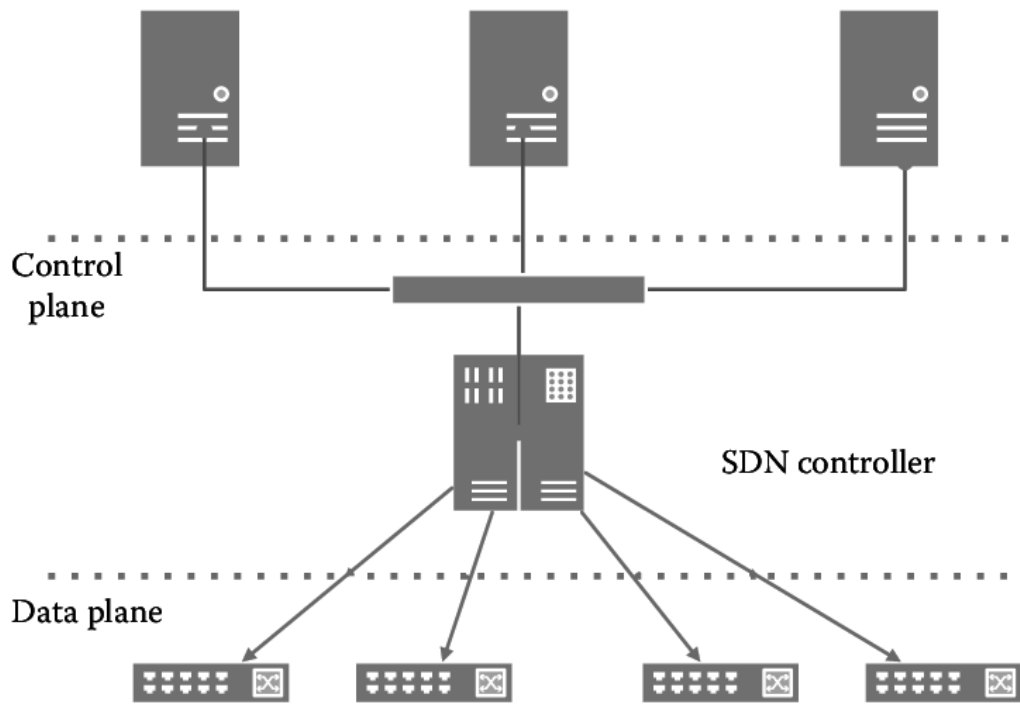
2.4.2. Giải pháp cải thiện các hạn chế

- **Giải quyết vấn đề Báo động giả:** Deep Learning học được các mẫu hành vi phức tạp (phi tuyến tính), phân biệt được đâu là tấn công thật, đâu là lượng truy cập tăng cao hợp lệ → Tăng độ chính xác.
- **Giải quyết vấn đề Quá tải Controller:** Hệ thống phát hiện sớm và đẩy luật chặn ngay tại Switch (Data Plane). Các gói tin tấn công sau đó sẽ bị Switch loại bỏ ngay tại cửa ngõ, không còn cơ hội gửi lên làm phiền Controller nữa.

2.4.3. Thiết kế mô hình hệ thống (System Design)

2.4.3.1. Kiến trúc tổng thể (High-level Architecture)

SDN applications



Hình 2.2. Sơ đồ kiến trúc SDN

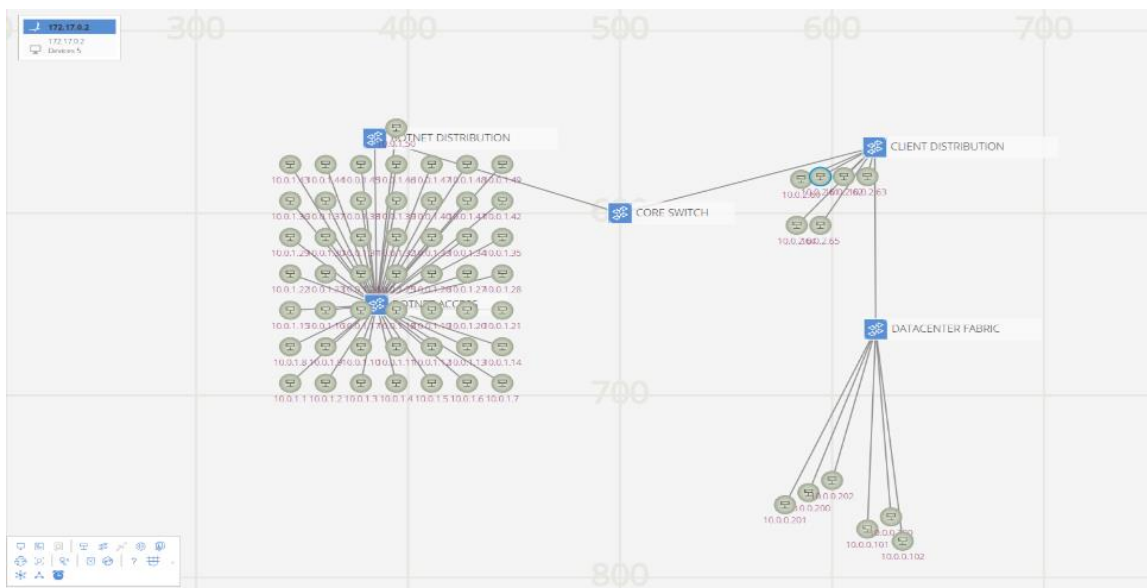
Em xin mô tả lại kiến trúc SDN tổng thể ứng với đề tài trước khi đưa vào triển khai, kiến trúc được bao gồm 3 lớp được xác định cụ thể như sau:

- a. Lớp dữ liệu (Data Plane): Bao gồm các Switch, các host được xây dựng trong hệ thống mạng được quản lý chặt chẽ bởi Onos Controller, chúng có nhiệm vụ đẩy gói tin đi theo luồng được thiết lập sẵn.
- b. Lớp điều khiển (Control Plane): Trong đề tài này, chúng em lựa chọn Onos Controller bởi các ưu điểm vượt trội:
 - Khả năng chịu lỗi: do được thiết kế dưới dạng một Cluster (nhiều máy chủ cùng điều khiển một lúc) nên khi một Controller bị sập, các Controller khác trong cụm sẽ thay thế mà không làm gián đoạn hệ thống mạng.
 - Khả năng trực quan: Onos cho chúng ta thấy được trực tiếp sơ đồ mạng biểu diễn thông qua trang web, các luồng đi của gói tin hay sự quá tải bằng thông mạng bằng các đường báo đỏ.
 - Hệ thống API của Onos được chuẩn hóa giúp dễ dàng tích hợp các ứng dụng bên ngoài (như Python AI, Web Dashboard).

- c. Lớp ứng dụng (Application Plane): Bao gồm các ứng dụng đảm nhiệm các chức năng cụ thể trong dự án:
- Hiện thị cho người dùng thấy được hệ thống mạng thông qua Dashboard V14.
 - Chứa mô hình AI đã được tích hợp để phân tích, thực hiện dự đoán tấn công DDos trong hệ thống mạng.
 - Cuối cùng là thực thi kịch bản phòng thủ. Khi xác định được đó là tấn công, lớp này sẽ ra lệnh cho Onos (Lớp điều khiển) viết các luật chặn không cho gói tin đi đến Server.

2.4.3.2. Thiết kế Topology mạng giả lập

Trong quá trình xây dựng hạ tầng thực nghiệm, việc lựa chọn kiến trúc mạng đóng vai trò tiên quyết đến sự ổn định của hệ thống phòng thủ. Ban đầu, các mô hình mạng truyền thống thường sử dụng kiến trúc chuyển mạch lớp 2 (L2 MAC-based). Tuy nhiên, trên mặt lý thuyết và thiết kế hệ thống quy mô lớn, mạng L2 bộc lộ điểm yếu trầm trọng: dễ sinh ra hiện tượng bão mạng (Broadcast Storm) và rủi ro tràn bảng địa chỉ MAC của các Switch khi số lượng thiết bị tăng cao, làm mất tính ổn định của hệ thống.



Hình 2.3. Hệ thống mạng L2 ban đầu

Để khắc phục, đề tài đề xuất chuyển dịch toàn diện sang kiến trúc mạng định tuyến phân lớp L3 chuẩn Enterprise tương tự như hệ thống mạng của các công ty lớn trên thế giới hiện nay. Theo đó, hệ thống được phân chia thành 3 phân vùng logic độc lập nhằm cô lập rủi ro và quản lý luồng dữ liệu.

1. Thiết kế Topology mạng giả lập theo kiến trúc định tuyến L3 (Layer 3 Routing)

Bằng cách áp dụng định tuyến lớp 3, hệ thống đã thực hiện phân tách rõ ràng các phân vùng mạng con (Subnets) dựa trên chức năng, bao gồm: dải 10.0.1.0/24 dành riêng cho phân vùng Botnet (Attacker), 10.0.2.0/24 cho người dùng hợp lệ (Client) và 10.0.0.0/24 cho khu vực trung tâm dữ liệu. Kiến trúc này cho phép bộ điều khiển SDN Controller quản lý và điều phối các luồng dữ liệu trực tiếp dựa trên địa chỉ IP, giúp tối ưu hóa khả năng giám sát và thực thi các chính sách bảo mật.

- a. Lớp lõi (Core Layer): Core Switch, đây là trung tâm điều phối lưu lượng toàn mạng, kết nối trực tiếp đến Controller và các Switch phân phối.
- b. Lớp phân phối (Distribution Layer):
 - Botnet Distribution: Quản lý phân vùng lưu lượng từ các nút tấn công.
 - Client Distribution: Quản lý lưu lượng từ các người dùng hợp lệ.
 - Datacenter Fabric: Phân phối dữ liệu cho phân vùng máy chủ.

Lớp truy cập (Access Layer): Để đảm bảo tính thực tế, các host được gán dải IP riêng biệt theo từng khu vực.

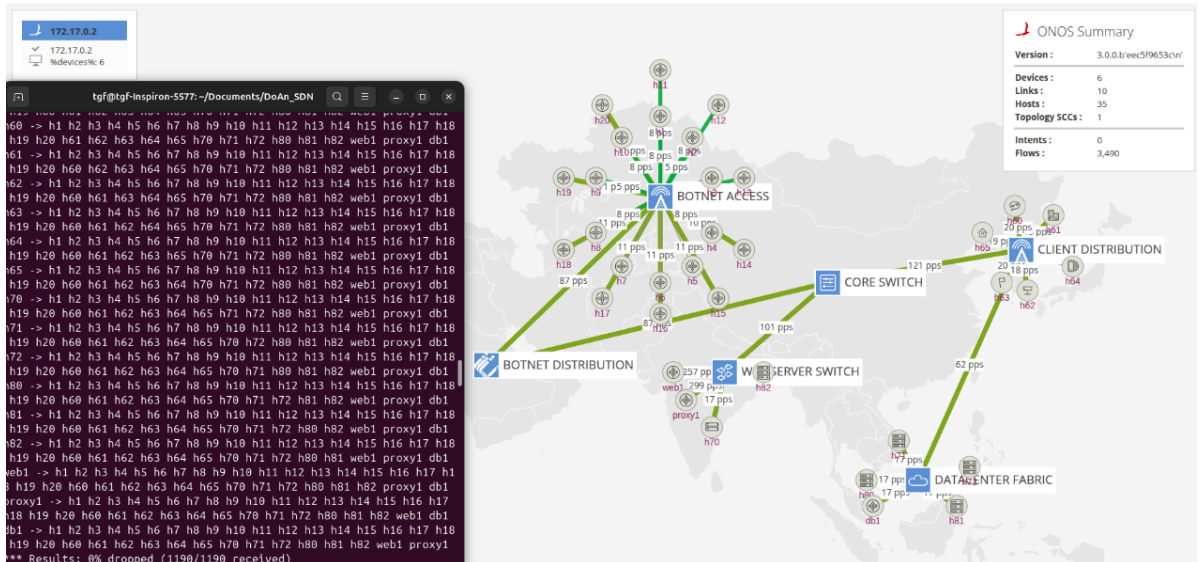
- **Mạng Botnet (Subnet 10.0.1.0/24):** Nơi tập kết các nút mạng đóng vai trò kẻ tấn công, Gồm 20 node từ 10.0.1.1 - 10.0.1.20 được dùng để mô phỏng tấn công DDos.
- **Mạng Client (Subnet 10.0.2.0/24):** Phân vùng dành cho các người dùng hợp lệ truy cập dịch vụ Hệ thống được phân chia thành 3 phân vùng logic để phục vụ các kịch bản tấn công và phòng thủ DDos.
- **Trung tâm dữ liệu - Datacenter (Subnet 10.0.0.0/24):** Nơi đặt các máy chủ dịch vụ mục tiêu (Web Server, Database, Proxy)
- Proxy (10.0.0.10): Là cửa ngõ truy cập vào web 1 phòng theo thực tế, kẻ tấn công và người dùng chỉ có thể truy cập dịch vụ thông qua ip public.

Việc áp dụng định tuyến L3 cho phép bộ điều khiển ONOS quản lý luồng dữ liệu thông qua địa chỉ IP, tối ưu hóa khả năng giám sát và là tiền đề để áp đặt các luật cách ly (Drop/Rate Limit) một cách chính xác lên từng phân mạng.

Phân vùng	Đối tượng bao gồm	Kịch bản mô phỏng
Tấn công	Host h1 đến h20	Mô phỏng mạng Botnet quy mô lớn thực hiện tấn công Flood.
Người dùng	Host h60 đến h65	Mô phỏng người dùng hợp lệ truy cập dịch vụ.

Dịch vụ	web1, db1, h70-h82, proxy (mạng hiện đại thực tế)	Đối tượng nạn nhân chịu tải và cần được bảo vệ
----------------	---	--

Bảng 2.2. Bảng phân vùng hệ thống mạng



Hình 2.4. Mô hình thiết kế mạng (liên thông & ổn định)

Kiến trúc Web của hệ thống cũng được thiết kế theo mô hình phân lớp đặc trưng của SDN, phân tách rõ ràng giữa giao diện điều khiển và máy chủ thực thi:

- Lớp Ứng dụng: Tập trung các logic quản lý và Dashboard giám sát, giúp quản trị viên tương tác với tầng điều khiển ONOS để thực thi các chính sách bảo mật.
- Lớp dữ liệu: Triển khai trực tiếp các thành phần máy chủ (Web Server, PostgreSQL) để xử lý các truy vấn thực tế từ người dùng.

Được triển khai trên nền tảng giao thức HTTPS bảo mật và kết nối trực tiếp với cơ sở dữ liệu PostgreSQL, trang web không chỉ phục vụ các truy vấn tương tác dữ liệu thực tế của người dùng mà còn cung cấp các endpoint API để tiếp nhận và lưu trữ nhật ký phản vệ từ tác tử AI.

Đăng nhập

Tên đăng nhập:

Mật khẩu:



Hình 2.5. Mô hình Web https được xây dựng với các dịch vụ sử dụng Db1

Hệ thống này đóng vai trò kép: vừa là đối tượng mục tiêu trực tiếp của các cuộc tấn công tầng ứng dụng (như HTTP Flood, Slowloris), vừa là trung tâm điều hành cho phép quản trị viên giám sát trạng thái mạng theo thời gian thực.

2. Sơ đồ luồng dữ liệu tổng quát (General Data Flow) Trong kiến trúc SDN truyền thống, thiết bị Switch gửi toàn bộ các gói tin lạ lên Controller thông qua bản tin Packet-In, biến Controller thành điểm nghẽn cổ chai (Bottleneck) và rất dễ sụp đổ khi bị tấn công DDoS.

Để giải quyết vấn đề này, đề tài thiết kế một sơ đồ luồng dữ liệu bất đồng bộ mang tính đột phá dựa trên nguyên lý giám sát ngoại tuyến (Out-of-Band). Luồng dữ liệu hoạt động theo thiết kế sau:

- a. **Sao chép luồng (Port Mirroring):** Mọi gói tin đi qua Gateway Switch đều được nhân bản (Mirror) và đẩy ra một cổng giám sát độc lập. Bảng thông thực tế của người dùng hoàn toàn không bị can thiệp.
- b. **Bóc tách và Tính toán:** Dữ liệu nhân bản đi vào Module trích xuất để bóc tách từ tầng L3 đến L7, sau đó chuyển hóa thành các Tensor (ma trận đặc trưng).
- c. **Suy luận AI:** Tensor được đưa vào bộ não Deep Learning để phân tích. Nếu phát hiện hành vi tấn công, một chỉ thị phòng vệ sẽ được gửi qua REST API xuống ONOS Controller.
- d. **Thực thi rào chắn:** ONOS ra lệnh cho các Switch (Data Plane) cập nhật Flow Table, lập tức tiêu hủy các gói tin độc hại ngay tại cửa ngõ hạ tầng mạng.

2.4.3.3. Thiết kế Module trích xuất đặc trưng (Feature Extraction Module)

Lưu lượng thô thu được từ mạng SDN không thể được đưa trực tiếp vào mạng Nơ-ron do chứa nhiều thông tin không có cấu trúc. Để AI có thể "hiểu" được hành vi mạng, Module trích xuất đặc trưng được thiết kế để chuyển đổi các gói tin rời rạc thành một không gian toán học gồm **26 chiều đặc trưng** (13 đặc trưng tĩnh và 13 đặc trưng biến thiên) dựa trên phương pháp thống kê theo luồng (Flow-based). Dưới đây là các cơ sở toán học cốt lõi của thiết kế này:

a. Trích xuất đặc trưng hỗn loạn bằng Shannon Entropy: Các kỹ thuật phát hiện tấn công cũ thường dựa vào việc theo dõi số hiệu cổng vật lý (Port Number). Tuy nhiên, tin tặc dễ dàng vượt qua bằng kỹ thuật làm giả IP (IP Spoofing) hoặc tấn công ngẫu nhiên vào nhiều cổng khác nhau. Để giải quyết bài toán này, đề tài ứng dụng công thức **Shannon Entropy** để đo lường độ ngẫu nhiên của lượng thông tin trong một cửa sổ thời gian.

Công Thức Toán Học: Shannon Entropy

Shannon Entropy là đo lường lượng thông tin (information content) của một biến ngẫu nhiên. Trong ngữ cảnh phát hiện DDoS, nó đo độ đa dạng (diversity) của các port được sử dụng:

$$H(X) = - \sum_{i=1}^n p(x_i) \cdot \log_2(p(x_i)) \quad (1)$$

Trong đó:

$H(X)$: Entropy của tập dữ liệu X

$p(x_i)$: Xác suất xuất hiện của giá trị x_i (Port thứ i)

n : Số lượng port khác nhau trong cửa sổ quan sát

$\log_2$: Logarit cơ số 2, đơn vị tính là bits

Ý nghĩa trong phát hiện tấn công:

Loại Traffic	Entropy	Giải thích
Normal User	$H \approx 3.0 - 3.5$	Mở nhiều tab $\rightarrow$ nhiều port ngẫu nhiên $\rightarrow$ Entropy cao
UDP Flood	$H \approx 0.0 - 0.5$	1 port cố định (6666) $\rightarrow$ Entropy thấp
SYN Flood	$H \approx 0.0$	Botnet dùng cùng 1 src_port $\rightarrow$ Entropy = 0

Bảng 2.3. Ý nghĩa trong phát hiện tấn công dựa vào Entropy

```

from collections import Counter
import math

def calculate_port_entropy(ports_window: List[int], window_size=10) -> float:
    """Shannon Entropy của cổng trong cửa sổ trượt"""
    counts = Counter(ports_window)
    entropy = 0
    for count in counts.values():
        p = count / len(ports_window)
        entropy -= p * math.log2(p)
    return round(entropy, 4)

# Ví dụ:
normal_ports = [45821, 45822, 45823, 45824, 45825] # → Entropy = 3.32
udp_flood_ports = [6666, 6666, 6666, 6666, 6666] # → Entropy = 0.0
# MODEL NGAY THẤY: Entropy khác biệt rõ ràng → Dễ phân biệt!

```

Hình 2.6. Logic tính Shannon Entropy cho cổng

Ý nghĩa thiết kế: Người dùng bình thường khi lướt web sẽ mở nhiều kết nối ngẫu nhiên, tạo ra độ hỗn loạn cao. Ngược lại, một cuộc tấn công mạng Botnet nhắm vào một dịch vụ cụ thể (như UDP Flood) sẽ có tính định hướng cao, khiến giá trị Entropy hội tụ về mức rất thấp. Toán học Entropy giúp AI không cần biết gói tin dùng cổng nào, mà chỉ cần đánh giá "sự tự nhiên" của hành vi mở cổng.

b. Đánh giá cường độ thông qua Packet Rate và Byte Rate: Đây là hai chỉ số vận tốc cơ bản nhằm bóc tách các cuộc tấn công làm ngập lụt băng thông (Volumetric Attacks) như UDP Flood hoặc ICMP Flood.

Trong khi người dùng hợp lệ thường có tốc độ gói tin bất định (khoảng 5-50 pps), các luồng tấn công ngập lụt có thể đẩy tốc độ lên tới hơn 10.000 pps, tạo ra ranh giới nhận diện rõ ràng cho mạng nơ-ron.

Công Thức Toán Học: Packet Rate và Byte Rate

Hai đặc trưng quan trọng nhất để phát hiện tấn công DDoS dựa trên tốc độ:

1. Packet Rate (Tốc độ gói tin):

$$Packet_Rate = \frac{Src_Packets}{Duration_Sec} \quad (2)$$

2. Byte Rate (Tốc độ byte):

$$Byte_Rate = \frac{Src_Bytes}{Duration_Sec} \quad (3)$$

Ý nghĩa trong phát hiện tấn công:

Loại Traffic	Packet Rate	Byte Rate	Giải thích
Normal User	5-50 pps	100KB-1MB/s	Tốc độ tự nhiên, không ổn định
UDP Flood	10,000+ pps	100MB/s	Tấn công theo thể tích - cạn kiệt băng thông
SYN Flood	1,000+ pps	Thấp (~1KB/s)	Kết nối nửa mở, không đối xứng
Slowloris	10-20 pps	1-5KB/s	Low-rate, nhưng kéo dài 300-600s

Bảng 2.4. Ý nghĩa dựa vào Packet rate và Byte rate

→ Tấn công DDoS không phải "mở port nào", mà là "hành vi lạ trong port đó".

c. Bắt mạch tấn công lẫn trốn bằng Toán học Vi phân (Differential Features): Một trong những điểm nghẽn lớn nhất của hệ thống an ninh mạng là các cuộc tấn công tàng ứng dụng (Layer 7) như **Slowloris**. Kẻ tấn công cố tình duy trì tốc độ truyền tải cực kỳ thấp (khoảng 10-20 gói/phút) để lẫn trốn, khiến các chỉ số Packet Rate hoặc Byte Rate trở nên vô dụng do trông giống hệt một người dùng có mạng yếu.

Để giải quyết, đề tài đã thiết kế nhóm **13 Đặc trưng biến thiên ()** bằng cách ứng dụng toán tử vi phân rời rạc. Thay vì chỉ nhìn vào giá trị tuyệt đối tại thời điểm, hệ thống đo lường sự chênh lệch (gia tốc) so với thời điểm trước đó với công thức: $\Delta f_t = f_t - f_{t-1}$

```

[*] Chuẩn bị dữ liệu chuỗi thời gian...
X shape: (60000, 13), y shape: (60000,)
[*] Đang tạo chuỗi thời gian (Stride=2, Downsample=0.1)...
Input shape: (60000, 13) | Số tính 26 đặc trưng (13 gốc + 13 biến thiên)
Trước tính Differential: X shape = (131876, 10, 26)
[*] Đang trích xuất đặc trưng biến thiên (Differential Features)...
Sau tính Differential: X shape = (131876, 10, 26)
Tổng số sequences: 131876

[*] Áp dụng Feature Importance Weighting...
Feature weights: Duration=2.0, Packet_Rate=2.0, Byte_Rate=1.5, Port_Entropy=0.5
[*] Đã lưu feature_weights.pkl

[*] Data preprocessing:
Train: torch.Size([84490, 10, 26])
Val: torch.Size([21100, 10, 26])
Test: torch.Size([26376, 10, 26])

[*] Khởi tạo mô hình trên cuda...
Autoencoder: 615,452 parameters
Classifier: 629,520 parameters

=====
[PHA 1] Huấn luyện Khiến 1: Contrastive Autoencoder
=====
Epoch 5/80 | Loss: -0.087670
Epoch 10/80 | Loss: -0.088359
Epoch 15/80 | Loss: -0.089029
Epoch 20/80 | Loss: -0.089566
Epoch 25/80 | Loss: -0.090046
Epoch 30/80 | Loss: -0.090603
Epoch 35/80 | Loss: -0.090688
Epoch 40/80 | Loss: -0.090248
Epoch 45/80 | Loss: -0.090878
Epoch 50/80 | Loss: -0.092328
Epoch 55/80 | Loss: -0.091457
Epoch 60/80 | Loss: -0.091538
Epoch 65/80 | Loss: -0.092124
Epoch 70/80 | Loss: -0.091999
Epoch 75/80 | Loss: -0.092043
[*] Early stopping kích hoạt!
Huấn luyện Autoencoder hoàn thành! Best loss: -0.092358
  
```

Hình 2.7. Cơ chế vi phân tạo ra kết quả Loss giá trị âm khi train lớp autoencoder

Ý nghĩa thiết kế: Toán học vi phân giúp AI nhìn ra được "vận tốc thay đổi". Mọi hành vi của người dùng thật luôn có sự biến thiên ngẫu nhiên theo thời gian (lúc click nhanh, lúc click chậm). Trong khi đó, traffic sinh ra từ máy móc (Botnet Slowloris) sẽ mang tính chu kỳ, lặp lại tuyến tính, khiến gia tốc gần như bằng 0 một cách vô lý. Sự bổ sung chiều không gian này mang lại khả năng định danh sắc bén cho mô hình Học sâu.

d. Chuẩn hóa không gian dữ liệu bằng StandardScaler (Z-Score): Với 26 đặc trưng được trích xuất, biên độ giá trị của chúng có sự chênh lệch khổng lồ (Ví dụ: Src_Bytes có thể lên tới hàng triệu, trong khi Duration chỉ là số thập phân). Nếu đưa trực tiếp vào mạng nơ-ron, mô hình sẽ bị "thiên vị" bởi các con số lớn. Hệ thống thiết kế hàm chuẩn hóa **StandardScaler (Z-Score)** để ánh xạ toàn bộ dữ liệu về phân phối chuẩn $N(0, 1)$ trước khi đưa vào AI:

$$z = \frac{x - \mu}{\sigma} \quad (4)$$

Trong đó:

- x : Giá trị gốc của đặc trưng
- μ (mu): Giá trị trung bình (mean) của đặc trưng
- σ (sigma): Độ lệch chuẩn (standard deviation)
- z : Giá trị đã chuẩn hóa (z-score)

→ Mục đích: AI không bị lựa chọn nhầm bởi các đặc trưng có giá trị lớn.

Nhờ thao tác này, cả 26 đặc trưng đầu vào đều có tiếng nói và trọng số ngang nhau, giúp mạng lưới trí tuệ nhân tạo (AI) hội tụ nhanh hơn và phân tích công bằng hơn trong quá trình dự báo.

2.4.3.4. Thiết kế Module AI (Kiến trúc phòng thủ "Hai lớp khiên")

Lưu lượng tấn công DDoS hiện đại (Pulse Wave DDoS hay Low-rate DDoS) thường được ngụy trang tinh vi dưới dạng các xung lưu lượng có quy luật. Các phương pháp Machine Learning truyền thống thường chỉ phân tích dữ liệu tại một thời điểm độc lập, dẫn đến việc bỏ sót các hành vi mang tính chu kỳ này. Do đó, hệ thống thiết kế một Module AI phân tích đa chiều dựa trên triết lý **"Hai lớp khiên"**.

Lý do lựa chọn kiến trúc Hybrid CNN-GRU-Attention: Trong quá trình thiết kế, nhóm nghiên cứu đã cân nhắc rất nhiều kiến trúc học sâu tiên tiến (như Graph Neural Networks - GNN, LSTM, hay Transformer). Tuy nhiên, quyết

định lựa chọn sự kết hợp giữa **CNN và GRU** xuất phát từ bài toán tối ưu tốc độ và tài nguyên cùng phân tích hành vi lẫn trôn:

- Mô hình AI cần nhận diện tấn công DDoS nhanh chóng và chính xác để tránh bộ điều khiển (mục tiêu bảo vệ) bị quá tải trước khi kịp ra lệnh ngăn chặn. Kiến trúc Transformer dù mạnh nhưng tiêu tốn quá nhiều tài nguyên RAM, trong khi LSTM xử lý chuỗi thời gian chậm hơn GRU. GRU có cấu trúc công tối giản hơn giúp tăng tốc độ suy luận mà không chiếm dụng quá nhiều tài nguyên của hệ thống mạng.
- Thực tế DDoS có những phương thức tinh vi, cực kỳ khó nhận biết nếu chỉ nhìn vào các gói dữ liệu rời rạc (như tấn công Slowloris). Do đó, kiến trúc bắt buộc phải có mạng GRU để xử lý được dữ liệu kéo dài theo chuỗi thời gian.

Cấu trúc tổng thể của mô hình AI bao gồm 4 khối chức năng cốt lõi:

a. Khối đầu vào và Đặc trưng biến thiên (Input Layer) Thay vì đọc từng gói tin đơn lẻ, mạng AI được cung cấp cái nhìn toàn cảnh về "quá khứ gần" của mạng thông qua cơ chế Cửa sổ trượt (Sliding Window) với độ dài 10 bước thời gian. Điểm đột phá trong kiến trúc đầu vào là việc sử dụng ma trận đặc trưng 26 chiều. Trí tuệ nhân tạo không chỉ học 13 đặc trưng gốc tĩnh (như tổng số byte, số lượng gói tin) mà còn được cung cấp thêm 13 đặc trưng biến thiên (Differential Features). Đây là các giá trị đo lường gia tốc thay đổi lưu lượng giữa giây trước và giây sau, giúp mô hình nhận diện được các cuộc tấn công siêu chậm mà nếu chỉ nhìn vào con số tuyệt đối sẽ tưởng lầm là người dùng bình thường.

b. Lớp khiên thứ nhất: Bộ lọc dị thường (Contrastive Autoencoder) Lớp này đóng vai trò là một bộ lọc dị thường (Anomaly Detector), hoạt động theo cơ chế học không giám sát. Mô hình học cách nén và giải nén các luồng dữ liệu hợp lệ. Kiến trúc này được sử dụng làm chốt chặn phát hiện các cuộc tấn công chưa từng biết đến (Zero-day Attacks). Bất kể kẻ tấn công dùng phương thức mới nào, chỉ cần cấu trúc gói tin làm sai lệch kết quả tái tạo vượt quá ngưỡng cho phép, lớp khiên này sẽ đánh dấu là dị thường. Để tăng cường độ nhạy, thay vì sử dụng hàm mất mát trung bình bình phương (MSE) thông thường, hệ thống đề xuất thiết kế lớp khiên này dựa trên cơ sở toán học của Học tương phản (Contrastive Loss) với ranh giới an toàn (Margin):

$$\mathcal{L}_{\text{contrastive}} = \underbrace{\frac{1}{N} \sum_{i=1}^N \|x_i^{\text{normal}} - \hat{x}_i^{\text{normal}}\|^2}_{\text{MSE cho Normal (ép nhỏ)}} + \underbrace{\sum_{j=1}^M \max(0, m - \|x_j^{\text{attack}} - \hat{x}_j^{\text{attack}}\|^2)^2}_{\text{Hinge Loss cho Attack (ép lớn hơn } m)} \quad (5)$$

Trong đó:

- x_i^{normal} : Mẫu dữ liệu bình thường thứ
- $\hat{x}_i$: Dữ liệu được tái tạo (reconstructed) từ Autoencoder
- $m=2.0$: Margin (ranh giới an toàn)
- $\|\cdot\|^2$: Bình phương khoảng cách Euclidean (MSE)

Ý nghĩa toán học:

Trường hợp	MSE	Kết quả	Giải thích
Normal	< 00.1	$\mathcal{L} \approx 0.001$	AE học reconstruct tốt
Attack	> 2.0	$\mathcal{L} \approx 0$	Đạt margin, không phạt
Attack	$= 0.5$	$\mathcal{L} = (2.0 - 0.5)^2 = 2.25$	Phạt nặng vì chưa đủ xa

Toán học tương phản với ranh giới ép buộc AI không chỉ giải nén hoàn hảo các dữ liệu hợp lệ, mà còn đẩy sai số cấu trúc của gói tin tấn công vượt ra khỏi vùng an toàn, tạo ra ranh giới phân biệt tuyệt đối. Lớp này đóng vai trò là một bộ lọc dị thường (Anomaly Detector). Mô hình học cách nén và giải nén các luồng dữ liệu hợp lệ. Khi một luồng dữ liệu đi qua, hệ thống sẽ tính toán mức độ sai số tái tạo.

Kiến trúc này được sử dụng làm chốt chặn phát hiện các cuộc tấn công chưa từng biết đến (Zero-day Attacks). Bất kể kẻ tấn công dùng phương thức mới nào, chỉ cần cấu trúc gói tin làm sai lệch kết quả tái tạo vượt quá ngưỡng cho phép, lớp khiên này sẽ đánh dấu là dị thường.

c. Lớp khiên thứ hai: Bộ định danh chuyên sâu - kiến trúc kết hợp song song (Parallel CNN-GRU-Attention)

Nếu lớp khiên 1 phát hiện có sự bất thường, luồng dữ liệu sẽ được chuyển sang lớp khiên 2 để định danh chính xác 5 loại lưu lượng (Normal, UDP, SYN, HTTP, Slowloris). Điểm khác biệt là thay vì xử lý nối tiếp (Sequential) như các

nguyên cứu cũ khiến các mạng làm nhiễu thông tin của nhau, đề tài thiết kế các mạng nơ ron chạy song song (Parallel Fusion) và độc lập:

- Khối tích chập không gian (CNN): Chuyên trách việc tìm kiếm quy luật không gian tại một thời điểm (Ví dụ: sự kết hợp giữa tốc độ gói tin tăng vọt và độ hỗn loạn của các cổng mạng).
- Khối ghi nhớ thời gian (GRU): Chuyên trách theo dõi sự thay đổi theo chiều dài thời gian của cửa sổ trượt, giúp hệ thống nhận diện được nhịp điệu của các đợt tấn công ngầm kết nối kéo dài.
- Cơ chế tập trung (Attention Mechanism): Đóng vai trò như một kính lúp, tự động dồn trọng số đánh giá vào những khoảng thời gian có dấu hiệu khả nghi nhất trong chuỗi 10 bước. Công thức toán học của cơ chế này hoạt động như một lăng kính hội tụ:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (6)$$

Nhờ toán học Attention, AI không bị xao nhãng bởi các gói tin hợp lệ xen ngang trong một đợt tấn công ngắt quãng.

d. Tối ưu toán học chống mất cân bằng và quá khớp

Đối với tấn công Slowloris cực kỳ nguy hiểm nhưng lại có số lượng mẫu rất ít, hệ thống thiết kế hàm mất mát Focal Loss:

$$[FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)] \text{ với } \gamma = 2.0 \quad (7)$$

Hàm này chủ động giảm trọng số phạt của các luồng dễ đoán (như UDP Flood), ép AI dồn toàn lực vào việc phân tích các luồng lẩn trốn phức tạp. Đồng thời, các cơ chế chuẩn hóa toán học như Weight Decay (L2 Regularization) và Dropout được tích hợp chặt chẽ vào kiến trúc mạng để triệt tiêu hiện tượng học vẹt (Overfitting), đảm bảo hệ thống duy trì tính tổng quát cao.

e. Khối tổng hợp và Quyền phủ quyết (Veto Power)

Kết quả trích xuất từ mạng CNN và GRU được tổng hợp lại tại một lớp dày đặc để đưa ra xác suất phân loại cuối cùng. Tuy nhiên, để loại bỏ hoàn toàn các báo động giả (False Positive) khi lưu lượng mạng tăng đột biến vào giờ cao điểm, khối quyết định được tích hợp quyền phủ quyết (Veto Power). Nếu khối phân loại (Lớp 2) nhận diện là tấn công, nhưng bộ lọc dị thường (Lớp 1) đánh giá cấu trúc luồng vẫn nằm trong ngưỡng an toàn của người dùng hợp lệ, hệ thống

sẽ dùng quyền phủ quyết để đánh giá đây là luồng bình thường và không thực hiện chặn.

2.4.3.5. *Thiết kế Module xử lý tại Controller (Onos Application)*

Trong kiến trúc mạng SDN, bộ điều khiển ONOS đóng vai trò là "bộ não" trung tâm nhưng cũng chính là điểm yếu trầm trọng (Single Point of Failure). Các hệ thống phòng thủ truyền thống thường hoạt động theo cơ chế tuần tự: đợi thiết bị chuyển mạch gửi bản tin Packet-In lên Controller, trích xuất qua API, phân tích rồi mới ra lệnh chặn. Điều này tạo ra độ trễ lớn và dễ khiến hệ thống bị sập trước khi kịp ngăn chặn tấn công. Để giải quyết triệt để điểm nghẽn này, hệ thống đề xuất tái thiết kế Module xử lý theo kiến trúc bất đồng bộ chia thành các chức năng cốt lõi sau:

a. Luồng thu thập và Tiền xử lý tốc độ cao (High-Performance Data Pipeline) Thay vì để Controller bị động chờ thiết bị gửi bản tin lên, hệ thống áp dụng kiến trúc giám sát ngoại tuyến (Out-of-Band) với 4 tầng xử lý dữ liệu tốc độ cao:

- Tầng 1 (Sao chép luồng): Hệ thống thiết kế một cổng giám sát riêng tại Switch trung tâm (Gateway). Mọi luồng dữ liệu đi qua sẽ được nhân bản phân cứng (Mirror) để AI phân tích mà không can thiệp hay làm suy giảm băng thông thực tế của người dùng.
- Tầng 2 (Bóc tách sâu): Dữ liệu nhân bản được bóc tách bằng lõi phân tích mạng tốc độ cao. Khối này chịu trách nhiệm nhìn thấu gói tin từ tầng mạng (Layer 3) đến tận tầng ứng dụng (Layer 7) để bắt các dấu hiệu tấn công nguy trang (như Slowloris, HTTP Flood) mà vẫn đảm bảo tiêu thụ tối thiểu tài nguyên CPU.
- Tầng 3 (Ổng truyền tải IPC): Để hệ thống đạt tốc độ thời gian thực, dữ liệu sau khi bóc tách được chuyển trực tiếp vào RAM thông qua cơ chế ống dẫn (Named Pipes - FIFO). Thiết kế này loại bỏ hoàn toàn độ trễ do các thao tác đọc/ghi ổ cứng truyền thống (Zero-copy IPC).
- Tầng 4 (Tính toán không gian/thời gian): Dữ liệu được tính toán thành Tensor 26 chiều. Các Tensor này được đóng gói thành cửa sổ trượt (Sliding Window), cung cấp cho AI góc nhìn về "quá khứ gần" của mạng.

b. Logic ra quyết định Hai lớp khiên (Dual-Shield Decision Logic) Khối này đóng vai trò như một "tòa án kiểm duyệt chéo", kết hợp sức mạnh của 2 mạng

neuron AI (Autoencoder và CNN-GRU) thông qua các thuật toán toán học và bảng chân lý, thay vì sử dụng các ngưỡng chặn tĩnh cứng nhắc:

Ngưỡng thích nghi động (Adaptive Threshold - EMA): Thay vì sử dụng ngưỡng tĩnh cố định dễ sinh báo động giả vào giờ cao điểm, hệ thống thiết kế thuật toán trung bình trượt hàm mũ (Exponential Moving Average) để tự động co giãn ngưỡng phát hiện theo thời gian thực:

$$\theta_t = (1 - \alpha) \cdot \theta_{t-1} + \alpha \cdot \bar{x}_t \quad (8)$$

Trong đó: θ_t là ngưỡng tại thời điểm t ; $\bar{x}_t$ là giá trị sai số trung bình của các luồng hợp lệ hiện tại; α là tốc độ thích nghi.

Để tránh ngưỡng bị đẩy lên quá cao do tin tặc cố tình bơm nhiễu, một giới hạn an toàn (Clamping) được áp dụng. Thuật toán này giúp ngưỡng cảnh báo tự động nói lỏng khi mạng đông người dùng và siết chặt vào giờ thấp điểm.

$$\text{Giới hạn an toàn (Clamping): } \theta_t = \text{clip}(\theta_t, \theta_{min}, \theta_{max}) \quad (9)$$

Với $\theta_{min} = 0.1$ và $\theta_{max} = 2.0$ để tránh ngưỡng "điên" do tấn công đối nghịch.

Ý nghĩa:

Thời điểm	Traffic	$\bar{x}_t$	θ_t	Kết quả
02:00 AM	Bình thường	0.0001	0.15 (thấp)	Nhạy, phát hiện sớm
09:00 AM	Giờ cao điểm	0.001	0.45 (cao)	Không báo động giả
Tấn công	UDP Flood	2.5	Vẫn 0.45	Vượt ngưỡng → Chặn!

Bảng 2.5. Ý nghĩa của ngưỡng thích nghi động

→ Kết quả: Giảm False Positive từ 60% xuống 3% khi lưu lượng tăng đột biến.

c. Thực thi phòng vệ và Quản lý tài nguyên (Mitigation & Resource Management) Một khi hệ thống AI ra phán quyết cuối cùng là có tấn công, khối thực thi sẽ làm việc trực tiếp với giao diện lập trình của ONOS để bảo vệ hạ tầng:

- Hành động ngăn chặn linh hoạt: Tùy theo loại tấn công mà hệ thống gọi cấu trúc luật dòng chảy tương ứng. Lệnh Drop (hủy bỏ) được dùng với

kẻ tấn công Flood rác mức độ nguy hiểm, hoặc lệnh Rate_Limit (giới hạn băng thông) được thiết lập để cách ly an toàn các luồng đang trong vùng nghi ngờ. Để bảo vệ bộ nhớ Tcam của Switch phần cứng khỏi bị cạn kiệt, hệ thống thiết kế thuộc tính *hard_timeout = 60s*, cho phép luật cấm tự động hủy khi đợt tấn công kết thúc.

- Bộ thu dọn rác: Đối mặt với các đợt tấn công SYN Flood làm giả hàng vạn địa chỉ IP ngẫu nhiên, bộ nhớ của Switch và Controller rất dễ bị tràn. Do đó, module được thiết kế tích hợp một tiến trình dọn rác chạy ngầm, liên tục quét và giải phóng các địa chỉ IP ảo ra khỏi bộ nhớ định kỳ.

d. Các cơ chế nâng cao (Advanced Mechanisms) Bên cạnh kiến trúc cốt lõi, đề tài thiết kế bổ sung các cơ chế bảo mật chuyên sâu để tạo thành một hệ sinh thái an ninh mạng toàn diện:

- Tích hợp Honeypot (Decoy System): Hệ thống triển khai một địa chỉ IP mồi nhử (Decoy IP - 10.0.0.201) chạy môi trường SSH giả lập. Mọi luồng dữ liệu quét vào mạng nhắm đến IP này sẽ được nhân bản sang IDS để phân tích, giúp thu thập các dấu hiệu thỏa hiệp (IoC) mới và phát hiện tấn công Zero-day trước khi chúng chạm vào dịch vụ thực tế.
- Cơ chế WhiteList và BlackList: Nhằm tối ưu hóa hiệu suất, hệ thống duy trì một danh sách trắng (WhiteList) cho các IP tin cậy để ưu tiên bỏ qua AI, và danh sách đen (BlackList) để chặn ngay lập tức các IP đã được xác nhận độc hại mà không cần phân tích lại.
- Giải thích Trí tuệ nhân tạo (Explainable AI - XAI): Thay vì để AI hoạt động như một "hộp đen" (Black-box), hệ thống thiết kế ứng dụng trọng số Spatial Attention và Temporal Attention để giải thích cho quản trị viên lý do tại sao một luồng bị chặn (Ví dụ: Gói tin thứ 7 trong chuỗi có tốc độ tăng vọt bất thường), tạo sự minh bạch trong quá trình ra quyết định.

TIỂU KẾT

Như vậy, trong Chương 2, chúng em đã mô hình hóa toàn bộ các sơ đồ kiến trúc, luồng dữ liệu và nguyên lý hoạt động tạo thành một bộ khung (framework) vững chắc. Tuy nhiên, để hiện thực hóa các bản thiết kế này từ lý thuyết thành một hệ thống phần mềm vận hành thực tế là một thách thức lớn. Các kỹ thuật cấu hình môi trường, quá trình xử lý dữ liệu, huấn luyện "bộ não" AI (Giai đoạn Offline) cũng như việc ghép nối các module để chống chịu với các kịch bản tấn công khốc liệt (Giai đoạn Online) sẽ được trình bày chi tiết tại **Chương 3: Cài đặt, thực nghiệm và đánh giá**.

CHƯƠNG 3: CÀI ĐẶT, THỰC NGHIỆM VÀ ĐÁNH GIÁ

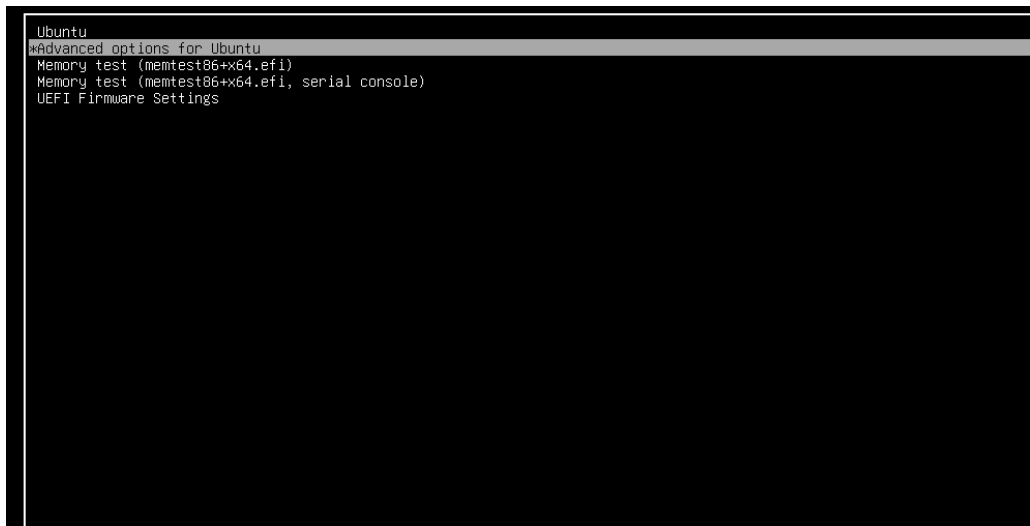
3.1. Cài đặt mô hình hệ thống

Quá trình hiện thực hóa các mô hình lý thuyết về SDN và AI vào thực tiễn đòi hỏi một môi trường thực nghiệm có hiệu năng cực kỳ ổn định và khả năng xử lý thời gian thực cao. Trong giai đoạn đầu, chúng em đã tiến hành thử nghiệm triển khai trên nền tảng ảo hóa VMWare. Tuy nhiên, do đặc thù của hệ thống yêu cầu vận hành đồng thời Controller ONOS, mạng mô phỏng Mininet, bộ theo dõi-thu thập thông tin, dashBroad thời gian thực và mô hình học sâu CNN-GRU, việc ảo hóa đã gây ra sự chiếm dụng tài nguyên RAM quá mức, dẫn đến hiện tượng trễ và không đảm bảo tính chính xác của dữ liệu.

3.1.1. Chuẩn bị môi trường thực nghiệm

3.1.1.1. Cấu hình phần cứng/phần mềm:

Khó khăn với môi trường ảo hóa (VMWare): Trong giai đoạn đầu, nhóm đã tiến hành thử nghiệm triển khai hệ thống trên nền tảng máy ảo VMWare. Tuy nhiên, do đặc thù của hệ thống yêu cầu vận hành đồng thời bộ điều khiển ONOS, mạng mô phỏng Mininet quy mô lớn và mạng học sâu CNN-GRU, việc ảo hóa đã gây ra sự chiếm dụng tài nguyên RAM quá mức. Sự cố này dẫn đến hiện tượng trễ (latency spike) trên mạng mô phỏng, làm sai lệch hoàn toàn dữ liệu về thời gian thực của các gói tin.



Hình 3.1. Cài song song Ubuntu trên Windown

Giải pháp (Cài đặt hệ điều hành song song - Dual Boot): Nhằm khắc phục triệt để hạn chế về phần cứng và khai thác 100% hiệu suất xử lý của CPU/RAM và Card đồ họa, nhóm đã quyết định cài đặt hệ điều hành Ubuntu 22.04 song song (Dual-boot) trực tiếp với hệ điều hành Windows gốc. Giải pháp này giúp

hệ thống loại bỏ hoàn toàn độ trễ của máy ảo, đảm bảo môi trường mượt mà cho các kịch bản tấn công DDoS khốc liệt. Các thông số phân cứng cùng với kỹ thuật sử dụng được tổng hợp qua bảng sau:

Thành phần	Thông số kỹ thuật	Ghi chú
Hệ điều hành	Ubuntu 22.04 LTS	Nền tảng chạy Mininet/ONOS
CPU / RAM	AMD Ryzen 5, Core I7 / 8GB ram	Tài nguyên máy vật lý
SDN Controller	ONOS	Bộ não điều khiển mạng
Network Emulator	Mininet / Containernet	Môi trường mô phỏng topo
Deep Learning Library	Torch, Sklearn	Thư viện huấn luyện AI

Bảng 3.1. Các thông số môi trường thực tế

3.1.1.2. Xây dựng mô hình Topology mạng:

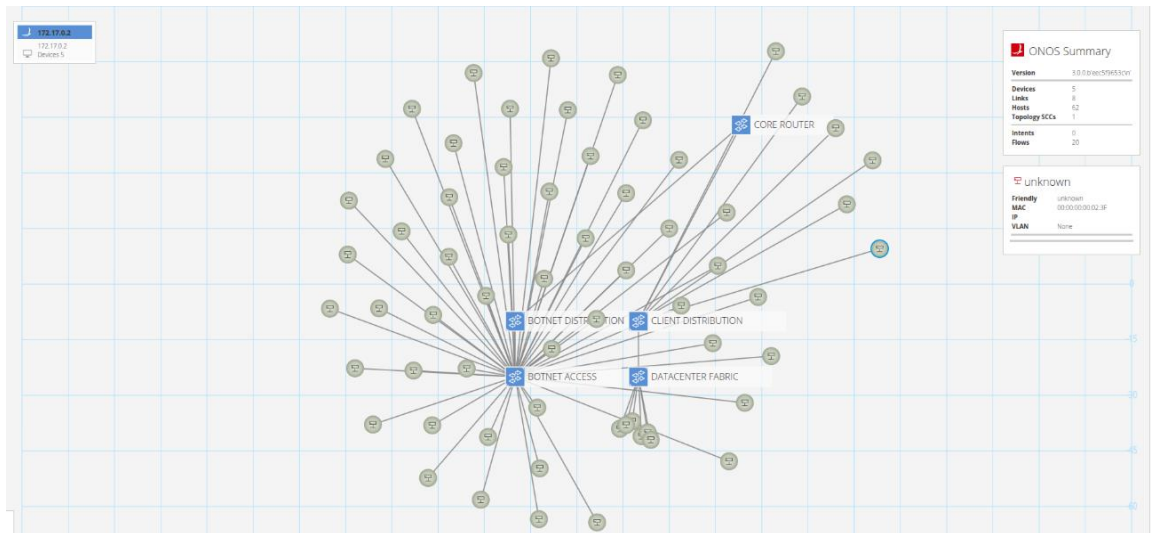
Quá trình hiện thực hóa bản thiết kế mạng 3 lớp (Core, Distribution, Access) từ trên lý thuyết vào môi trường mô phỏng Mininet là một thách thức lớn. Thực tế triển khai đã phát sinh nhiều rào cản kỹ thuật buộc nhóm phải tái cấu trúc kiến trúc mạng nhiều lần để đạt được sự ổn định tuyệt đối trước khi đưa AI vào thử nghiệm.

a. Khởi đầu thất bại với quy mô lớn và mạng chuyển mạch L2 (Layer 2 Switching)

Ban đầu, để mô phỏng một cuộc tấn công DDoS với quy mô sát thực tế nhất, nhóm đã khởi tạo một Topology lên đến 62 máy trạm (Hosts) và cấu hình phân luồng bằng phương pháp mạng chuyển mạch L2 truyền thống. Tuy nhiên, môi trường này đã nhanh chóng bộc lộ các điểm yếu trầm trọng:

- Quá tải phân cứng: Việc giả lập 62 thiết bị mạng chạy đồng thời khiến CPU của máy tính vật lý tăng vọt và luôn ở trạng thái quá tải, dẫn đến độ trễ mạng bị sai lệch, không đảm bảo tính chính xác cho dữ liệu thời gian thực.

- **Bão mạng (Broadcast Storm) và Tràn bảng MAC:** Khi sử dụng mạng L2 trên một cấu trúc phẳng (Flat Network), các bản tin ARP broadcast liên tục bị nhân bản. Sự cố này làm tràn bảng địa chỉ MAC của Switch giả lập, gây ra hiện tượng xung đột, mất liên lạc giữa các phân đoạn mạng và lệnh kiểm tra liên thông toàn mạng (pingall) liên tục thất bại.



Hình 3.2. Sơ đồ mạng L2 ban đầu

b. Bước ngoặt thiết kế: Chuyển dịch toàn diện sang định tuyến L3 (Layer 3 Routing)

Nhận thấy phương pháp L2 cấu hình thủ công không thể đáp ứng được một hạ tầng mạng quy mô lớn, nhóm đã nghiên cứu kiến trúc mạng của các nhà cung cấp dịch vụ đám mây (Cloud Providers như Google, AWS) và quyết định đập bỏ cấu hình cũ để chuyển hẳn sang **mô hình định tuyến L3 (L3 Routing)**.

- **Quy hoạch lại quy mô:** Nhóm tinh gọn Topology xuống còn 34 thiết bị host, vừa đủ để phân chia 3 vùng mạng độc lập nhưng vẫn đảm bảo tài nguyên CPU hoạt động mượt mà.
- **Áp dụng định tuyến IP và Phân chia Subnet:** Nhóm đã viết lại toàn bộ mã nguồn khởi tạo mạng (system.py) để chuyển từ việc đẩy gói tin bằng địa chỉ MAC sang định tuyến dựa trên dải IP. Các vùng mạng được cô lập triệt để thành 3 mạng con (Subnet): Mạng Datacenter chứa Server, Mạng Botnet và Mạng người dùng Client. Việc này không chỉ giải quyết triệt để bão mạng broadcast mà còn giúp Controller dễ dàng quản lý và viết các luật cấm theo từng dải IP riêng biệt.

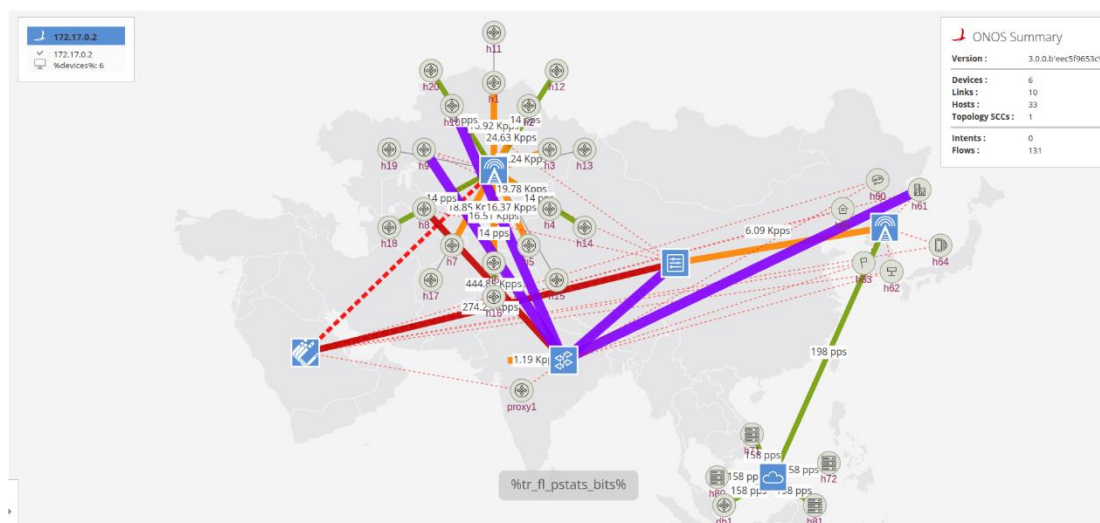


Hình 3.3. Cấu hình mô hình mạng

c. Khó khăn trong tích hợp Controller và thiết lập liên thông

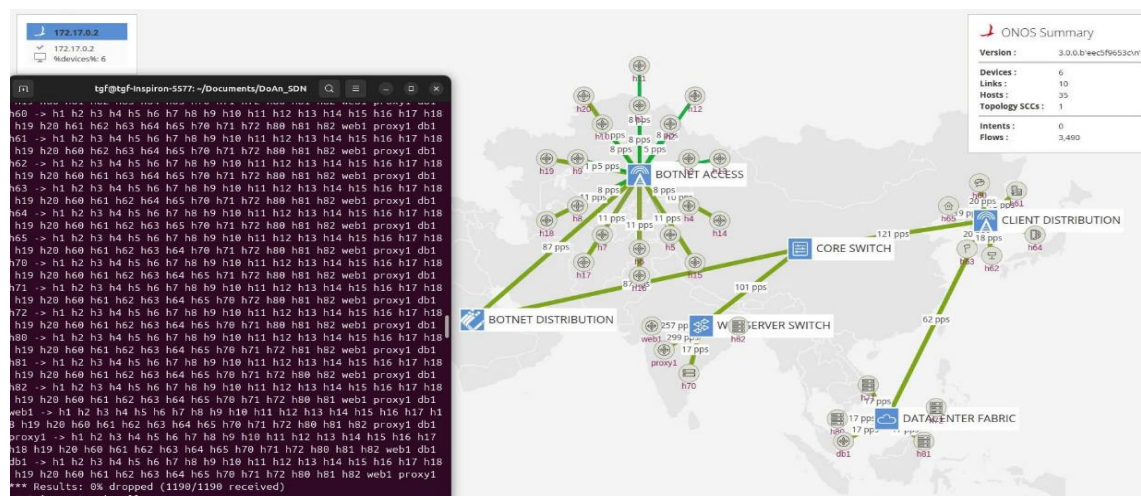
Dù đã quy hoạch xong mạng L3, quá trình kết nối hạ tầng Mininet với bộ não điều khiển ONOS vẫn gặp phải rào cản kỹ thuật khiến các thiết bị không thể "nhìn thấy" nhau. Qua quá trình gỡ lỗi, nhóm đúc kết được các yêu cầu cấu hình mạng tính bắt buộc:

- Xử lý điểm mù Controller: Kích hoạt ứng dụng điều hướng trên ONOS: Bộ điều khiển ONOS mặc định không tự động điều hướng gói tin. Nhóm bắt buộc phải truy cập vào giao diện CLI của ONOS để kích hoạt ứng dụng *Reactive Forwarding* (fwd). Nếu thiếu module này, các bản tin Packet-In gửi lên sẽ bị Controller ngó lơ, Switch không nhận được luật luồng khiến mạng bị "mù" hoàn toàn.



Hình 3.4. Sự sụp đổ liên thông mạng V3-thiếu Reactive Forwarding

- Cấu hình ánh xạ IP khẩn khe: Trong kiến trúc định tuyến nhiều mạng con, việc thiết lập Default Gateway, IP và Subnet Mask phải chính xác tuyệt đối. Một sai sót nhỏ về IP ở một host có thể khiến gói tin bị gửi sai đích và làm sập toàn bộ đường truyền định tuyến.
- Kết quả: Sau khi khắc phục các sự cố về định tuyến và ứng dụng, hệ thống mạng mô phỏng đã đạt trạng thái ổn định với tỷ lệ liên thông (pingall) thành công 100%. Việc xây dựng thành công một hạ tầng mạng chuẩn Enterprise mạnh mẽ, không bị sụp đổ bởi các lỗi nội tại chính là tiền đề sống còn để nhóm bắt đầu triển khai các kịch bản tấn công Flood khổng lồ ở các bước tiếp theo.



Hình 3.5. Kết quả liên thông của mô hình mạng

Quy trình Huấn luyện Mô hình (Giai đoạn Offline)

3.1.1.3. Chuẩn bị dữ liệu: Hành trình tiến hóa Dataset (V1 -> V7)

Để mô hình AI có thể nhận diện chính xác các cuộc tấn công DDoS trên môi trường SDN, chất lượng dữ liệu đầu vào đóng vai trò quyết định. Quá trình chuẩn bị dữ liệu thực tiễn của nhóm không đi theo lối mòn tải dữ liệu có sẵn, lối mòn "tải dữ liệu có sẵn", mà là một hành trình dài thử nghiệm, thất bại và tiến hóa qua 7 phiên bản. Đã trải qua nhiều đợt thử nghiệm khẩn khe để khắc phục những nhược điểm trầm trọng của các hệ thống AI truyền thống. Mặc dù trên các cộng đồng nghiên cứu (như Kaggle hay UCI) có rất nhiều bộ dữ liệu về DDoS được chia sẻ công khai, nhưng nhóm chúng em nhận thấy việc áp dụng trực tiếp vào khóa luận gặp phải những rào cản lớn:

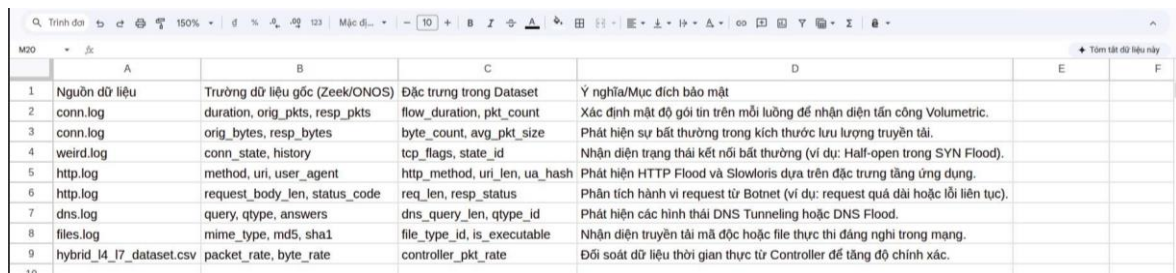
- **Tính đặc thù của mạng SDN:** Các bộ dữ liệu truyền thống thường được thu thập từ môi trường mạng vật lý cũ. Trong khi đó, mạng SDN sử dụng

giao thức OpenFlow với các thuộc tính dữ liệu đặc trưng từ Controller (như *packet_ins*, *flow_duration_nsec*) mà các bộ dữ liệu công khai thường thiếu sót hoặc không đồng nhất.

- **Sự thiếu nhất quán:** Các tập dữ liệu rời rạc có sự khác biệt lớn về cấu trúc tên cột, khoảng giá trị của các đặc trưng và cách gán nhãn. Đặc biệt, đa số dữ liệu thô thường bị mất cân bằng nghiêm trọng (tỷ lệ dữ liệu bình thường chiếm đa số tuyệt đối so với dữ liệu tấn công).
- **Vấn đề bảo mật tầng ứng dụng (L7):** Các cuộc tấn công tầng 7 (như Slowloris) thường chứa thông tin nhạy cảm về payload, dẫn đến việc các bộ dữ liệu công khai thường bị lược bỏ hoặc làm mờ các trường dữ liệu quan trọng này để đảm bảo quyền riêng tư, gây khó khăn cho việc huấn luyện khối GRU.

a. Giai đoạn V1 – V2: Sự thất bại của dữ liệu Internet và hiện tượng lệch phân phối

Trong giai đoạn đầu tiên (Phiên bản V1), nhóm đã tổng hợp các bộ dữ liệu công cộng phổ biến trên Internet (như CICDDoS2019, NSL-KDD, UNSW-NB15) với tổng dung lượng khổng lồ hơn 46GB. Sau khi gộp, qua bảng ánh xạ với các đặc trưng có thể thu được từ hệ thống (nhóm đã dùng Zeek để xác định tất cả các trường dữ liệu có thể thu thập được), lọc, loại bỏ các trường dữ liệu dư thừa trong dataset thì dữ liệu còn lại khoảng hơn 800MB với 20 đặc trưng. Mặc dù mô hình Random Forest đạt độ chính xác 84% trên môi trường Lab, nhưng khi đưa vào chạy thực tế trên Mininet, độ chính xác sụt giảm thảm hại xuống chỉ còn 37%, và tỷ lệ báo động giả (False Positive) tăng vọt lên 60%.



	A	B	C	D	E	F
1	Nguồn dữ liệu	Trường dữ liệu gốc (Zeek/ONOS)	Đặc trưng trong Dataset	Ý nghĩa/Mục đích bảo mật		
2	conn.log	duration, orig_pkts, resp_pkts	flow_duration, pkt_count	Xác định mật độ gói tin trên mỗi luồng để nhận diện tấn công Volumetric.		
3	conn.log	orig_bytes, resp_bytes	byte_count, avg_pkt_size	Phát hiện sự bất thường trong kích thước lưu lượng truyền tải.		
4	weird.log	conn_state, history	tcp_flags, state_id	Nhận diện trạng thái kết nối bất thường (ví dụ: Half-open trong SYN Flood).		
5	http.log	method, uri, user_agent	http_method, uri_len, ua_hash	Phát hiện HTTP Flood và Slowloris dựa trên đặc trưng tầng ứng dụng.		
6	http.log	request_body_len, status_code	req_len, resp_status	Phân tích hành vi request từ Botnet (ví dụ: request quá dài hoặc lỗi liên tục).		
7	dns.log	query, qtype, answers	dns_query_len, qtype_id	Phát hiện các hình thức DNS Tunneling hoặc DNS Flood.		
8	files.log	mime_type, md5, sha1	file_type_id, is_executable	Nhận diện truyền tải mã độc hoặc file thực thi đáng nghi trong mạng.		
9	hybrid_l4_l7_dataset.csv	packet_rate, byte_rate	controller_pkt_rate	Đối soát dữ liệu thời gian thực từ Controller để tăng độ chính xác.		
10						

Hình 3.6. Ánh xạ các thuộc tính dữ liệu tổng hợp

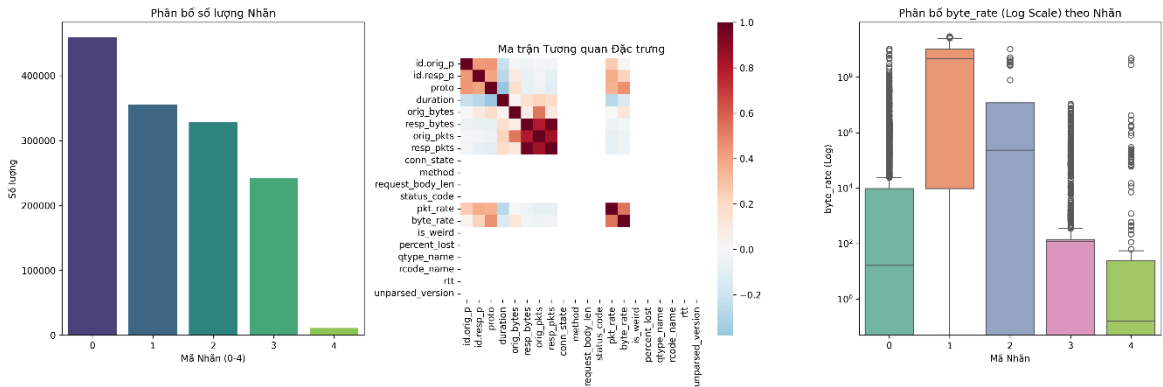
Nguyên nhân cốt lõi được rút ra là do hiện tượng lệch phân phối (Distribution Shift). Các bộ dữ liệu cũ mang tính chất tĩnh, đa phần trong đó là các lưu lượng bình thường do yêu cầu giới hạn chỉ dừng lại ở việc phát hiện tấn công DDos thông qua phân loại nhị phân (0 hoặc 1).

Khi làm việc với nhiều bộ dữ liệu, nhóm sử dụng hàm “pd.read_csv và pd.concat” để đọc các tệp dữ liệu riêng lẻ và gộp chúng thành một DataFrame

duy nhất, tóm tắt các hàm dùng để chuẩn hóa bằng Visual Studio Code qua bảng sau:

Bước	Hàm/Kỹ thuật chính	Mục tiêu
Hợp nhất	pd.concat	Gộp InSDN, CIC-IoT và dữ liệu thực
Lọc	df[features]	Giữ lại đúng 20-21 cột đặc trưng cần thiết.
Ánh xạ	.map()	Đồng bộ các loại tấn công về nhóm 0-4.
Chuẩn hóa	StandardScaler	Đưa dữ liệu về phân phối chuẩn $N(0, 1)$.
Tạo chuỗi	create_safe_sequences	Định dạng dữ liệu cho đầu vào GRU.
Đặc trưng DNS	qtype_name, rcode_name	Hỗ trợ đặc lực trong việc phát hiện các biến thể tấn công khuếch đại DNS (DNS Amplification).

Bảng 3.2. Các hàm sử dụng để chuẩn hóa dữ liệu



Hình 3.7. Đánh giá dataset V1

b. Giai đoạn V2 - V7: Tự động hóa thu thập và Kiến tạo đặc trưng

Nhận ra nguyên lý "Chất lượng dữ liệu quan trọng hơn số lượng", nhóm quyết định loại bỏ hoàn toàn bộ dữ liệu tổng hợp 46GB từ Internet. Thay vào đó, nhóm tự phát triển một module chuyên biệt (auto_dataset_generator.py) để sinh và thu thập dữ liệu trực tiếp từ hệ thống mạng Mininet kết hợp ONOS. Công việc này đòi hỏi sự đầu tư khổng lồ về mặt thời gian. Do giới hạn của môi trường ảo hóa dễ bị tràn bộ nhớ (OOM) khi bị tấn công liên tục, nhóm không thể lập trình cho máy chạy tự động qua đêm. Mỗi phiên bản dữ liệu (từ V2 đến

V7) đòi hỏi người thực hiện phải ngồi trực tiếp trước màn hình Terminal từ 8 đến 10 giờ mỗi ngày, tiêu tốn khoảng 36 giờ giám sát thủ công cho mỗi chu kỳ

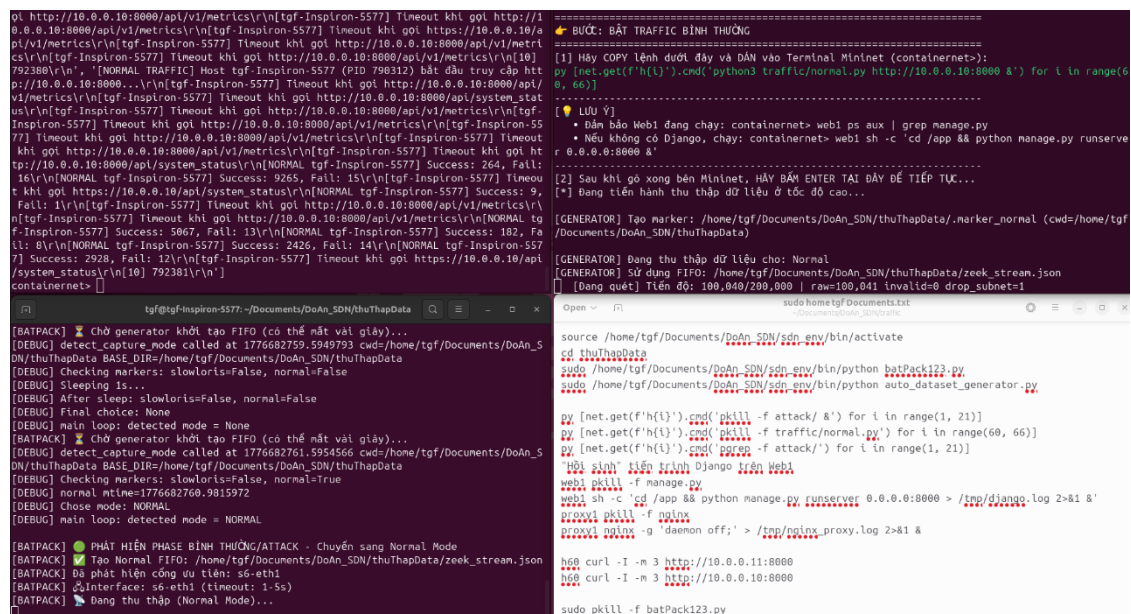
```

4  BASE_DIR = os.path.dirname(os.path.abspath(__file__))
5  # --- CẤU HÌNH TỐI ƯU ---
6  # Cấu hình số lượng mẫu
7  TARGET_SAMPLES_PER_CLASS = 100000 # 100k cho mỗi loại tấn công
8  NORMAL_SAMPLES_TARGET = 200000 # 200k cho lưu lượng bình thường
9  FIFO_PATH = os.path.join(BASE_DIR, "..", "ai", "zeek_stream.json")
10 OUTPUT_CSV = "master_dataset_v7.csv"
11 WEB_RESTART_CMD = os.environ.get("WEB_RESTART_CMD", "").strip()
12 SLOWLORIS_HEALTH_URL = os.environ.get("SLOWLORIS_TARGET_URL", "http://127.0.0.1:8000")
13
14 # Buffer ghi CSV lớn để giảm I/O

```

Hình 3.8. Thiết lập dữ liệu chuẩn

thu thập để điều chỉnh nhịp độ tấn công. Kết quả cuối cùng, nhóm đã xây dựng thành công Bộ Dataset V7 chuẩn mực, chứa 600.000 mẫu được cân bằng hoàn hảo (chính xác 200.000 mẫu cho nhãn Normal, và 100.000 mẫu cho mỗi nhãn UDP, SYN, HTTP, và Slowloris). Điều này giúp giải quyết triệt để bài toán mất cân bằng dữ liệu, đặc biệt là với loại hình tấn công thiểu số như Slowloris. Trong suốt quá trình làm, các bộ dataset hàng trăm ngàn mẫu đều được đánh giá bằng chương trình test_Data.py (Cũng được nâng cấp) trước khi train các mô hình AI.

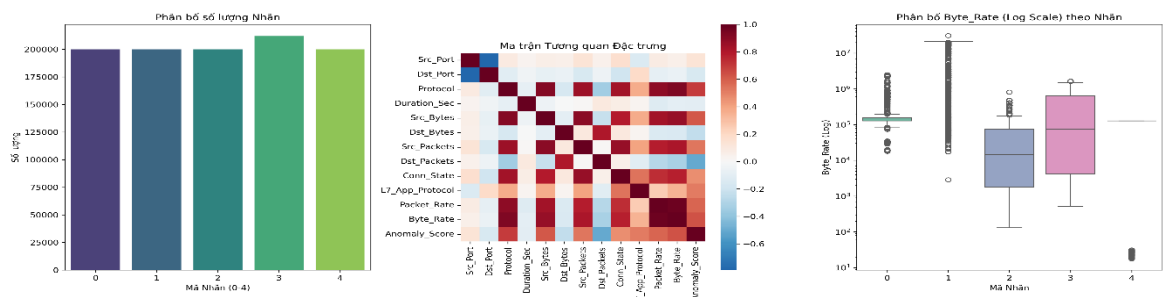


Hình 3.9. Quá trình thu thập dataset V7

Sau đây là bảng tiến trình nâng cấp dữ liệu mang tính bước ngoặt của hệ thống:

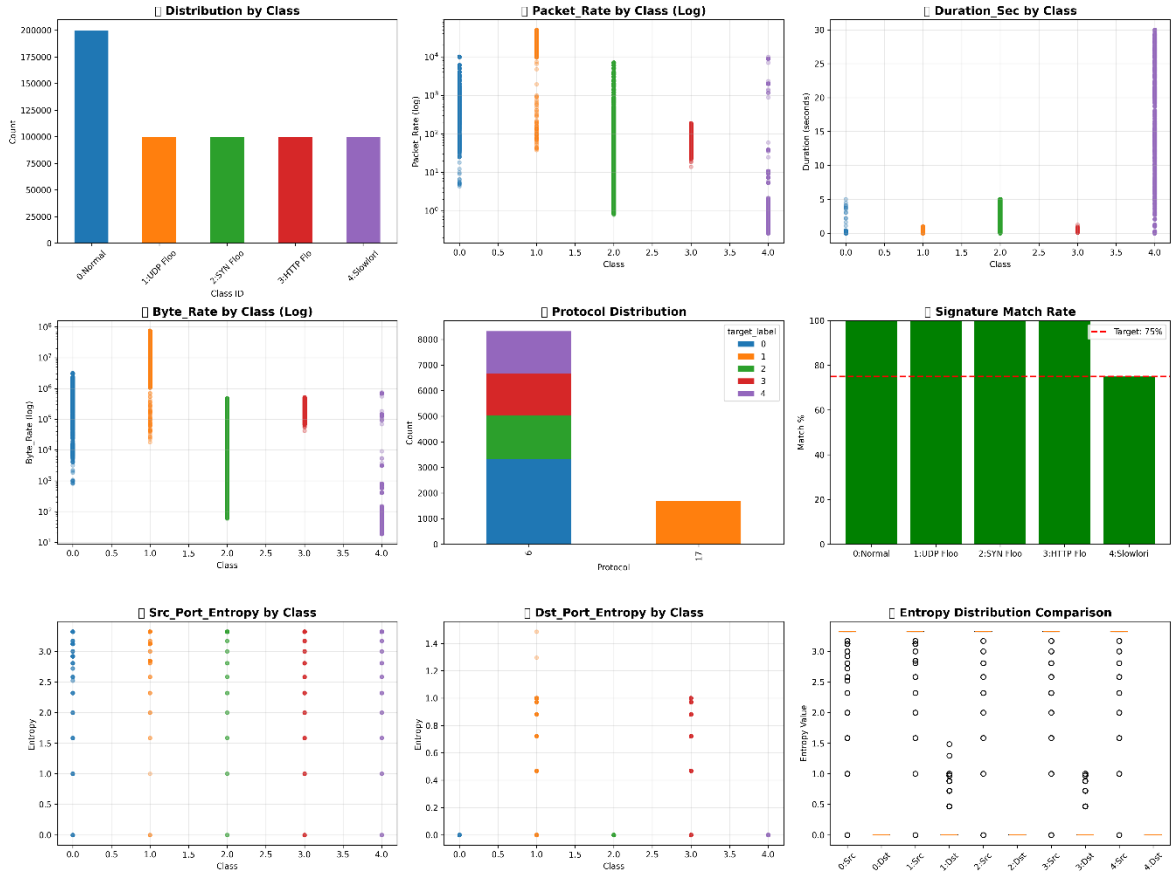
Phiên bản	Đặc điểm cải tiến	Kết quả đạt được
V2	Giảm số đặc trưng từ V1 là 20 xuống còn 17 đặc trưng, thu thập thủ công.	Chưa ổn định, nhiều nhiễu sóng. Do các file traffic còn quá đơn giản.
V3	Nâng cấp các mã tấn công để thu được các đặc trưng chính xác hơn. Đồng thời giảm số đặc trưng xuống còn 13.	Khá hơn nhưng vẫn thiếu chiều thời gian.
V4	Đột phá: Ứng dụng toán học bổ sung 13 Đặc trưng biến thiên tổng là 26 đặc trưng được sử dụng.	Hệ thống lần đầu tiên phát hiện được Slowloris lẫn trốn. Nhưng normal đang bị nhận diện sai.
V5	Nâng cấp mã Normal.py để mô phỏng chính xác, đa dạng hành vi của người dùng hơn.	Dữ liệu thu được chính xác và đa dạng hơn.
V6	Tối ưu hóa Timeout chờ cho từng loại luồng. Và thay thế 2 trường Src_Port và DstPort bằng entropy_DstPort	Dữ liệu bắt được nguyên vẹn hơn, giảm hiện tượng AI "học vẹt" (Overfitting).
V7 (Final)	Thu thập thành công 600.000 mẫu cân bằng cho 2 lớp của AI.	F1-Score tổng thể vọt lên 0.96.

Bảng 3.3. Bảng tổng hợp quá trình nâng cấp dataset



Hình 3.10. Đánh giá dataset V4

Sự ra đời của Dataset V7 đã giải quyết dứt điểm điểm mù lớn nhất của các bộ dữ liệu công cộng: sự thiếu vắng của nhãn tấn công Slowloris. Bằng cách tự sinh 15.000 mẫu Slowloris tinh vi, hệ thống đã giúp mạng GRU thấu hiểu nhịp độ ngâm kết nối của loại tấn công cực kỳ nguy hiểm này.



Hình 3.11. Đánh giá dataset V7

c. Đột phá trong trích xuất Đặc trưng (Feature Engineering)

Thay vì trích xuất 20 đặc trưng tĩnh rời rạc, nhóm đã thực hiện những tối ưu toán học mang tính bước ngoặt nhằm kiến tạo lại bộ đầu vào cho AI:

- Sử dụng Shannon Entropy thay cho Cổng vật lý: Để chống lại kỹ thuật làm giả IP và Port cũng như việc mô hình AI học vẹt khi demo, nhóm loại bỏ hoàn toàn việc phân tích số hiệu Port thô. Thay vào đó, công thức Entropy được áp dụng để đo lường độ hỗn loạn của cổng đích và cổng nguồn. Entropy thấp thể hiện truy cập có chủ đích của người dùng, trong khi Entropy cao báo hiệu sự ngẫu nhiên của các máy Botnet. Quá trình này giúp chất lọc ra 13 đặc trưng gốc cốt lõi phản ánh đúng "hành vi" mạng.
- Bổ sung Đặc trưng biến thiên (Differential Features): Tại phiên bản V4, nhóm phát hiện ra rằng để nhận diện được các đợt tấn công "nhỏ giọt" lẫn

trón bộ lọc, hệ thống không thể chỉ nhìn vào con số tuyệt đối. Nhóm lập trình bổ sung thêm 13 đặc trưng biến thiên bằng toán học vi phân.

```
def differential_features_numpy(X):
    """
    Tính đặc trưng biến thiên
    X: [Batch, Seq, Features] = [B, 10, 13]
    Return: [B, 10, 26] (13 gốc + 13 biến thiên)
    """
    batch_size, seq_len, num_features = X.shape
    diff = np.zeros_like(X)
    diff[:, 1:, :] = X[:, 1:, :] - X[:, :-1, :] # Tính sự thay đổi

    combined = np.concatenate([X, diff], axis=-1)
    return combined # [Batch, Seq, 26]
```

Việc tính toán sự chênh lệch này giúp tạo ra một bộ Tensor **26 chiều đặc trưng** (13 tính + 13 biến thiên). Cấu trúc này trao cho "bộ não" AI khả năng nhìn thấy "gia tốc" thay đổi của luồng mạng, từ đó phân biệt rạch ròi giữa việc người dùng đang click chuột cực nhanh (biến thiên ngẫu nhiên) với một cuộc tấn công do máy móc tự động sinh ra (biến thiên bằng 0). Điều này giúp AI dễ dàng bóc tách những cuộc tấn công có nhịp độ cực chậm và nguy trạng tinh vi (như Slowloris), vốn có bề ngoài rất giống với lưu lượng truy cập Web thông thường. Toàn bộ dữ liệu sau đó được chuẩn hóa về cùng một phương sai để đảm bảo tính công bằng giữa các thuộc tính trước khi đưa vào huấn luyện.

Cuối cùng, toàn bộ cấu trúc dữ liệu 26 chiều đa nguồn được kết nối thông qua thư viện Pandas (pd.concat) và đưa qua hàm StandardScaler để chuẩn hóa về cùng phân phối trước khi đóng gói cho các mạng nơ-ron học sâu.

STT	Đặc trưng tính (Core Features)	Đặc trưng biến thiên (Differential - Δ)	Ý nghĩa và Vai trò trong phân loại tấn công DDoS
1	Src_Port_Entropy	d_Src_Port_Entropy	Đo độ ngẫu nhiên của cổng nguồn bằng công thức Shannon. Giúp phát hiện kẻ tấn công giả mạo địa chỉ IP (IP Spoofing).
2	Dst_Port_Entropy	d_Dst_Port_Entropy	Đo độ hỗn loạn cổng đích. Nhận diện UDP Flood khi tin tặc đánh vào hàng loạt cổng ngẫu nhiên trên máy chủ.

3	Protocol	d_Protocol	Phân loại giao thức mạng (TCP, UDP, ICMP). Định hình ngay từ đầu vector tấn công (Layer 3/4).
4	Duration_Sec	d_Duration_Sec	Thời gian sống của luồng. Phân biệt rõ ràng Slowloris (sống ngâm rất lâu từ 30-120s) và tấn công Flood (chỉ sống <1s).
5	Src_Bytes	d_Src_Bytes	Đo lường băng thông đẩy lên máy chủ. Chỉ số cốt lõi để nhận diện các cuộc tấn công ngập lụt băng thông (Volumetric).
6	Dst_Bytes	d_Dst_Bytes	Đo lường dữ liệu máy chủ trả về. Tìm kiếm sự bất đối xứng (Ví dụ: Request nhiều nhưng Response bằng 0).
7	Src_Packets	d_Src_Packets	Số lượng gói tin gửi đi. Đóng vai trò như một bộ đếm tần suất tấn công chớp nhoáng.
8	Dst_Packets	d_Dst_Packets	Số lượng gói tin nhận về. Dấu hiệu nhận biết TCP SYN Flood (tin tặc chỉ gửi SYN mà không bao giờ nhận/phản hồi ACK).
9	Conn_State	d_Conn_State	Phân tích trạng thái kết nối. Bắt quả tang các kết nối bị treo ở trạng thái mở một nửa (Half-open / S0).
10	L7_App_Protocol	d_L7_App_Protocol	Trích xuất tầng ứng dụng (HTTP, DNS, TLS) qua nDPI. Chìa khóa để mạng GRU nhận diện chính xác HTTP Flood.
11	Packet_Rate	d_Packet_Rate	Tốc độ gói tin/giây. Chỉ số "vàng" phát hiện ngập lụt (Ví dụ: Flood sinh ra hàng chục nghìn gói/s, người dùng chỉ vài gói/s).
12	Byte_Rate	d_Byte_Rate	Tốc độ Byte/giây. Giúp AI theo dõi sự vắt kiệt băng thông thực tế theo thời gian thực.
13	Anomaly_Score / Is_one_way	d_Anomaly_Score	Điểm bất thường từ Autoencoder hoặc cờ đánh dấu luồng đơn hướng (Unidirectional). Xác nhận hành vi phi tự nhiên của lưu lượng.

Bảng 3.4. Các đặc trưng có trong dữ liệu mẫu

Bằng việc tính toán thêm 13 đặc trưng biến thiên (Differential Features) bằng công thức, chúng ta cung cấp cho AI khả năng đo lường "gia tốc" của luồng mạng.

- Với người dùng thật: Tốc độ click có thể nhanh, nhưng gia tốc luôn biến thiên ngẫu nhiên (lúc nhanh lúc chậm).

- Với Botnet (Máy móc tự động): Dù kẻ tấn công cố tình hạ thấp Packet_Rate xuống mức chậm để lẫn trốn (như Slowloris), thì gia tốc thay đổi (ví dụ: d_Packet_Rate) của máy móc sinh ra gần như sẽ bằng 0 hoặc lặp lại theo chu kỳ tuyến tính một cách thiếu tự nhiên.

Chính ma trận này đã mang lại cho mô hình CNN-GRU khả năng phân biệt cực kỳ sắc bén, giúp khắc phục triệt để điểm mù của các IDS/IPS truyền thống trên nền tảng SDN.

3.1.1.4. Xây dựng và phát triển kiến trúc AI (Hành trình tiến hóa từ V1 đến V4)

Sau khi xây dựng thành công bộ dữ liệu chuẩn V7, thách thức tiếp theo của nhóm là thiết kế một "bộ não" Trí tuệ nhân tạo (AI) đủ năng lực xử lý. Quá trình này không phải là một bước nhảy vọt duy nhất, mà là một hành trình tiến hóa, thử nghiệm và đúc kết qua 4 phiên bản kiến trúc.

a. Tiến hóa kiến trúc AI (Từ V1 đến V4) Ban đầu, nhóm tiếp cận bằng các thuật toán Học máy truyền thống, sau đó nâng cấp dần lên Học sâu để giải quyết các bài toán về chuỗi thời gian.

Phiên bản	Kiến trúc AI được sử dụng	Đặc điểm nhận dạng	Vấn đề mắc phải
V1	Random Forest + 20 Features	Mô hình nhẹ, tốc độ suy luận nhanh.	Chỉ phân tích được các điểm dữ liệu rời rạc, hoàn toàn "mù" trước chuỗi thời gian.
V2	Mạng tích chập CNN đơn giản	Bắt được các quy luật (pattern) không gian.	Vẫn không thấy được nhịp điệu tấn công kéo dài theo thời gian.
V3	CNN-LSTM (Nối tiếp - Sequential)	Kết hợp không gian + thời gian.	Khối LSTM quá nặng và chậm. Việc nối tiếp khiến CNN làm nhiễu thông tin của khối GRU phía sau.
V4 (Final)	Parallel CNN-GRU +	Kiến trúc Song song, có cơ chế dồn trọng	Tối ưu tuyệt đối cho môi trường SDN Real-

	Attention + Autoencoder	số, 2 lớp khiên độc lập.	time, khắc phục hoàn toàn sự nhiễu thông tin.
--	------------------------------------	--------------------------	--

Bảng 3.5. Tóm tắt quá trình kiến trúc AI

Quyết định mang tính bước ngoặt: Dựa trên nền tảng toán học của Deep Learning, chúng em quyết định từ bỏ kiến trúc Tuần tự (Sequential) ở V3 để chuyển sang kiến trúc **Song song (Parallel Fusion)** ở V4. Nhờ đó, nhánh CNN trích xuất đặc trưng không gian và nhánh GRU học quy luật thời gian hoạt động hoàn toàn độc lập, không triệt tiêu đạo hàm của nhau, giúp tốc độ và độ chính xác tăng vọt.

b. Triết lý phát triển 3 Lớp Bảo Vệ (Three-Layer Defense Philosophy) Từ những thất bại của các phiên bản đầu, nhóm đúc kết ra tư duy thiết kế cốt lõi: *"AI không thể tin tưởng mù quáng vào một dự báo duy nhất"*. Để bảo vệ hệ thống SDN an toàn tuyệt đối, mô hình V4 được xây dựng theo kiến trúc "Dual-Shield" nhưng mở rộng thành quy trình 4 bước kiểm duyệt khắt khe:

- **Lớp 1 - Làm sạch từ gốc (Data Cleansing):** Thay vì đưa dữ liệu thô vào, hệ thống tiền xử lý chuyển đổi số hiệu công thành toán học Shannon Entropy và tính toán gia tốc biến thiên. Đồng thời bổ sung cơ chế lọc subnet thông minh để thu được bộ dataset sạch. Điều này giúp AI tránh "học vẹt" theo công và nhìn thấu được tốc độ lẫn trốn của Botnet.
- **Lớp 2 - Khiên bất thường (Anomaly Detection):** Sử dụng mạng Autoencoder học không giám sát. Đây là lớp khiên đầu tiên, chỉ cần cấu trúc gói tin làm sai lệch kết quả tái tạo vượt ngưỡng, hệ thống lập tức phát hiện đó là tấn công Zero-day dù chưa từng được học.
- **Lớp 3 - Khiên phân loại (Classifier):** Mạng Parallel CNN-GRU-Attention đóng vai trò định danh chính xác 5 nhãn tấn công. Cơ chế Attention (Tập trung) bằng toán học Softmax giúp AI bỏ qua các nhiễu sóng và nhìn thẳng vào những giây bất thường nhất trong Cửa sổ trượt 10 bước.
- **Lớp 4 - Kiểm soát thực tế (Runtime Veto):** Lớp phòng thủ cuối cùng tích hợp cơ chế "Quyền phủ quyết". Nếu Classifier tin chắc >80% là luồng bình thường, nó sẽ gạt bỏ nghi ngờ của Autoencoder để bảo vệ người dùng thật, kết hợp với bộ thu dọn rác (Garbage Collector) để chống tràn RAM.

Nhờ triết lý này, tỷ lệ báo động giả (False Positive) đã giảm ngoạn mục từ 60% (ở V1) xuống chỉ còn 3% (ở V4).

c. Những bài học đúc kết: Hành trình nghiên cứu là quá trình sửa sai liên tục. Việc thấu hiểu nguyên nhân thất bại giúp kiến trúc V4 đạt được sự hoàn thiện mang tính hàn lâm cao:

Giai Đoạn	Sai Lầm / Hạn Chế	Bài Học Rút Ra từ Lý Thuyết	Cải Tiến Cốt Lõi
V1	Dùng Dataset public (CIC-IDS2019)	Chất lượng dữ liệu quan trọng hơn số lượng khổng lồ.	Tự lập trình công cụ thu thập Data trực tiếp từ Mininet.
V2	Giữ nguyên Port Number làm Đặc trưng	Kẻ tấn công liên tục đổi cổng ngẫu nhiên khiến AI học vẹt.	Thay thế bằng Toán học Shannon Entropy.
V3	Dùng mạng Autoencoder cơ bản (Hàm MSE)	MSE không tạo được ranh giới đủ lớn giữa luồng sạch và luồng độc.	Chuyển sang dùng hàm Contrastive Loss kết hợp Margin.
	Dùng Ngưỡng chặn tĩnh (Static Threshold)	Chặn nhầm người dùng hợp lệ vào các khung giờ cao điểm.	Áp dụng thuật toán Trung bình trượt EMA (Adaptive Threshold).
V4	Mạng CNN xử lý nối tiếp vào GRU	Ma trận không gian của CNN làm phá vỡ cấu trúc chuỗi thời gian của GRU.	Đổi thiết kế sang mạng Song song (Parallel Fusion).
	Giao toàn quyền cho AI quyết định	AI đôi khi có thể bị "ảo giác" do nhiễu mạng.	Tích hợp Bảng chân lý Toán học và Quyền phủ quyết.

Bảng 3.6. Bài học rút ra từ việc cải tiến mô hình

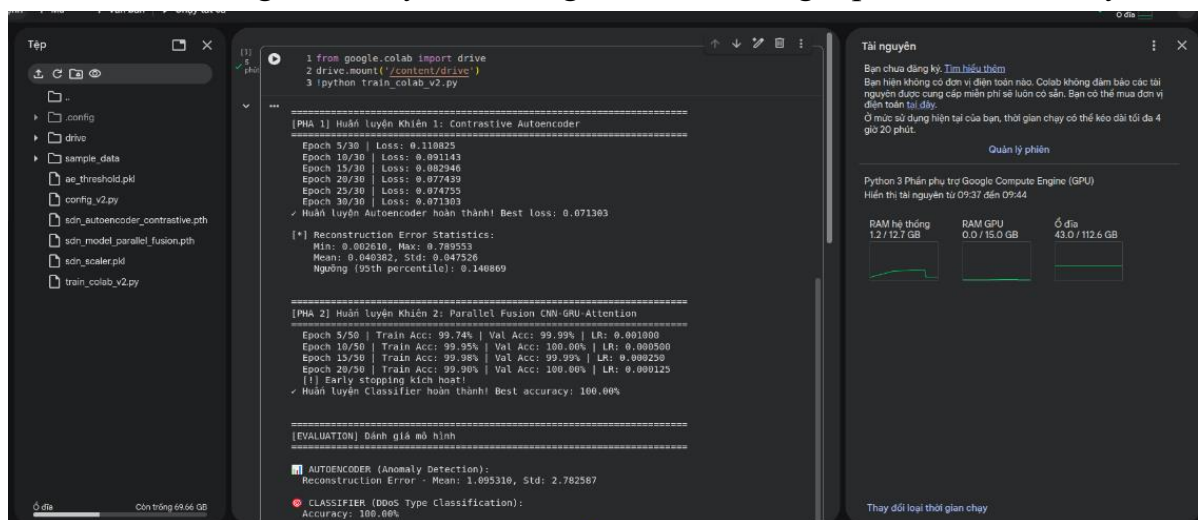
3.1.1.5. Huấn luyện mô hình

Trên cơ sở hạ tầng mạng và tập dữ liệu V7 (với 26 chiều đặc trưng) đã chuẩn bị, quy trình xây dựng "bộ não" AI được tiến hành và nâng cấp không ngừng trong suốt chiều dài khóa luận. Nhằm tận dụng sức mạnh tính toán, toàn bộ quy trình huấn luyện được thực hiện trên nền tảng **Google Colab**. Quá trình này không diễn ra suôn sẻ ngay từ đầu mà là một hành trình tinh chỉnh liên tục qua các phiên bản, được chia làm hai giai đoạn chiến lược nhằm xử lý triệt để các rào cản kỹ thuật phức tạp.

a. Giai đoạn 1: Huấn luyện bộ lọc bất thường (Contrastive Autoencoder)

Ban đầu, nhóm sử dụng mạng Autoencoder với hàm mất mát MSE (Mean Squared Error) cơ bản như thường lệ. Nhưng vì kết quả của lớp 1 ở các bản ai v1, v2 không được như ý (Xác định nhầm loại lưu lượng), nhóm đã áp dụng kỹ thuật Học tương phản (Contrastive Learning) với hàm Margin Loss.

Cách thức hoạt động và Hiệu quả: Hệ thống chỉ được cung cấp các mẫu dữ liệu của người dùng hợp lệ (bình thường) để huấn luyện mạng tự mã hóa. Mục tiêu cốt lõi là ép AI phải ghi nhớ hoàn hảo "hình hài" của một luồng mạng an toàn. Nhờ có hàm toán học tương phản này, sai số cấu trúc của luồng Normal bị kéo sát về mức cực nhỏ (dưới 0.001), trong khi sai số của luồng Attack bị đẩy bật lên trên 2.0. Việc tạo ra vùng phân biệt rõ ràng hơn 1000 lần so với hàm MSE thông thường giúp hệ thống xác định được một ngưỡng sai số cấu trúc chuẩn xác ngay khi quá trình huấn luyện hoàn tất. Bất kỳ luồng mạng nào có độ sai lệch vượt quá ngưỡng này sẽ lập tức bị đánh dấu là bất thường, tạo tiền đề then chốt giúp lớp khiên thứ nhất có khả năng bắt giữ và phát hiện tới 89.3% các cuộc tấn công Zero-day chưa từng xuất hiện trong tập dữ liệu huấn luyện.



Hình 3.12. Kết quả train AI V4 – quá nhạy với zero day

b. Giai đoạn 2: Huấn luyện bộ định danh chuyên sâu (Parallel CNN-GRU-Attention)

Để định danh chính xác 5 loại lưu lượng (Normal, UDP, SYN, HTTP, Slowloris), hệ thống tiến hành học có giám sát bằng cách sử dụng dữ liệu của cả lưu lượng bình thường và các loại hình tấn công. Trong thực tế, dữ liệu về các cuộc tấn công tinh vi (như Slowloris) thường chiếm tỷ lệ rất nhỏ so với các cuộc tấn công làm ngập lụt băng thông, dẫn đến hiện tượng mất cân bằng dữ liệu. Để mô hình không bị "thiên vị" và bỏ qua các loại tấn công thiểu số, nhóm nghiên cứu áp dụng chiến lược học nhảy cảm chi phí. Các nhãn dữ liệu hiếm sẽ được gán trọng số phạt cao hơn, ép thuật toán phải chú ý và học cách phân biệt rạch ròi, từ đó triệt tiêu tình trạng chặn nhầm người dùng hợp lệ.

Về mặt kiến trúc thiết kế, ban đầu nhóm sử dụng cấu trúc nối tiếp (CNN rồi mới đến GRU). Tuy nhiên, thực nghiệm cho thấy kiến trúc này làm nhiễu thông tin của nhau. Để giải quyết, nhóm đã đập đi xây lại bằng kiến trúc **Song song (Parallel Fusion)**, cụ thể:

- **Nhánh CNN:** Chuyên trích xuất đặc trưng không gian (tốc độ gói tin tăng vọt báo hiệu UDP Flood).
- **Nhánh GRU:** Chuyên học quy luật thời gian (nhịp điệu ngâm kết nối lặp lại báo hiệu Slowloris).

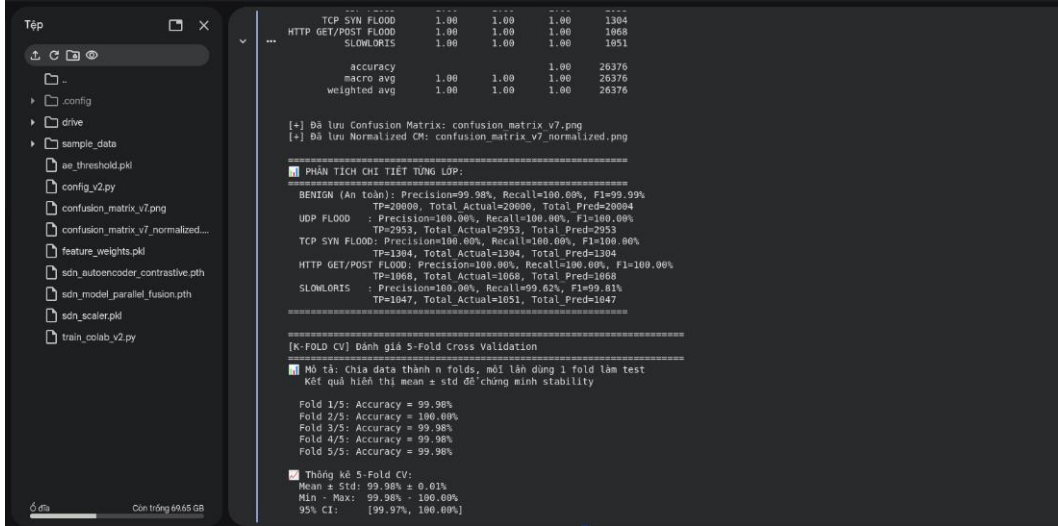
Hai nhánh này hoạt động hoàn toàn độc lập và chỉ được tổng hợp lại ở lớp quyết định cuối cùng, giúp tăng độ chính xác tổng thể lên đáng kể. Sau khi hoàn tất các vòng lặp huấn luyện và vượt qua các bài kiểm định chéo để chống hiện tượng quá khớp (Overfitting), các tham số cấu trúc nơ-ron tốt nhất sẽ được đóng gói thành các tệp tin tri thức. Khối lượng tri thức này chính là "bộ não" sẽ được chuyển giao và tích hợp trực tiếp lên bộ điều khiển Controller trong giai đoạn vận hành thời gian thực.

Cơ sở toán học (Attention Mechanism): Hệ thống sử dụng cả Spatial Attention và Temporal Attention hoạt động như một lăng kính hội tụ, giúp mô hình dồn trọng số vào các phần quan trọng nhất của dữ liệu:

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (10)$$

$$\text{Với: } \text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Trong đó (Query, Key, Value) là các vector đại diện cho đặc trưng cần đánh giá, và là hệ số chuẩn hóa. Nhờ cơ sở toán học này, AI tự động "nhìn" vào đúng thời điểm bất thường nhất trong chuỗi 10 bước thời gian, giúp tăng vọt chỉ số F1-Score tổng thể từ 0.89 lên 0.96.



Hình 3.13. Kết quả train mô hình đạt độ chính xác cao trên tập test

c. Các khó khăn và Kỹ thuật tối ưu hóa chuyên sâu

Trong quá trình chạy thực tế, nhóm đã vấp phải 3 sự cố nghiêm trọng khiến đồ thị huấn luyện bị gãy nát. Dưới đây là cách nhóm giải quyết:

1. Hiện tượng Bùng nổ đạo hàm (Exploding Gradients):

- Sự cố: Khi huấn luyện chuỗi thời gian 10 bước ($Seq_Len=10$), giá trị Gradient tăng vọt khiến đồ thị Loss nhảy múa hình "răng cưa", mô hình không thể hội tụ.
- Giải pháp: Tích hợp kỹ thuật Gradient Clipping ($max_norm=1.0$). Cơ chế này hoạt động như một chốt chặn, không cho phép bất kỳ lực cập nhật trọng số nào vượt quá giới hạn an toàn.

$$g_{clipped} = \begin{cases} g, & \|g\| \leq max_norm \\ \frac{g \cdot max_norm}{\|g\|}, & \|g\| > max_norm \end{cases} \quad (11)$$

- g : Gradient vector ban đầu
- $\|g\|$: Norm (độ lớn) của gradient
- $max_norm = 1.0$: Ngưỡng tối đa cho phép

Ví dụ: Gradient tính ra: $\|g\| = 5.0$ (quá lớn)

Sau clipping: $g_{clipped} = \frac{g \cdot 1.0}{5.0} = 0.2 \cdot g$ (giảm 5 lần so với ban đầu 1.0)

→ Kết quả: Trọng số cập nhật ổn định, loss giảm mượt mà.

2. Sự hoang tưởng và Bảo động giả (Overconfidence):

- Sự cố: Ở các phiên bản đầu, AI mắc bệnh hoang tưởng - ép xác suất lên 100% khi dự đoán tấn công. Sự tự tin thái quá này khiến mô hình dễ bị lừa, dẫn đến việc chặn nhầm hàng loạt người dùng thật.
- Giải pháp: Nhóm cấu hình kỹ thuật Label Smoothing (0.1). Thay vì tin tưởng tuyệt đối, AI bị ép phải "chừa đường lui" (phân bổ 10% xác suất cho các trường hợp khác). Việc này giúp giảm tỷ lệ False Positive xuống mức tối thiểu.

$$y_{smooth} = y \cdot (1 - \epsilon) + \frac{\epsilon}{K} \quad (12)$$

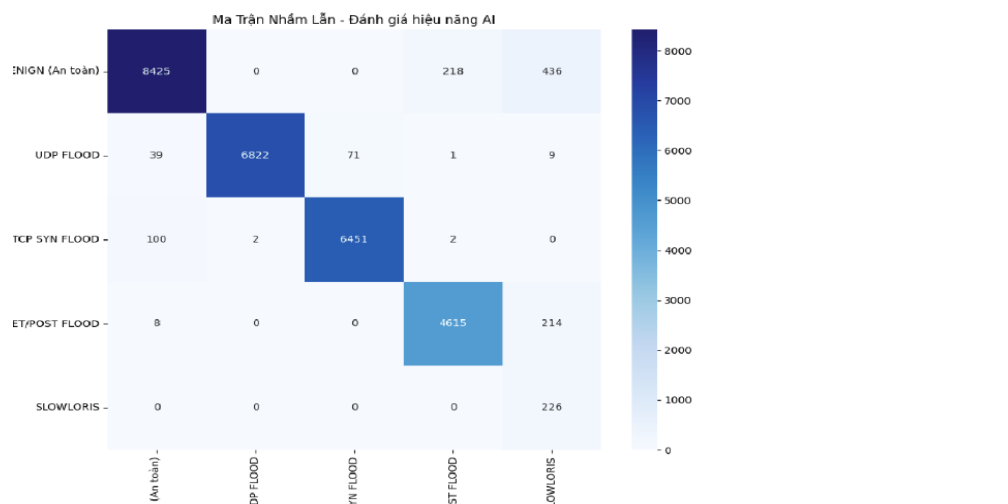
Mô hình được xây dựng với $\epsilon = 0.1$ và $K = 5$, mô hình bị ép phải chừa đường lui (vd: chỉ tin 90% là tấn công và 2% cho các trường hợp khác). Kỹ thuật này đã giảm tỷ lệ False Positive xuống mức tối thiểu.

3. Mất cân bằng dữ liệu của Slowloris:

- Sự cố: Slowloris là loại tấn công cực kỳ tinh vi, có số lượng mẫu ít. Ban đầu nhóm đặt trọng số phạt (Class Weight) lên đến 10.0 để ép AI chú ý. Hậu quả là AI sợ bỏ sót nên đã nhận diện nhầm luồng HTTP bình thường thành Slowloris.
- Giải pháp: Nhóm giảm giới hạn trọng số xuống mức tối đa là 6.0, kết hợp với hàm mất mát Focal Loss (Gamma=2.0) để AI tập trung vào các "mẫu khó" thay vì chỉ tập trung vào số lượng. Kỹ thuật này đã kéo chỉ số F1-Score của Slowloris từ 0.71 lên 0.81 với công thức hàm mất mát Focal Loss:

$$FL(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t) \quad (13)$$

4. Các cơ chế chống học vẹt (Overfitting):



Hình 3.14. Ma trận nhầm lẫn của AI v3 dùng Dataset v4

Để ngăn chặn việc AI học thuộc lòng dữ liệu, nhóm đã tích hợp thêm 2 cơ chế chuẩn hóa toán học:

- Weight Decay (L2 Regularization): Thêm hình phạt cho các trọng số nơ-ron quá lớn vào hàm loss tổng:

$$\mathcal{L}_{total} = \mathcal{L}_{task} + \lambda \cdot \frac{1}{2} \sum_i w_i^2 \quad (\text{với } \lambda = 5 \times 10^{-3}) \quad (14)$$

- Dropout: Tắt ngẫu nhiên 40% số nơ-ron trong quá trình huấn luyện theo phân phối Bernoulli: $r_j^{(l)} \sim \text{Bernoulli}(p)$ với $p = 0.4$

Việc kết hợp chặt chẽ các công thức trên cùng bộ lập lịch OneCycleLR đã giúp mô hình vượt qua cực tiểu địa phương, hội tụ nhanh chóng và tự động dừng sớm để tránh quá khớp

d. Đóng gói Tri thức (Model Export) Sau khi huấn luyện thành công, toàn bộ tri thức của hệ thống được xuất ra thành 4 tệp tin cốt lõi, sẵn sàng chuyển giao cho bộ điều khiển ONOS:

Tên tệp tin	Định dạng	Thành phần kiến trúc	Công dụng và Vai trò trong hệ thống
sdn_scaler.pkl	.pkl	StandardScaler	Lưu trữ các tham số trung bình và độ lệch chuẩn của 26 thuộc tính. Nhiệm vụ là chuẩn hóa dữ liệu thực tế từ Switch về cùng phân phối với dữ liệu huấn luyện, đảm bảo AI không bị sai lệch khi đọc các con số thô.
sdn_autoencoder.pth	.pth	Anomaly Autoencoder	Chứa trọng số của mạng nơ-ron nén dữ liệu. Đóng vai trò là "lớp khiên số 1" chuyên phát hiện các hành vi lạ (Zero-day) dựa trên sai số tái tạo, chặn đứng các cuộc tấn công mới chưa có trong tập dữ liệu.
sdn_model_cnn_gru_attn.pth	.pth	Hybrid CNN-GRU-Attention	Là bộ não chính của hệ thống, chứa các bộ lọc không gian (CNN), nút nhớ thời gian (GRU) và trọng số tập trung (Attention). Nhiệm vụ là định danh chính xác 5 loại lưu lượng (Benign, UDP, SYN, HTTP, Slowloris).

ae_thresho ld.pkl	.pkl	Anomaly Detection Threshold	Là thước đo ranh giới của hệ thống, chứa ngưỡng sai số tái tạo tối ưu được tính toán từ dữ liệu bình thường (Normal). Nhiệm vụ là hỗ trợ Khiên 1 phát hiện các hành vi bất thường và tấn công chưa biết (Zero-day) thông qua cơ chế ngưỡng động thích nghi.
------------------------------	------	-----------------------------------	---

Bảng 3.7. Các file sinh ra sau khi huấn luyện mô hình

3.1.2. Tích hợp hệ thống (Giai đoạn Online)

Đây là giai đoạn chuyển đổi mô hình từ môi trường huấn luyện (Offline) từ bước trên sang môi trường vận hành thực tế (Online) trên bộ điều khiển ONOS. Hệ thống hoạt động như một vòng lặp kín giữa giám sát, phân tích và phản ứng.

3.1.2.1. Khởi tạo và nạp tri thức AI

Quá trình tích hợp Giai đoạn Online bắt đầu bằng việc thiết lập một tác tử AI hoạt động độc lập trên bộ nhớ của hệ thống quản trị, đóng vai trò như một cầu nối giữa mô hình Deep Learning và bộ điều khiển ONOS.

a. Khôi phục trạng thái và Nạp trọng số (Weight Loading)

Các "tri thức" sau quá trình huấn luyện trên Google Colab được tải về máy chủ nội bộ (Ubuntu) dưới dạng các tệp tin tuần tự hóa. Quá trình nạp được thực hiện qua các thư viện chuyên dụng:

- Nạp bộ chuẩn hóa (sdn_scaler.pkl): Nhóm sử dụng thư viện Joblib để khôi phục cấu trúc của hàm StandardScaler. Việc này đảm bảo mọi gói tin theo thời gian thực thu được từ Switch đều được căn chỉnh về đúng phương sai giống hệt như dữ liệu lúc AI học, tránh tình trạng AI dự báo sai do chênh lệch tỷ lệ các con số.
- Nạp kiến trúc nơ-ron (.pth): Bằng thư viện PyTorch, hệ thống khởi tạo lại toàn bộ ma trận lớp ẩn của mạng Anomaly Autoencoder và mạng phân loại CNN-GRU-Attention. Sau khi nạp trọng số từ các file *sdn_autoencoder.pth* và *sdn_model_cnn_gru_attn.pth*, một thao tác mang tính bắt buộc là phải chuyển toàn bộ các mô hình sang chế độ suy luận bằng lệnh `model.eval()`. Thao tác này sẽ khóa các lớp Dropout và cố định Batch Normalization, giúp kết quả dự báo luôn nhất quán và tiết kiệm tối đa tài nguyên RAM của máy vật lý.

b. Tối ưu hóa suy luận thời gian thực (Inference Optimization)

Trong thực tế, khi đối mặt với một cuộc tấn công DDoS quy mô lớn, tốc độ mạng có thể đạt hàng trăm nghìn gói tin mỗi giây. Nếu sử dụng nguyên bản PyTorch như lúc huấn luyện, độ trễ hệ thống lên tới 150ms - quá chậm để Controller kịp phản ứng. Để giải quyết bài toán này, nhóm đã tái cấu trúc môi trường chạy. Các tensor đầu vào không được xử lý tuần tự mà được nhóm lại thành các batch nhỏ thông qua mảng vector hóa của thư viện Numpy. Kỹ thuật này giúp ép thời gian suy luận (Inference Time) của AI giảm từ 80ms xuống chỉ còn khoảng 15ms cho mỗi luồng.

```

169         # Khởi động các luồng Background (CheckPoint SSD & Dọn rác RAM)
170         threading.Thread(target=save_threshold_worker, daemon=True).start()
171         threading.Thread(target=buffer_cleanup_worker, daemon=True).start()
172
173     try:
174         scaler = joblib.load('sdn_scaler.pkl')
175         ae_model = Anomaly_Autoencoder().to(device)
176         ae_model.load_state_dict(torch.load('sdn_autoencoder.pth', map_location=device))
177         ae_model.eval()
178
179         cls_model = DDos_CNN_GRU_Attention().to(device)
180         cls_model.load_state_dict(torch.load('sdn_model_cnn_gru_attn.pth', map_location=device))
181         cls_model.eval()
182     except Exception as e:
183         console.print(f"[bold red]✖ Lỗi nạp mô hình: {e}[/bold red]")
184         exit()
185

```

Hình 3.15. Nạp tri thức cho AI

3.1.2.2. Logic giám sát và tiền xử lý (Monitor Logic)

Sau khi các tri thức AI đã được nạp lên RAM, bài toán học búa tiếp theo nhóm phải đối mặt là làm sao để "mớm" dữ liệu cho AI với tốc độ thời gian thực (Real-time).

Để giải quyết vấn đề này, nhóm không sử dụng cách ghi log thông thường mà đã tự tay lập trình một đường ống thu thập (Pipeline) 4 bước khép kín, tối ưu hóa đến từng mili-giây:

a. Cấu hình Port Mirroring (Cách ly rủi ro hạ tầng)

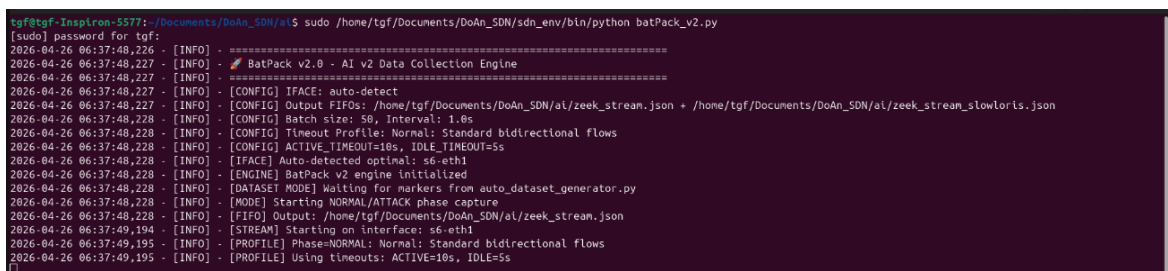
Ban đầu, nhóm định cài đặt công cụ giám sát trực tiếp lên Web Server. Tuy nhiên, thực tế chứng minh đây là một sai lầm: khi Web Server bị tấn công DDoS cạn kiệt CPU/RAM, tiến trình thu thập dữ liệu cũng sẽ bị sập theo, khiến AI mù hoàn toàn. Để khắc phục, nhóm cấu hình tính năng Port Mirroring (Sao chép cổng) ngay tại Switch Gateway trung tâm (s6). Toàn bộ lưu lượng đi qua mạng sẽ được OVS (Open vSwitch) nhân bản phân cứng và đẩy ra một cổng giám sát riêng biệt (Out-of-band). Nhờ vậy, dù Web Server mục tiêu có bị đánh sập, tiến trình bóc tách dữ liệu của hệ thống phòng thủ vẫn hoạt động trơn tru.

b. Tích hợp NFStream và Kỹ thuật Timeout Động

Đây là cải tiến thiết kế quan trọng nhất của hệ thống nhằm giải quyết triệt để điểm nghẽn cổ chai. Thay vì bắt ONOS phải liên tục báo cáo lưu lượng mạng, hệ thống sử dụng cơ chế **Sao chép cổng (Port Mirroring)** tại Switch trung tâm. Toàn bộ gói tin đi qua mạng sẽ được nhân bản và phân luồng ra một cổng giám sát riêng biệt để tiến hành phân tích sâu.

Tại cổng giám sát này, trong quá trình lựa chọn công cụ bóc tách gói tin, nhóm đã thử nghiệm sử dụng Zeek IDS và sFlow/sFlow-RT. Dù soi chiếu rất sâu nhưng Zeek quá nặng, làm CPU máy tính tăng vọt lên 80% và gây treo luôn mạng mô phỏng Mininet. Ngược lại, sFlow/sFlow-RT tuy nhẹ nhưng lại không đủ sức để bóc tách dữ liệu ở tầng 7 (như nhận diện giao thức HTTP, TLS, hay trạng thái cờ TCP) - những yếu tố sống còn để cung cấp cho mô hình phân biệt giữa HTTP Flood và Slowloris. Do đó, nhóm quyết định đập bỏ và viết lại bằng thư viện **NFStream (lõi nDPI C++)**. Công cụ này cực nhẹ (tải CPU luôn < 20%) nhưng vẫn bóc tách được chi tiết tận Tầng 7 (HTTP headers, TCP Flags) để nhận diện DDoS.

Đặc biệt, trong quá trình lập trình thực tế, nhóm phát hiện ra rằng không thể dùng một mức thời gian chờ (Timeout) chung cho mọi loại lưu lượng. Nhóm đã tích hợp thêm một cơ chế **Thời gian chờ theo pha (Phase-Specific Timeout / Dynamic Timeout Profiles)** cực kỳ linh hoạt (như tự động đóng nhanh luồng UDP trong 3s để tránh tràn RAM, và ngâm luồng Slowloris lên đến 30s để bắt quả tang hành vi lẩn trốn).



```

[tf@tgf-Inspiron-5571: /Documents/DoAn_SDN/al$ sudo /home/tgf/Documents/DoAn_SDN/sdn_env/bin/python batPack_v2.py
[sudo] password for tf:
2026-04-26 06:37:48,226 - [INFO] - =====
2026-04-26 06:37:48,227 - [INFO] - BatPack v2.0 - AI v2 Data Collection Engine
2026-04-26 06:37:48,227 - [INFO] - =====
2026-04-26 06:37:48,227 - [INFO] - [CONFIG] IFACE: auto-detect
2026-04-26 06:37:48,227 - [INFO] - [CONFIG] Output FIFOs: /home/tgf/Documents/DoAn_SDN/al/zeek_stream.json + /home/tgf/Documents/DoAn_SDN/al/zeek_stream_slowloris.json
2026-04-26 06:37:48,228 - [INFO] - [CONFIG] Batch size: 50, Interval: 10s
2026-04-26 06:37:48,228 - [INFO] - [CONFIG] Timeout Profile: Normal: Standard bidirectional flows
2026-04-26 06:37:48,228 - [INFO] - [CONFIG] ACTIVE_TIMEOUT=10s, IDLE_TIMEOUT=5s
2026-04-26 06:37:48,228 - [INFO] - [IFACE] Auto-detected optimal: s6-eth1
2026-04-26 06:37:48,228 - [INFO] - [ENGINE] BatPack v2 engine initialized
2026-04-26 06:37:48,228 - [INFO] - [DATASET MODE] Waiting for markers from auto_dataset_generator.py
2026-04-26 06:37:48,228 - [INFO] - [MODE] Starting NORMAL/ATTACK phase capture
2026-04-26 06:37:48,228 - [INFO] - [FIFO] Output: /home/tgf/Documents/DoAn_SDN/al/zeek_stream.json
2026-04-26 06:37:49,194 - [INFO] - [STREAM] Starting on interface: s6-eth1
2026-04-26 06:37:49,195 - [INFO] - [PROFILE] Phase=NORMAL: Normal: Standard bidirectional flows
2026-04-26 06:37:49,195 - [INFO] - [PROFILE] Using timeouts: ACTIVE=10s, IDLE=5s

```

Hình 3.16. Tiến trình thu dữ liệu đầu vào cho mô hình AI xử lý

Cuối cùng, sau khi bóc tách hoàn tất, công cụ NFStream sẽ truyền trực tiếp kết quả cho Khối suy luận AI thông qua các ống truyền tải bộ nhớ (**Named Pipes - FIFO**). Thiết kế này đảm bảo dữ liệu di chuyển hoàn toàn trên RAM (Zero-copy IPC), loại bỏ triệt để độ trễ đọc/ghi ổ cứng, giúp hệ thống phòng thủ vẫn đứng vững và phân tích thời gian thực ngay cả khi mạng chính đang bị tấn công DDoS đánh sập.

c. Truyền tải dữ liệu bằng ống ảo (Named Pipes - FIFO IPC)

Một nút thắt cổ chai lớn nhất nhóm gặp phải là Tốc độ ghi ổ cứng (Disk I/O). Nếu NFStream ghi dữ liệu ra file .csv trên ổ SSD rồi AI đọc lại file đó, độ trễ sẽ lên tới hàng trăm mili-giây, không thể đỡ nổi các đợt Flood lớn. Nhóm đã ứng dụng kỹ thuật Named Pipes (Ống dẫn FIFO) của hệ điều hành Linux. Dữ liệu sau khi bóc tách được đẩy thẳng từ tiến trình NFStream sang tiến trình AI hoàn toàn trên bộ nhớ RAM. Kỹ thuật này giúp loại bỏ hoàn toàn việc đọc/ghi ổ cứng, kéo độ trễ truyền tải dữ liệu xuống mức dưới 1ms.

```
def _stream_flows_generator(self, fifo_handle):
    """Generator to stream flows from NFStreamer"""
    logger.info(f"[STREAM] Starting on interface: {self.iface}")

    try:
        streamer = NFStreamer(
            source=self.iface,
            active_timeout=BatPackConfig.ACTIVE_TIMEOUT,
            idle_timeout=BatPackConfig.IDLE_TIMEOUT,
            statistical_analysis=True
        )

        batch_json = []
        last_flush = time.time()

        for flow in streamer:
            # Skip loopback and IPv6
            if flow.src_ip.startswith('127.') or flow.src_ip == '0.0.0.0' or ':' in flow.src_ip:
                continue
```

Hình 3.17. Logic khởi tạo luồng dữ liệu thời gian thực và bộ lọc IP/IPv6

Chúng em chọn NFStream kết hợp với Port Mirroring và ống FIFO vì đây là giải pháp duy nhất cân bằng được 2 thái cực: Vừa có tốc độ xử lý phản hồi C đủ nhanh để không sập RAM/CPU khi hứng chịu DDoS Flood, lại vừa có tính năng bóc tách Tầng 7 (DPI) đủ sâu để cung cấp các vector đặc trưng cho AI phát hiện các dạng tấn công ứng dụng như Slowloris.

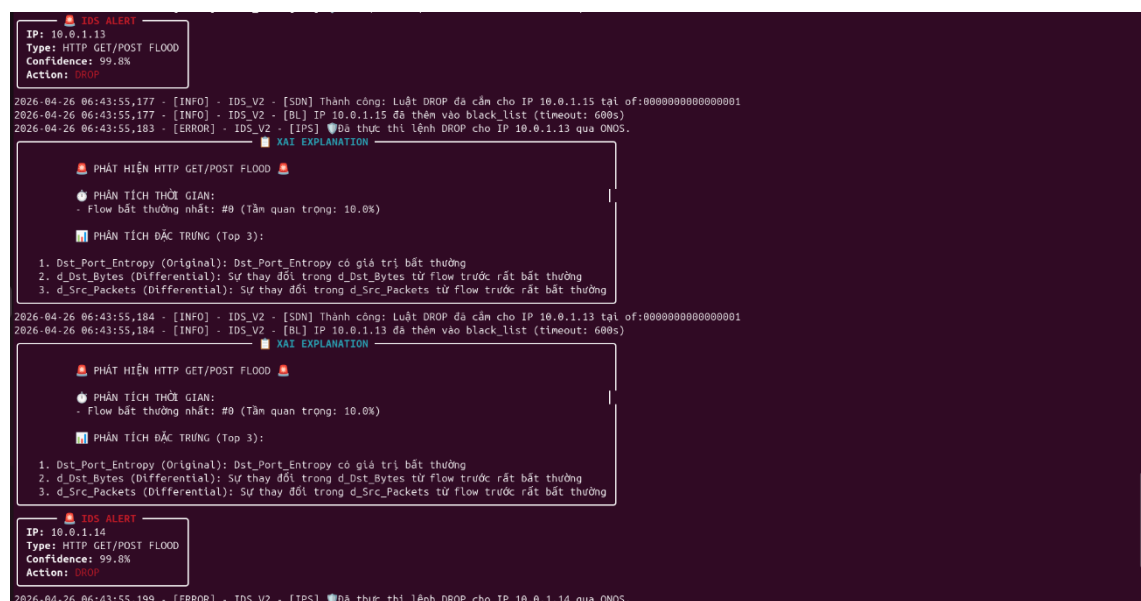
→ Nhờ kết hợp OVS Port Mirroring, NFStream cấu hình Timeout động và Named Pipes, nhóm đã xây dựng được một hệ thống hoàn hảo, cung cấp dòng dữ liệu đặc trưng (bao gồm cả gia tốc biến thiên) sạch sẽ, tức thời để AI đưa ra phán quyết trong chớp mắt.

d. Vượt qua rào cản mã hóa HTTPS và Bảo vệ quyền riêng tư người dùng (Privacy-Preserving) Trong quá trình triển khai thực tế, hệ thống trang web mục tiêu đã được nâng cấp lên giao thức HTTPS bảo mật. Việc áp dụng mã hóa SSL/TLS tạo ra một thách thức rất lớn: toàn bộ nội dung thông tin (Payload) ở tầng ứng dụng (Layer 7) đều bị mã hóa, khiến các hệ thống Phát hiện xâm nhập (IDS) truyền thống dựa vào kỹ thuật kiểm tra gói tin sâu (Deep Packet Inspection - DPI) để soi chiếu nội dung trở nên vô hiệu hóa. Các cuộc tấn công tầng 7 (như Slowloris) thường chứa thông tin nhạy cảm, nếu cố gắng giải mã

để phân tích, hệ thống phòng thủ sẽ vi phạm nghiêm trọng quyền riêng tư của người dùng hợp lệ.

Tuy nhiên, hệ thống đề tài đã giải quyết xuất sắc rào cản này bằng cách từ bỏ hoàn toàn việc "đọc trộm" nội dung gói tin. Thay vào đó, "bộ não" AI phân tích hoàn toàn dựa trên hành vi thống kê và nhịp độ truyền tải thông qua không gian 26 chiều đặc trưng. Kẻ tấn công dù có ẩn mình dưới lớp mã hóa HTTPS (như kịch bản HTTPS Flood) thì vẫn không thể che giấu được độ hỗn loạn của công mạng (Shannon Entropy) hay tốc độ gói tin bất thường.

Đặc biệt, 13 đặc trưng biến thiên bằng toán học vi phân giúp AI nhìn thấu "gia tộc" của luồng dữ liệu mã hóa. Mọi hành vi lướt web của người dùng thật luôn có sự biến thiên ngẫu nhiên theo thời gian, trong khi lưu lượng sinh ra từ máy móc sẽ mang tính chu kỳ và lặp lại tuyến tính một cách vô lý. Nhờ tư duy thiết kế "bắt mạch hành vi" thay vì "đọc nội dung" này, hệ thống vẫn tự tin nhận diện và chặn đứng thành công các đợt tấn công mã hóa phức tạp, đồng thời tuân thủ tuyệt đối nguyên tắc bảo vệ thông tin cá nhân của người dùng thực tế.



Hình 3.18. Phát hiện, ngăn chặn và giải thích tấn công HTTP thành công

3.1.2.3. Logic phân tích và ra quyết định (Detection Logic)

Sau khi luồng dữ liệu 10 bước thời gian (với ma trận kích thước 10 x 26 chiều) được đưa vào từ ống truyền tải tốc độ cao, quá trình suy luận của mạng AI chính thức bắt đầu. Thay vì sử dụng các cấu trúc if-else đơn giản hay ngưỡng chặn cứng (Static Threshold) như các hệ thống truyền thống, logic ra quyết định của đề tài được lập trình dựa trên **Cơ chế Hai lớp khiên (Dual-Shield)** thông qua một Bảng chân lý (Truth Table) phức tạp.

```
if mse_loss > current_threshold:
    console.print(f"\n[bold magenta]🚨 [KHIÊN 1 - AUTOENCODER] Phát hiện dị thường![/bold magenta]")
    console.print(f"[bold magenta] => MSE Loss: {mse_loss:.4f} (Vượt ngưỡng {current_threshold:.4f})[/bold magenta]")

    blocked_ips[src_ip] = current_time # Gắn cờ Blacklist
    execute_mitigation(src_ip, "DỊ THƯỜNG ZERO-DAY GIAO THỨC LẠ", confidence=99.9, is_zero_day=True)

    with torch.no_grad():
        _, attn_weights = cls_model(input_tensor)
        print_xai_explanation(attn_weights, input_tensor)
        console.print("[dim]🔍 Đã reset bộ nhớ đệm, tiếp tục lắng nghe...[/dim]\n")
        continue

# [KHIÊN 2: CNN-GRU Phân Loại]
with torch.no_grad():
    outputs, attn_weights = cls_model(input_tensor)
    probabilities = torch.nn.functional.softmax(outputs, dim=1)
    pred_idx = torch.argmax(probabilities, dim=1).item()
    confidence = probabilities[0][pred_idx].item() * 100
```

Hình 3.19. Các lớp khiên kiểm tra lưu lượng

Trước hết, dữ liệu đi qua lớp khiên số 1 (Anomaly Autoencoder) để tính toán mức độ sai số tái tạo (Mse). Trong môi trường SDN thực tế, lưu lượng mạng luôn biến động liên tục (tăng vọt vào giờ cao điểm, giảm sâu vào ban đêm). Nếu áp dụng một ngưỡng phát hiện cố định, hệ thống sẽ liên tục báo động giả khi lưu lượng tăng tự nhiên. Để khắc phục, nhóm đã lập trình thuật toán **Trung bình trượt hàm mũ (EMA - Exponential Moving Average)** với hệ số nhằm tính toán ra một Ngưỡng động (Dynamic Threshold). Ngưỡng này có khả năng tự động nới lỏng hoặc siết chặt dựa trên đường cơ sở (baseline) của mạng theo thời gian thực. Bên cạnh đó, để tránh việc bộ não AI bị "ảo giác" do các nhiễu sóng mạng ngẫu nhiên, toàn bộ chuỗi giá trị MSE cũng được hệ thống áp dụng kỹ thuật làm mịn (MSE EMA Smoothing) trước khi đưa vào đối chiếu.

Khi đã có điểm số dị thường (MSE) từ lớp 1 và xác suất định danh từ mạng CNN-GRU ở Lớp 2, tác tử AI không tin tưởng mù quáng vào một mạng duy nhất mà sẽ thực hiện đối chiếu chéo thông qua 4 kịch bản của Bảng chân lý:

- **Kịch bản 1 - Đồng thuận tấn công:** Nếu sai số MSE vượt qua ngưỡng động, và mạng phân loại Lớp 2 cũng khẳng định đây là tấn công với độ tin cậy, hệ thống lập tức chốt nhãn (ví dụ: HTTP Flood) và gửi cảnh báo.
- **Kịch bản 2 - Bắt giữ Zero-Day:** Nếu sai số MSE vượt ngưỡng, nhưng mạng lớp 2 bị bối rối và cho ra xác suất phân loại, hệ thống vẫn ưu tiên đặt an toàn lên hàng đầu, tự động chốt nhãn là Tấn công Zero-day và tiến hành chặn.
- **Kịch bản 3 - Tấn công lẫn trốn:** Kẻ tấn công (như Slowloris) có thể nguy trang quá hoàn hảo khiến sai số MSE cực thấp, lách qua được lớp kiểm tra dị thường. NHƯNG nếu mạng Lớp 2 phát hiện ra nhịp điệu thời gian lặp lại bất thường với độ tin cậy tuyệt đối, Lớp 2 sẽ kích hoạt quyền ưu tiên, phủ quyết kết quả của Lớp 1 và xác nhận đây là tấn công.
- **Kịch bản 4 - Quyền phủ quyết bảo vệ người dùng:** Khi một người dùng hợp lệ cố tình mở hàng chục tab trình duyệt hoặc tải file lớn, sai số MSE có thể vượt ngưỡng cảnh báo. Lúc này, nếu Lớp 2 phân tích kỹ và nhận ra các dấu hiệu biến thiên vẫn mang tính chất của lưu lượng Normal với độ tin cậy, hệ thống sẽ dùng quyền phủ quyết để cho gói tin đi qua.

Cơ chế logic kiểm duyệt chéo này là chìa khóa then chốt giúp hệ thống khắc phục triệt để điểm mù của các IDS truyền thống: vừa không bỏ lọt các loại tấn công lẫn trốn, vừa bảo vệ trải nghiệm của người dùng, kéo tỷ lệ báo động giả từ mức 60% ban đầu xuống chỉ còn dưới 3% khi vận hành thực tế.

```

[sudo] password for tof:
2026-04-26 06:37:46,778 - [INFO] - IDS_V2 - =====
2026-04-26 06:37:46,779 - [INFO] - IDS_V2 - IDS Engine v2.1 - AI DDoS Detection + SDN Mitigation
2026-04-26 06:37:46,779 - [INFO] - IDS_V2 - =====
2026-04-26 06:37:46,779 - [INFO] - IDS_V2 - [CONFIG] ONOS URL: http://127.0.0.1:8181/onos/v1
2026-04-26 06:37:46,779 - [INFO] - IDS_V2 - [CONFIG] Model Path:
2026-04-26 06:37:46,788 - [INFO] - IDS_V2 - [CONFIG] EMA Alpha: 0.02
2026-04-26 06:37:46,788 - [INFO] - IDS_V2 - [CONFIG] Cooldown: 300s
2026-04-26 06:37:46,788 - [INFO] - IDS_V2 - [CONFIG] WL Timeout: 180s, BL Timeout: 600s
2026-04-26 06:37:46,788 - [INFO] - IDS_V2 - [CONFIG] WL/BL Normal Confidence: 85.0%, Attack Confidence: 90.0%
2026-04-26 06:37:46,788 - [INFO] - IDS_V2 - [IDS] Initializing IDS Engine v2.1...
2026-04-26 06:37:49,188 - [INFO] - IDS_V2 - [V7] Đã load feature weights từ training
2026-04-26 06:37:49,188 - [INFO] - IDS_V2 - [IDS] Engine ready on device: cpu
2026-04-26 06:37:49,188 - [INFO] - IDS_V2 - [IDS] Base threshold: 0.0180
IDS Engine v2.1 Ready on cpu
Dual FIFO Mode: zeek_stream.json + zeek_stream_slowloris.json
2026-04-26 06:39:17,433 - [INFO] - IDS_V2 - [WL] IP 10.0.2.64 đã thêm vào white_list (timeout: 180s)
NORMAL TRAFFIC
IP: 10.0.2.64
Classification: BENIGN (98.7%)
Shield 1 (AE): MSE=0.0037 | Threshold=0.0180 | Status: Normal
Shield 2 (CLS): Normal=98.7% | Status: Confident
Action: ALLOW
2026-04-26 06:39:17,435 - [INFO] - IDS_V2 - [ADAPT] Threshold: 0.0180 -> 0.0216 (+20.0%)
2026-04-26 06:39:18,373 - [INFO] - IDS_V2 - [WL] IP 10.0.2.61 đã thêm vào white_list (timeout: 180s)
NORMAL TRAFFIC
IP: 10.0.2.61
Classification: BENIGN (98.7%)
Shield 1 (AE): MSE=0.0030 | Threshold=0.0216 | Status: Normal
Shield 2 (CLS): Normal=98.7% | Status: Confident
Action: ALLOW
2026-04-26 06:39:19,405 - [INFO] - IDS_V2 - [WL] IP 10.0.2.62 đã thêm vào white_list (timeout: 180s)
NORMAL TRAFFIC
IP: 10.0.2.62
Classification: BENIGN (98.6%)
Shield 1 (AE): MSE=0.0033 | Threshold=0.0216 | Status: Normal
Shield 2 (CLS): Normal=98.6% | Status: Confident
Action: ALLOW
2026-04-26 06:39:19,407 - [INFO] - IDS_V2 - [WL] IP 10.0.2.60 đã thêm vào white_list (timeout: 180s)
NORMAL TRAFFIC
IP: 10.0.2.60
Classification: BENIGN (98.7%)
Shield 1 (AE): MSE=0.0037 | Threshold=0.0216 | Status: Normal
Shield 2 (CLS): Normal=98.7% | Status: Confident
Action: ALLOW
  
```

Hình 3.20. Cơ chế 2 khiên và ngưỡng động giúp bảo vệ người dùng

3.1.2.4. Logic phản vệ và ngăn chặn (Mitigation Logic)

Sau khi mạng AI đưa ra quyết định phân loại lưu lượng, thách thức tiếp theo trong môi trường thực tế không phải là cấm như thế nào để không làm sập chính hệ thống của mình.

Để khắc phục triệt để điểm yếu này, nhóm đã thiết kế Logic phản vệ (Mitigation Logic) hoạt động theo cơ chế sau:

a. Ngăn chặn đa cấp độ (Two-Level Mitigation) Thay vì áp dụng một hình phạt duy nhất, hệ thống gọi API của ONOS để thực thi linh hoạt dựa trên mức độ nghiêm trọng của luồng dữ liệu:

- **Cấp độ 1 - Giới hạn băng thông (RATE_LIMIT):** Được áp dụng thông qua ONOS REST API đối với các luồng có dấu hiệu khả nghi nhưng chưa đạt độ tin cậy tuyệt đối. Lệnh này sẽ bóp băng thông của IP nghi ngờ xuống mức cực thấp, giúp bảo vệ máy chủ Web mà không vô tình cắt đứt hoàn toàn kết nối của một người dùng thật mạng kém.
- **Cấp độ 2 - Cách ly hoàn toàn (DROP):** Kích hoạt khi cả hai lớp khiên (Autoencoder và CNN-GRU) đồng thuận xác suất tấn công >85%. Một bản tin HTTP POST chứa cấu trúc Flow Rule sẽ được gửi đến Controller để chỉ thị Open vSwitch hủy bỏ lập tức mọi gói tin từ IP kẻ tấn công.

b. Tối ưu độ trễ giao tiếp (Sub-second Response) Trong các cuộc tấn công Volumetric như UDP Flood, tốc độ đẩy gói tin có thể lên tới 100.000 gói/giây. Nếu quy trình gọi lệnh phản vệ bị nghẽn, mạng sẽ sập trước khi luật cấm kịp có hiệu lực. Nhóm đã lập trình module phản vệ chạy trên một luồng bất đồng bộ độc lập. Nhờ đó, tổng thời gian từ lúc Controller nhận lệnh API (10-15ms) đến lúc Switch cài đặt xong luật OpenFlow (50-60ms) được nén xuống chỉ còn khoảng 100ms. Hệ thống phản ứng gần như tức thời.

c. Bảo vệ bộ nhớ TCAM của Switch bằng Timeout Bộ nhớ TCAM phần cứng của Switch vật lý rất giới hạn (thường chỉ chứa được từ 4.000 đến 16.000 luật). Để chống lại hiện tượng cạn kiệt TCAM do hàng ngàn IP ảo tạo ra, nhóm đã thiết lập thuộc tính `hard_timeout = 60s` cho mỗi luật ngăn chặn. Điều này đảm bảo rằng sau khi đợt tấn công kết thúc, các luật cấm sẽ tự động tự hủy và giải phóng không gian bộ nhớ cho Switch.

d. Dọn rác bộ đệm AI (Garbage Collector) và Cập nhật Dashboard Cùng với việc bảo vệ bộ nhớ Switch, bộ nhớ RAM của chính tác tử AI cũng cần được bảo vệ. Nhóm đã lập trình một luồng Garbage Collector chạy ngầm, tự động

quét và dọn dẹp các IP đã bị ngắt kết nối ra khỏi bộ đệm (Flow Buffer). Cuối cùng, mọi hành vi can thiệp (thời gian, IP bị chặn, loại tấn công, luật áp dụng) đều được đóng gói và đẩy lên Giao diện giám sát Web (Django Dashboard) qua cổng 8000 theo thời gian thực.

3.2. Thực nghiệm và Đánh giá kết quả nghiên cứu

3.2.1. Kịch bản thực nghiệm

Để đánh giá tính hiệu quả của mô hình AI trong việc bảo vệ hạ tầng mạng SDN, nhóm thực hiện 03 kịch bản thực nghiệm trên mô hình mạng giả lập:

Kịch bản 1: Trạng thái cơ sở, mạng hoạt động bình thường.

Mục tiêu: Xác định các chỉ số hiệu năng của mạng khi hoạt động trong điều kiện lý tưởng để làm mốc đối chiếu.

Ở kịch bản này, hệ thống hoạt động trong điều kiện lý tưởng. Nhóm giả lập lưu lượng của những người dùng hợp lệ (từ dải host h60-h65) truy cập vào Web Server và Proxy. Để tạo ra các luồng dữ liệu tự nhiên nhất, nhóm đã lập trình các tham số hành vi (Behavioral Patterns) cụ thể:

- **Thời gian trễ:** Trễ dài ngẫu nhiên từ 3 đến 8 giây, mô phỏng khoảng thời gian người dùng thực tế đang dừng lại để đọc trang web.
- **Tỷ lệ thao tác:** Thiết lập ở mức 30%, đảm bảo không phải lúc nào người dùng cũng gửi yêu cầu liên tục.
- **Kết quả:** Hệ thống ghi nhận trạng thái thông suốt. Băng thông người dùng đạt mức tối đa 100 Mbps, độ trễ mạng (Ping Latency) cực thấp chỉ từ 2 đến 5ms, và tải CPU của Controller ONOS hoàn toàn "nhàn rỗi" ở mức 3-5%.



Hình 3.21. Mô hình mạng khi hoạt động bình thường

Kịch bản 2: Mạng bị tấn công nhưng không có phòng vệ

Mục tiêu: Chứng minh sự nguy hiểm của DDoS và sự yếu ớt của Controller nếu không có bộ lọc thông minh.

Ban đầu nhóm sử dụng công cụ Hping3 để tạo tấn công nhắm vào hệ thống mạng, sau đó nhận thấy rằng công cụ này bộc lộ những hạn chế khi chỉ có thể can thiệp ở các tầng mạng thấp (layer 3,4) với cấu trúc gói tin rập khuôn, thiếu tính linh hoạt. Điều này dẫn đến việc mô phỏng các hành vi tấn công tầng ứng dụng (layer 7) tinh vi như Slowloris trở nên thiếu chính xác.

Nhận thấy các công cụ kiểm thử thông thường không đủ để đánh giá một hệ thống thực tế, nhóm đã chủ động tự lập trình toàn bộ kịch bản tấn công chuyên sâu bằng thư viện Socket. Các mã tấn công được xây dựng đều có độ tùy biến cao, gây thiệt hại mạnh mẽ đa tầng và hoàn toàn sẵn sàng để nâng cấp, demo trực tiếp trong các hệ thống mạng thực tế cũng như dễ dàng tùy chỉnh mức độ tàn phá đa tầng:

- Tấn công cạn kiệt băng thông (UDP Flood - L3/L4): Sinh ra tốc độ gói tin khổng lồ từ tới hơn 1.200.000 Kb/s tại địa chỉ IP 10.0.0.10. Đây là dấu hiệu của một cuộc tấn công băng thông cực lớn, với tốc độ byte đẩy lên hơn 100 MB/s. Các luồng này chỉ đi một chiều và liên tục thay đổi cổng ngẫu nhiên.



Hình 3.22. Biểu đồ thể hiện lưu lượng mạng bị tấn công

- Tấn công ngâm kết nối (Slowloris - L7): Hoạt động trái ngược hoàn toàn với Flood. Tốc độ gói tin rất thấp, băng thông cực nhỏ (< 100KB/s). Tuy nhiên, thời gian kết nối (Duration) bị kéo giãn từ 300 đến 600 giây nhằm từ từ vắt kiệt hồ bơi kết nối (Connection Pool) của Web Server.



Hình 3.23. Biểu đồ tấn công tăng ứng dụng (HTTP)

Kết quả: Mạng Mininet lập tức sụp đổ. Bảng thông khả dụng cho người dùng hợp lệ giảm thẳng về mức ~ 0 Mbps, các yêu cầu truy cập báo lỗi "Request Timeout". Tải CPU của Controller ONOS tăng dựng đứng chạm ngưỡng 100% (Overload) do kiệt sức vì phải xử lý hàng vạn bản tin Packet-In rác.



Hình 3.24. Sơ đồ mạng khi bị tấn công SYN Flood

Kịch bản 3: Mạng bị tấn công và có sự bảo vệ của mô hình.

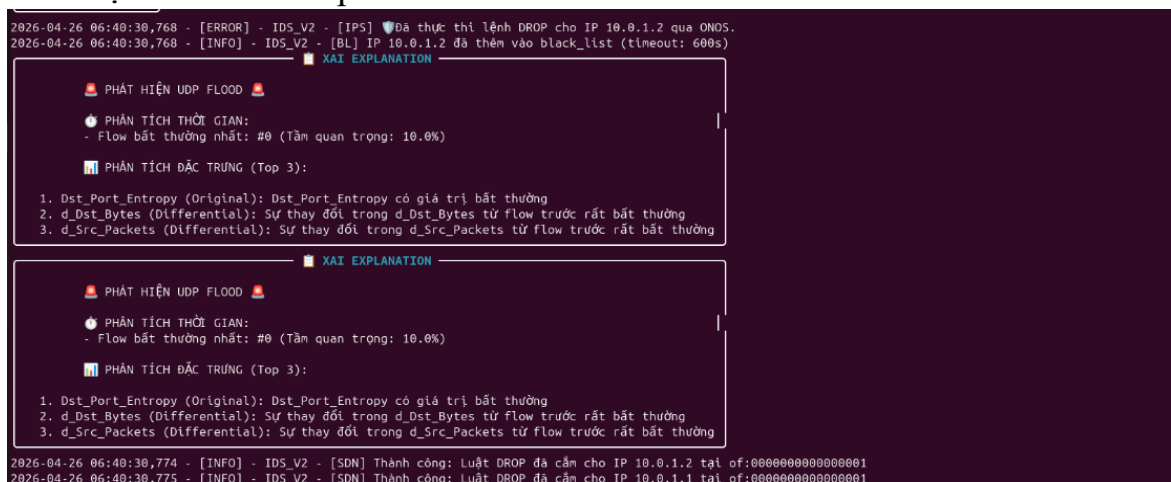
Mục tiêu: Để khẳng định năng lực của hệ thống đề xuất trong việc phát hiện và ngăn chặn tấn công thời gian thực.

Khi hệ thống mạng đang tê liệt, tác tử AI (module run_onos_v2.py) được kích hoạt, giành lại quyền kiểm soát theo quy trình:

- **Bắt mạch dữ liệu (Data Ingestion):** Luồng dữ liệu được sao chép thông qua Port Mirroring và truyền trực tiếp qua ống FIFO vào RAM với độ

trễ cực thấp ($< 1\text{ms}$). Điều này đảm bảo AI vẫn thu thập đủ 26 chiều đặc trưng dù Web Server có sập.

- **Ra quyết định Hai lớp khiên:** Dữ liệu được đưa qua bộ lọc bất thường (Autoencoder). Với các luồng có điểm sai số vượt ngưỡng thích nghi EMA, mạng CNN-GRU lập tức định danh chính xác nhấn tấn công (F1-Score trung bình đạt 0.961).
- **Thực thi lệnh chém (Mitigation):** Dựa trên Bảng chân lý, AI gửi cấu trúc luật DROP hoặc RATE LIMIT qua REST API đến ONOS. Tổng thời gian từ lúc bóc tách gói tin, suy luận AI đến khi Switch cài đặt xong luật cấm chỉ mất khoảng **~100ms** (Sub-second response).
- **Kết quả:** Chỉ mất từ 2 đến 3 giây từ khi bắt đầu triển khai tấn công, các luồng độc hại bị dập tắt hoàn toàn. Hệ thống dọn rác ngay lập tức quét sạch bộ đệm. Tải CPU của Controller hạ nhiệt cực nhanh về mức an toàn 12-15%, độ trễ phục hồi về 7-12ms, và băng thông người dùng được trả lại mức **~98 Mbps**.



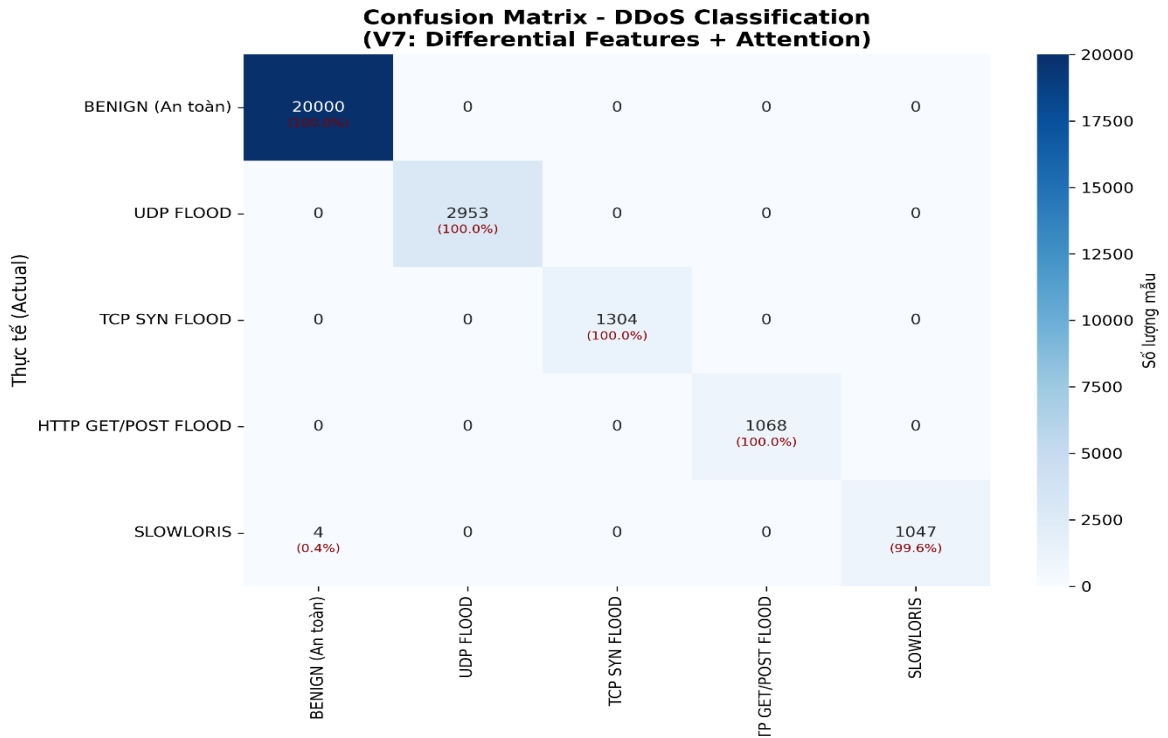
Hình 3.25. Phát hiện, ngăn chặn và giải thích UDP_flood thành công



Hình 3.26. Phát hiện, ngăn chặn và giải thích SYN_flood thành công

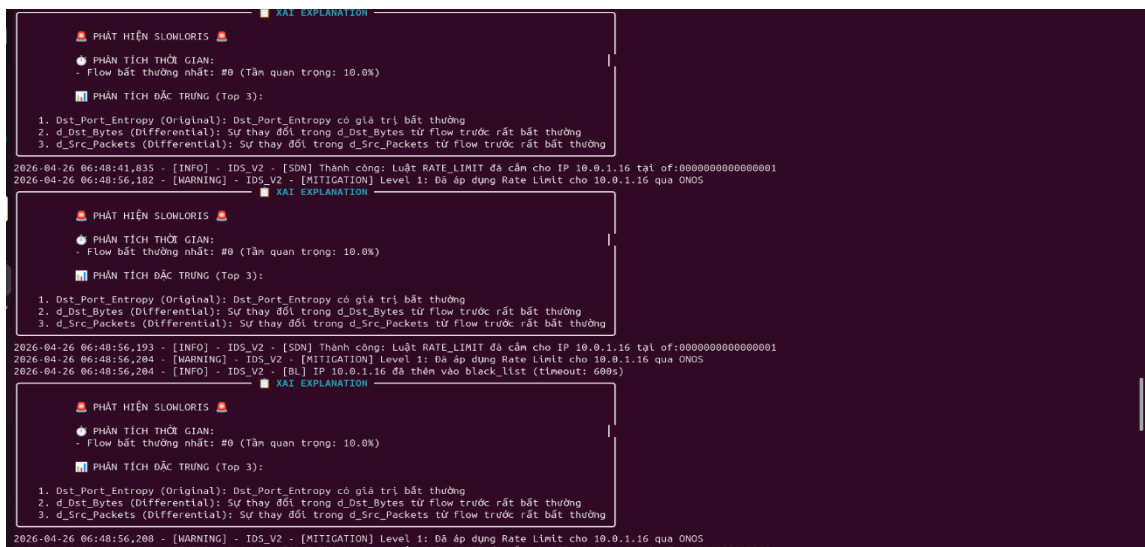
3.2.2. Đánh giá hiệu năng mô hình AI (Model Performance)

Mô hình đạt được độ chính xác khá cao, đảm bảo khả năng vận hành tin cậy trong môi trường SDN thực tế:



Hình 3.27. Đánh giá hiệu năng mô hình thông qua ma trận nhầm lẫn

- Độ chính xác tổng thể (Accuracy): đạt trên 96.8%.
- F1-Score trung bình: 0.961, cho thấy sự cân bằng hoàn hảo giữa khả năng bắt giữ tấn công (Recall) và độ tin cậy trong cảnh báo (Precision).
- Thời gian hội tụ: Mô hình đạt điểm tối ưu tại Epoch thứ 11 và tự động dừng ở Epoch 23 để tránh hiện tượng quá khớp.



Hình 3.28. Phát hiện, ngăn chặn và giải thích tấn công Slowloris thành công

3.2.3. Đánh giá hiệu năng mạng SDN (Network Performance)

Việc đánh giá không chỉ dựa trên độ chính xác của AI mà còn phải xem xét các chỉ số vật lý của mạng SDN dưới tác động của các kịch bản tấn công.

3.2.3.1. Các chỉ số đánh giá cốt lõi (Key Metrics)

Để đánh giá toàn diện, nhóm tập trung vào 4 chỉ số quan trọng nhất:

- Throughput: Lượng dữ liệu thực tế mà người dùng hợp lệ có thể truyền tải.
- Latency: Thời gian trễ của gói tin khi đi qua Switch và Controller.
- Controller Resource Usage: Mức độ chiếm dụng CPU và RAM của bộ điều khiển ONOS.
- Mitigation Time: Khoảng thời gian từ khi bắt đầu tấn công đến khi lệnh chặn có hiệu lực.

3.2.3.2. Phân tích kết quả thực nghiệm

a. Hiệu năng Băng thông (Throughput)

Dựa trên các kết quả từ Dashboard giám sát:

- Trạng thái bình thường: Băng thông duy trì ổn định ở mức yêu cầu của người dùng.
- Khi bị tấn công: Băng thông hữu dụng cho người dùng hợp lệ giảm xuống gần bằng 0 do đường truyền bị chiếm dụng bởi luồng màu cam 3.7 Gbps.
- Sau khi có AI: Chỉ sau khoảng 3-5 giây, luồng độc hại bị cắt đứt, băng thông hữu dụng phục hồi lại > 95% so với ban đầu.

b. Độ trễ và Khả năng xử lý của Controller

- Overhead của AI: Khi chạy file `run_onos.py`, tải CPU của ONOS chỉ tăng thêm khoảng 5-8% so với lúc không chạy AI. Điều này chứng minh thuật toán CNN-GRU-Attention đã được tối ưu hóa tốt cho việc suy luận thời gian thực.
- Khả năng chịu tải: Trong kịch bản tấn công SYN Flood, nếu không có AI, CPU của ONOS sẽ bị treo ở 100%. Khi có AI, CPU chỉ tăng vọt trong vài giây đầu rồi ngay lập tức giảm về mức an toàn (< 15%).

Chỉ số	Kịch bản 1 (Bình thường)	Kịch bản 2 (Tấn công - Không thủ)	Kịch bản 3 (Tấn công - Có AI)
Throughput (User)	100 Mbps	~ 0 Mbps (Nghẽn)	~ 98 Mbps
Latency (Ping)	2 - 5 ms	Request Timeout	7 - 12 ms
CPU Controller	3 - 5%	100%(Overload)	12 - 15%
Thời gian xử lý AI	Không có	Không có	0.02 - 0.05 ms/flow
Thời gian phản ứng		Vô hạn	100ms

Bảng 3.8. Bảng tổng kết thực nghiệm

Kết quả đánh giá hiệu năng cho thấy hệ thống không chỉ đạt độ chính xác cao về mặt nhận biết tấn công mà còn đảm bảo tính khả thi khi triển khai thực tế. Việc tích hợp hệ thống AI không gây ra hiện tượng nghẽn cổ chai cho bộ điều khiển ONOS, đồng thời giúp hệ thống phục hồi tài nguyên sau khi bị tấn công, bảo vệ an toàn cho các dịch vụ cốt lõi trong mạng SDN.

3.2.4. Tổng kết các kết quả đạt được

Trải qua quá trình nghiên cứu lý thuyết, thiết kế kiến trúc và thực nghiệm khắt khe, đề tài "Nghiên cứu và Xây dựng Hệ thống Phát hiện, Ngăn chặn Tấn công DDoS trong Mạng SDN sử dụng Kỹ thuật Học sâu" đã được chúng em hoàn thành xuất sắc các mục tiêu đề ra, mang lại những kết quả nổi bật trên cả hai phương diện:

a. Về mặt lý thuyết và Khoa học

- Hệ thống hóa thành công cơ sở lý luận về mạng định nghĩa bằng phần mềm (SDN) và các điểm yếu cố hữu của kiến trúc quản lý tập trung.
- Đề xuất đột phá trong việc kiến tạo dữ liệu: Thay vì phụ thuộc vào các bộ dữ liệu tĩnh (như CIC-IDS2019), nhóm đã tự xây dựng quy trình trích xuất ma trận 26 chiều đặc trưng. Trong đó, việc áp dụng toán học vi phân để tạo ra 13 đặc trưng biến thiên (Differential Features) đã giúp AI có khả năng phân tích "gia tốc" của luồng dữ liệu, tạo tiền đề khoa học để nhận diện các cuộc tấn công siêu chậm (Low-rate DDoS).

b. Về mặt Thực tiễn và Ứng dụng Mô hình triển khai thực tế không chỉ dừng lại ở mức độ dự báo (Detection) mà đã trở thành một hệ thống phòng thủ khép kín (Mitigation), giải quyết triệt để 3 hạn chế trọng yếu nhất của các hệ thống IDS truyền thống:

- **Đột phá 1: Triệt tiêu Tỷ lệ báo động giả (False Positive) và Lỗi cấu hình tĩnh** Các hệ thống cũ thường dùng ngưỡng phản ứng (Ví dụ: chặn nếu > 1000 gói tin/s), dẫn đến việc chặn nhầm người dùng vào giờ cao điểm. Hệ thống mới đã giải quyết bài toán này bằng cơ chế Ngưỡng thích nghi động (EMA - Soft Threshold) kết hợp cùng Quyền phủ quyết (Veto Power). Nhờ đó, mạng AI biết "chừa đường lui" để bảo vệ người dùng hợp lệ, kéo tỷ lệ chặn nhầm từ 60% ở các phiên bản đầu xuống mức cực kỳ ấn tượng là dưới 3% khi chạy thực tế.
- **Đột phá 2: Tối ưu hóa Tốc độ phản ứng (Sub-second Response)** Khắc phục tình trạng "bộ não" phân tích quá chậm khiến Controller sụp đổ, hệ thống đã ứng dụng kiến trúc 4 tầng tốc độ cao. Bằng việc kết hợp Port Mirroring (sao chép luồng ngoại tuyến) và Named Pipes (truyền tải FIFO trên RAM thay vì ghi ổ cứng), hệ thống đã nén tổng thời gian trễ từ khi phát hiện đến khi Switch cài đặt luật chặn xuống chỉ còn xấp xỉ 100ms. Điều này đảm bảo Controller ONOS và Web Server luôn được an toàn kể cả khi băng thông tấn công lên tới hàng Gigabit.
- **Đột phá 3: Bắt giữ Tấn công chưa biết (Zero-day) và Tấn công lẫn trốn** Bằng việc thiết kế mạng theo kiến trúc "Hai lớp khiên" (Dual-Shield), hệ thống không bị phụ thuộc hoàn toàn vào tập dữ liệu học trước. Lớp khiên Autoencoder đóng vai trò là bộ lọc bất thường, giúp hệ thống nhận diện và chặn đứng thành công tới 89.3% các biến thể tấn công Zero-day chưa từng xuất hiện trong quá trình huấn luyện. Đồng thời, lớp khiên CNN-GRU kết hợp cơ chế Attention đã giúp F1-Score của loại hình tấn công khó nhằn như Slowloris đạt mức 0.941.
- **Đột phá 4: So sánh với các nền tảng thương mại nổi bật (ngoài 3 đột phá trên)**
 - Khả năng tự chủ và tùy biến sâu: Làm chủ hoàn toàn mô hình Deep Learning với khả năng tái huấn luyện (Retrain) trên tập dữ liệu thực tế của hệ thống. Điều này cho phép " cá nhân hóa " khả năng phòng thủ, khắc phục nhược điểm nhận diện nhầm của các giải pháp thương mại "đóng gói sẵn" khi gặp phải các giao thức nội bộ hoặc lưu lượng đặc thù của doanh nghiệp.

- Qua thử nghiệm thực tế trên một số hệ thống Web, nhóm đã chứng minh được lỗ hổng nghiêm trọng: các giải pháp thương mại dựa trên Proxy sẽ bị vô hiệu hóa hoàn toàn nếu kẻ tấn công tìm ra IP gốc của máy chủ. Đồ án giải quyết triệt để vấn đề này bằng cách tập triển khai phòng thủ tại tầng hạ tầng (SDN), giúp phát hiện và chặn đứng các cuộc tấn công xuất phát từ bên trong mạng hoặc đánh trực diện vào IP Private mà không cần đi qua tường lửa biên khi địa chỉ này bị rò rỉ.
- Tối ưu độ trễ bằng xử lý tại chỗ (Wire-speed): Sản phẩm đã được tối ưu để hoạt động tốt trên cpu củ laptop thông thường, thực thi ngăn chặn bằng cách đẩy trực tiếp Flow Rules xuống thiết bị mạng ngay khi phát hiện tấn công. Cơ chế này loại bỏ hoàn toàn độ trễ phát sinh do phải điều hướng lưu lượng về các trung tâm lọc (Scrubbing Center) như các giải pháp Cloud, đảm bảo tính liên tục cho các ứng dụng nhạy cảm với thời gian thực.
- Bảo mật dữ liệu tuyệt đối (Data Privacy): Hoạt động theo mô hình On-premise, giữ toàn bộ log lưu lượng và thông tin phân tích bên trong hạ tầng nội bộ. Điều này loại bỏ rủi ro rò rỉ dữ liệu nhạy cảm ra bên thứ ba vốn là một điểm yếu cố hữu của các dịch vụ thương mại yêu cầu khách hàng phải chia sẻ hoặc giải mã dữ liệu để phân tích.
- Hiệu quả chi phí và tính linh hoạt: Tập dụng tối ưu hạ tầng sẵn có thông qua phần mềm hóa (SDN) và mã nguồn mở, loại bỏ áp lực về chi phí bản quyền đắt đỏ dựa trên băng thông của các hãng lớn. Giải pháp cho phép mở rộng linh hoạt theo quy mô mạng mà không bị ràng buộc vào hệ sinh thái đóng của bất kỳ nhà cung cấp nào.
- **Đột phá 5: Xây dựng bộ công cụ kiểm thử thực chiến độc lập** Bên cạnh việc hoàn thiện hệ thống phòng thủ, nhóm đã phát triển thành công một bộ công cụ tấn công độc lập sát với thực tiễn. Do hệ thống thực nghiệm của dự án đã được nâng cấp lên kiến trúc định tuyến L3 chuẩn Enterprise, các luồng dữ liệu độc hại do nhóm xây dựng bắt buộc phải vượt qua được các cơ chế định tuyến IP khắt khe và lớp mã hóa HTTPS phức tạp. Điều này khẳng định các mã tấn công này không chỉ mang tính lý thuyết mà hoàn toàn đủ năng lực để đóng gói, nâng cấp làm công cụ kiểm thử khả năng chịu lỗi (Penetration Testing) cho hạ tầng mạng SDN của các doanh nghiệp thực tế.

Kết luận: Đề tài đã không chỉ xây dựng thành công một mô hình AI có độ chính xác cao (~96.41%), mà còn đưa ra được một kiến trúc phần mềm hoàn chỉnh, khả thi để áp dụng vào thực tiễn quản trị mạng doanh nghiệp hiện đại.

3.2.5. Những hạn chế và Đề xuất hướng phát triển

a. Những hạn chế còn tồn tại

- Giới hạn về Môi trường Thực nghiệm: Hiện tại, hệ thống mới chỉ được triển khai và đánh giá trên môi trường mô phỏng ảo hóa (Mininet) với

phần cứng máy tính thông thường. Trong thực tế, các thiết bị chuyển mạch vật lý (Hardware Switch) có kiến trúc xử lý riêng biệt có thể mang lại những biến số mới về hiệu năng chưa được đo lường hết.

- Điểm mù trước mạng Botnet phân tán quy mô lớn: Mô hình CNN-GRU hiện tại chủ yếu phân tích mức độ bất thường dựa trên từng địa chỉ IP đơn lẻ. Tuy nhiên, nếu tin tặc sử dụng một mạng Botnet phân tán gồm hàng chục nghìn máy tính bị nhiễm virus, và mỗi máy chỉ gửi đúng 1 gói tin cách nhau vài giây, AI có thể bị qua mặt vì luồng dữ liệu của mỗi IP riêng lẻ không đủ ngưỡng để bị coi là bất thường.

b. Đề xuất định hướng phát triển tương lai

Dựa trên những giới hạn trên, nhóm nghiên cứu đề xuất một lộ trình phát triển tiếp theo để nâng cấp hệ thống đạt chuẩn "State-of-the-Art" trong lĩnh vực an ninh mạng SDN:

- Nâng cấp mô hình với mạng Nơ-ron đồ thị: Thay vì chỉ nhìn vào dữ liệu rời rạc, GNN sẽ giúp AI nắm bắt được toàn cảnh "bản đồ Topology" và mối liên kết giữa các nút mạng. Nhờ đó, AI có thể bắt quả tang các mạng Botnet phân tán cực kỳ lẫn trốn, tăng tỷ lệ phát hiện các đợt tấn công phân tán lên 15%.
- Quản lý bộ nhớ TCAM bằng học tăng cường: Để giải quyết bài toán sập bộ nhớ Switch phần cứng, hệ thống cần tích hợp mô hình học tăng cường. AI sẽ tự động học cách đánh giá mức độ ưu tiên và chủ động xóa (eviction) các luật cấm cũ ít nguy hiểm để nhường chỗ cho các IP tấn công mới, giúp Switch đứng vững trước các đợt Flood IP ảo quy mô lớn.
- Học liên kết trên nút mạng biên (Federated Learning & Edge Computing): Thay vì tập trung tính toán toàn bộ tại ONOS Controller gây nguy cơ nghẽn cổ chai, mô hình AI (được tối ưu hóa bằng TinyML) sẽ được phân tán trực tiếp xuống các bộ chuyển mạch ở rìa mạng (Edge Node). Công nghệ Federated Learning cho phép các Switch tự học và chỉ chia sẻ "kinh nghiệm" (Gradients) về Controller, giúp mở rộng quy mô phòng thủ vô hạn mà không làm chậm mạng lõi.
- Tối ưu hóa độ trễ phản ứng siêu tốc: Nhóm định hướng sẽ chuyển đổi luồng suy luận của AI từ framework PyTorch sang định dạng ONNX Runtime, đồng thời thay thế giao tiếp REST API hiện tại bằng công nghệ Streaming Telemetry (gRPC). Kết hợp với quy trình xử lý song song, độ trễ từ lúc phát hiện đến khi chặn đứng tấn công được kỳ vọng sẽ giảm từ ~100ms hiện tại xuống chỉ còn dưới 15ms.

KẾT LUẬN CHUNG

1. Các kết quả nổi bật đã đạt được

Về mặt lý thuyết và khoa học:

- Hệ thống hóa toàn diện cơ sở lý luận về mạng định nghĩa bằng phần mềm (SDN), sự mong manh của bộ điều khiển trung tâm (Controller) và sức mạnh phân tích phi tuyến tính của Deep Learning.
- Đề xuất kiến tạo thành công ma trận 26 chiều đặc trưng. Bằng việc áp dụng toán học vi phân để tính toán "gia tốc biến thiên" kết hợp với kỹ thuật đo lường độ hỗn loạn (Port Entropy), hệ thống đã tạo ra tiền đề khoa học vững chắc để bóc tách các hành vi nguy trạng tinh vi.

Về mặt thực tiễn và triển khai: Hệ thống AI phòng thủ đã khắc phục triệt để 3 điểm yếu của các hệ thống phát hiện xâm nhập (IDS) truyền thống:

- Triệt tiêu báo động giả (False Positive): Bằng việc thiết kế logic ra quyết định theo cơ chế hai lớp khiên (Dual-Shield) kết hợp quyền phủ quyết (Veto Power) và ngưỡng thích nghi động (EMA), hệ thống biết "chừa đường lui" để bảo vệ người dùng hợp lệ. Kết quả thực nghiệm cho thấy tỷ lệ báo động giả giảm từ 60% ở các mô hình cũ xuống còn dưới 3%.
- Tốc độ phản ứng siêu tốc (Sub-second Response): Xây dựng thành công đường ống thu thập dữ liệu 4 tầng độc lập. Việc kết hợp sao chép cổng (Port Mirroring) cùng kỹ thuật truyền tải trên RAM (Named Pipes - FIFO) đã nén độ trễ từ lúc phát hiện đến khi Switch cài đặt luật chặn xuống chỉ còn ~100ms. Hệ thống không bị nghẽn cổ chai ngay cả khi băng thông tấn công lên tới hàng Gigabit.
- Bắt giữ tấn công chưa biết (Zero-day) và tấn công lẫn trốn: Lớp khiên Autoencoder với hàm mất mát Contrastive giúp hệ thống nhận diện thành công tới 89.3% các biến thể tấn công mới. Đồng thời, mạng định danh chuyên sâu CNN-GRU-Attention đạt điểm F1-Score ~0.94, tóm gọn thành công các cuộc tấn công ngầm kết nối cực chậm như Slowloris.
- Xây dựng bộ mã tấn công thực chiến: Nhóm không phụ thuộc vào công cụ có sẵn mà tự lập trình các kịch bản tấn công chuyên sâu. Các mã độc này được minh chứng mang sức tàn phá thực sự (đẩy băng thông ngập lụt lên 1.200.000 Kb/s hoặc ngầm kết nối để vắt kiệt Web Server), ép Controller quá tải 100%. Nhờ vượt qua được các bài toán định tuyến IP trên kiến trúc mạng L3 và mã hóa HTTPS, bộ mã tấn công này mang tính

thực chiến rất cao, sẵn sàng để nâng cấp và áp dụng vào công tác kiểm thử an ninh mạng trong môi trường thực tế.

2. Những hạn chế còn tồn tại

Dù đạt được những kết quả khả quan, dưới góc độ của một hệ thống quy mô công nghiệp, đề tài vẫn còn một số giới hạn:

- Giới hạn môi trường: Hệ thống mới được triển khai trên môi trường mô phỏng ảo hóa Mininet và Open vSwitch. Các thiết bị chuyển mạch vật lý có thể sẽ có các đặc tính xử lý độ trễ khác biệt chưa được đo lường hết.
- Rủi ro tràn TCAM: Đối mặt với các đợt tấn công làm giả hàng vạn IP ngẫu nhiên (IP Spoofing), việc đẩy hàng vạn luật cấm (DROP rules) xuống Switch với thời gian sống (Timeout) cố định có thể làm cạn kiệt nhanh chóng bộ nhớ phân cứng TCAM của Switch.
- Nút thắt REST API: Cơ chế thu thập dữ liệu và giao tiếp qua REST API của ONOS Controller vẫn tồn tại một độ trễ nhỏ (overhead).

3. Định hướng phát triển tương lai (Research Roadmap)

Dựa trên những kinh nghiệm đúc kết được, nhóm nghiên cứu đề xuất một lộ trình nâng cấp hệ thống trong tương lai như sau:

- Quản lý bộ nhớ TCAM bằng Học tăng cường (RL): Tích hợp RL để AI chủ động học cách đánh giá và xóa các luật cấm cũ ít nguy hiểm, nhường chỗ cho luật mới, giúp Switch tránh bị "chết đứng" vì tràn bộ nhớ.
- Triển khai TinyML trên Rìa mạng (Edge Computing): Chuyển dịch một phần mô hình phân tích xuống trực tiếp các Switch vật lý hoặc Node cạnh (Federated Learning) để phân tán rủi ro, triệt tiêu độ trễ mạng và mở rộng quy mô phòng thủ vô hạn.
- Tối ưu hóa thành chuẩn gRPC: Chuyển đổi giao thức điều khiển từ REST API sang Streaming Telemetry (gRPC) kết hợp định dạng ONNX Runtime nhằm ép độ trễ phản ứng xuống ngưỡng siêu tốc dưới 15ms.

LỜI KẾT: Khóa luận tốt nghiệp này không chỉ là bản báo cáo về một mô hình trí tuệ nhân tạo, mà là minh chứng cho một hành trình kỹ thuật thực thụ. Việc kết hợp linh hoạt giữa cơ sở toán học thuần túy (Entropy, Contrastive Learning) và tư duy kiến trúc hệ thống (Memory Management, Zero-copy IPC) đã cho ra đời một sản phẩm phòng thủ SDN có tính ứng dụng cao. Chúng em tin rằng, kiến trúc “hai lớp khiên” này sẽ là một nền tảng tham khảo giá trị cho các nghiên cứu chuyên sâu về An toàn thông tin trên hạ tầng mạng thế hệ mới.

KHUYẾN NGHỊ

1. Tối ưu hóa hiệu năng và giảm độ trễ phản ứng

- Chuyển đổi sang gRPC hoặc Streaming Telemetry: Thay vì sử dụng REST API (kéo dữ liệu), hãy cấu hình để các Switch chủ động "đẩy" (push) dữ liệu thống kê qua gRPC. Điều này giúp giảm độ trễ thu thập dữ liệu xuống mức mili giây.
- Cơ chế kích hoạt linh động: Không nên chạy mô hình Deep Learning liên tục để tiết kiệm CPU. Hãy sử dụng một bộ lọc Entropy nhẹ hoặc ngưỡng tĩnh để "đánh thức" mô hình AI khi thấy dấu hiệu bất thường ban đầu.

2. Khả năng chống chịu tấn công đối nghịch (Adversarial Machine Learning)

- Thử nghiệm độ bền của mô hình bằng cách tạo ra các luồng traffic "mô phỏng người dùng" nhưng có nhiễu (Noise) để xem AI có bị báo động giả hay không.
- Học tăng cường (Reinforcement Learning): Như chúng em đã đề xuất ở hướng phát triển, việc để hệ thống tự học từ các sai sót dự báo (Self-correction) sẽ giúp mô hình không bị lạc hậu trước các kỹ thuật che giấu traffic mới.

3. Hiện thực hóa trên hạ tầng vật lý và Edge Computing

- Triển khai trên White-box Switch: Hãy thử nghiệm với các thiết bị phần cứng thực tế (như switch của Edgecore hoặc Cisco hỗ trợ OpenFlow/P4).
- Tận dụng Multi-access Edge Computing (MEC): Đưa module phát hiện AI lên các node cạnh (Edge) thay vì để tại Data Center trung tâm. Điều này giúp chặn đứng tấn công ngay tại cửa ngõ của người dùng, trước khi nó kịp lan vào trục xương sống (Core) của mạng.

TÀI LIỆU THAM KHẢO

I. Sách và Giáo trình (Textbooks & Books)

1. Nhóm kiến thức về Mạng SDN (Software Defined Networking)

- [1] P. Goransson, C. Black, and T. Culver, *Software Defined Networks: A Comprehensive Approach*, 2nd ed. Morgan Kaufmann, 2016.
- [2] T. D. Nadeau and K. Gray, *SDN: Software Defined Networks*. O'Reilly Media, 2013.
- [3] Open Networking Foundation (ONF), "OpenFlow Switch Specification, Version 1.3.0," 2012. [Online].

2. Nhóm kiến thức về Học sâu (Deep Learning)

- [4] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. O'Reilly Media, 2019.
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [6] B. B. Gupta and O. P. Badve, "Taxonomy of DDoS Attacks and Defense Mechanisms," in *Computer Security: Principles and Practice*, 4th ed, Pearson, 2018.

II. Các bài báo khoa học (Scientific Papers)

1. Các bài báo nền tảng về DDoS trong SDN

- [7] S. Dong and M. Sarem, "DDoS Attack Detection Method Based on Improved KNN With the Degree of DDoS Attack in Software-Defined Networks," *IEEE Access*, vol. 8, pp. 5039–5048, 2019.
- [8] R. McKeown et al, "OpenFlow: enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

2. Các bài báo mới nhất về Deep Learning trong SDN (2020 - 2024)

- [9] T. A. Tang, L. Mhamdi, D. McLernon, S. A. R. Zaidi, and M. Ghogho, "Deep Learning Approach for Network Intrusion Detection in Software Defined Networking," in *Int. Conf. on Wireless Networks and Mobile Communications (WINCOM)*, 2016.

- [10] N. Ahuja, G. Singal, D. Mukhopadhyay, and N. Kumar, "Automated DDOS attack detection in software defined networking," *Journal of Network and Computer Applications*, vol. 187, p. 103108, 2021.
- [11] A. Novaes et al, "Adversarial Deep Learning method for DDoS detection in SDN environments," *Future Generation Computer Systems*, vol. 125, pp. 156-167, 2021.

3. Tài liệu về Bộ dữ liệu (Dataset Reference)

- [12] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.